BACKEND DOCUMENTATION

HORIZON GRID

Backend / System Architecture Documentation

Version: 0.3.0
Documentation prepared: 2026-08-24

PREPARED FOR EVALUATION

Table of Contents

Backend System Overview and Lifecycle

This chapter describes the backend as an *operational system*: which Docker containers exist and what each one does, the exact sequence of events between `docker compose up` and the first request being served, and the anatomy of a single HTTP request as it passes through authentication, routing, business logic, and the database. Provider connectors, the AI pipeline, the database schema, and security controls each have their own chapter; this one is the connective tissue a backend engineer or DevOps/SRE reader needs before touching any of those.

All facts below are drawn directly from `docker-compose.yml`, `docker-compose.prod.yml`, `backend/Dockerfile`, `backend/app/main.py`, `backend/app/core/config.py`, `backend/app/core/db.py`, `backend/app/core/runtime_config.py`, `backend/app/auth/rbac.py`, and `backend/app/api/routes/lookup.py`.

1. Docker Service Inventory

`docker-compose.yml` defines eight services: four datastores, three backend-codebase processes (the FastAPI app plus two Celery roles), and the frontend. See the Database Architecture chapter for what Postgres/Redis/Neo4j/OpenSearch are actually used for, and the Security Architecture chapter for the network-exposure rationale behind the port bindings below.

Service	Image / entrypoint	Depends on (health-gated)	Port binding (host)	Restart policy
`postgres`	`postgres:16-alpine`	--	`127.0.0.1:${HOST_PORT_POSTGRES:-5433}:5432`	`unless-stopped`
`redis`	`redis:7-alpine`	--	`127.0.0.1:${HOST_PORT_REDIS:-6379}:6379`	`unless-stopped`
`neo4j`	`neo4j:5-community` + APOC	--	`127.0.0.1:${HOST_PORT_NEO4J_HTTP:-7475}:7474`, `127.0.0.1:${HOST_PORT_NEO4J_BOLT:-7688}:7687`	`unless-stopped`
`opensearch`	`opensearchproject/opensearch:2.17.0`	--	`127.0.0.1:${HOST_PORT_OPENSEARCH:-9200}:9200`	`unless-stopped`
`backend`	built from `backend/Dockerfile` (`python:3.12-slim`)	`postgres`, `redis` (`service_healthy`)	`${HOST_PORT_BACKEND:-8000}:8000`	`unless-stopped`
`celery_worker`	same `backend/` build context	`postgres`, `redis` (`service_healthy`)	none published	`unless-stopped`
`celery_beat`	same `backend/` build context	`redis` (`service_healthy`)	none published	`unless-stopped`
`frontend`	built from `frontend/Dockerfile` (Next.js)	`backend` (`service_healthy`)	`${HOST_PORT_FRONTEND:-3000}:3000`	`unless-stopped`

Operationally relevant details:

- **All four datastore ports bind to `127.0.0.1` only**, not `0.0.0.0` -- the application itself never uses these host bindings, since `DATABASE_URL` / `REDIS_URL` / `NEO4J_URI` / `OPENSEARCH_URL` all address the internal Docker network by service name. The bindings only affect what can reach the datastore directly from the host machine.
- **`backend` and `celery_worker` wait on both Postgres and Redis health checks** (`pg_isready` / `redis-cli ping`) before Compose starts them; `celery_beat` only waits on Redis, since it schedules jobs but never queries Postgres.
- **All 8 services have `restart: unless-stopped`** (added in a later mission-critical-reliability review, v0.2.3 -- previously only `celery_worker` / `celery_beat` did, and `backend`, the datastores, and `frontend` stayed down after a crash until a human ran `docker compose up`). Confirmed live: a container OOM-killed by its own `mem_limit` now restarts automatically within seconds; a container an operator deliberately stops or kills stays down, matching Docker's own intended "operator explicitly wants this down" semantics.
- **`backend`, `celery_worker`, and `celery_beat` build from the identical `backend/` context** and ship the same image; they differ only in the `command:` each service starts with (see §2), not in the code on disk.
- `OLLAMA_BASE_URL` is set directly in the `backend` service's `environment:` block rather than via `env_file: .env` like other credentials. A compose `environment:` entry always wins over `env_file:`, so this silently overrides whatever `OLLAMA_BASE_URL` is written into `.env`.
- `docker-compose.prod.yml` (used by the Windows installer) layers on top of the base file: it strips every bind-mount (`volumes: !reset []`) from `backend`, `celery_worker`, `celery_beat`, and `frontend`, and swaps their commands for non-hot-reload equivalents.

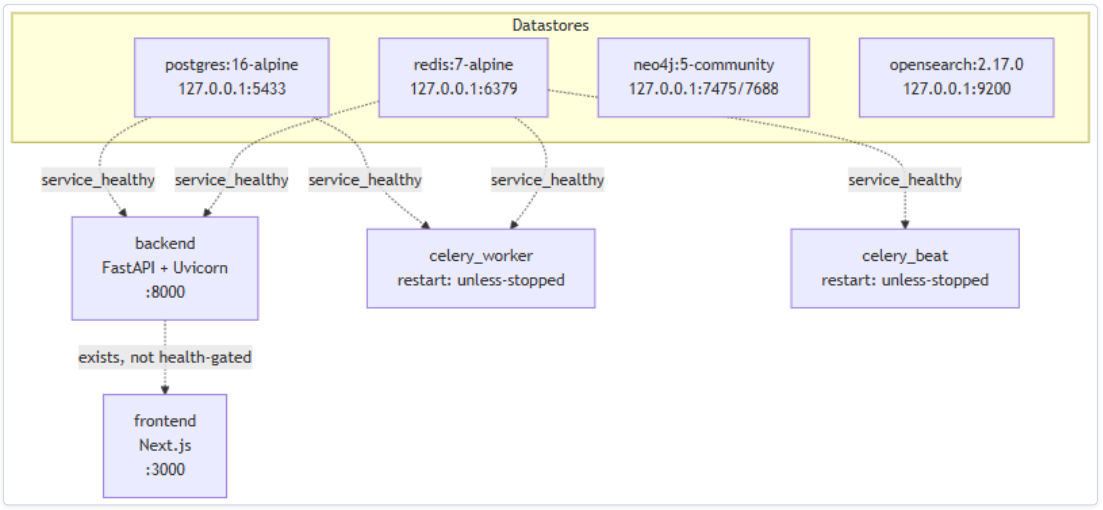

Figure 1 — Diagram: 1. Docker Service Inventory

Diagram: Docker Service Topology and Dependency Graph. Dotted arrows are Compose `depends_on` health gates, not runtime data paths -- `backend` and `celery_worker` wait on Postgres and Redis health checks, `celery_beat` waits on Redis only, and `frontend` waits only for `backend` to exist, not to be healthy.

2. Application Startup Lifecycle

Startup happens in three layers, in this exact order, every time the `backend` container starts.

2.1 Container entrypoint: migrate, then serve

`backend/Dockerfile`'s baked-in `CMD` is simply `uvicorn app.main:app --host 0.0.0.0 --port 8000` -- **it does not run migrations**. The migration step is added at the Compose layer:

- Development: `sh -c "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload"`
- Production: the same command, minus `--reload`.

Two consequences follow: running the built image directly (bypassing Compose) starts the API **without** applying pending migrations; and because `alembic upgrade head` runs synchronously before `uvicorn` is even exec'd, a failed migration exits the container non-zero and the API never starts against a stale schema. `celery_worker` / `celery_beat` do not run `alembic upgrade head` themselves (`celery -A app.workers.celery_app worker|beat --loglevel=info`) -- they rely on the `backend` container's migration having already brought the schema up to date, since Alembic migrations are idempotent and not tied to any one process.

2.2 FastAPI application construction (import time)

Everything in `backend/app/main.py` at module scope runs once, before Uvicorn accepts any connection: structured logging is configured first (`structlog.configure(processors=[JSONRenderer()])`); the `@lru_cache`'d `Settings` singleton is constructed and frozen for the process lifetime (`get_settings()`); the `FastAPI` app is built with `openapi_url=f"{settings.api_v1_prefix}/openapi.json"` and `docs_url="/docs"`; `CORSMiddleware` is added with `allow_origins=["http://localhost:3000"] if settings.debug else []` (since `debug` defaults to `True`, a deployment that never sets `DEBUG=false` will only ever allow `localhost:3000` as a CORS origin, regardless of where the frontend is actually hosted); `Instrumentator().instrument(app).expose(app, endpoint="/metrics")` is wired in ahead of every router so it observes all of them; then ten routers are registered via `app.include_router(..., prefix=settings.api_v1_prefix)` in this fixed order: `auth`, `lookup`, `providers`, `ai_config`, `analysis`, `hunting`, `pivot`, `basket`, `cases`, `runtime` (`app/main.py:38-47`). Finally, `GET /health` is registered directly on `app`, outside any router and outside the `/api/v1` prefix, requiring no authentication -- this is the endpoint a container healthcheck or load balancer should target.

2.3 The `startup` event: seeding runtime provider configuration

`app.main` registers exactly one `@app.on_event("startup")` handler, `_seed_runtime_config()`, calling `seed_from_env_if_empty()` (`app/core/runtime_config.py`), once per process start, after the app is fully constructed but before Uvicorn begins accepting connections. If `provider_runtime_configs` already has any row, this is a no-op. Otherwise, for each of the 11 AI backends and 18 registered IOC providers, it reads that backend's/provider's credential fields off the frozen `Settings` singleton (i.e. whatever was in `.env` at container start) and inserts a `ProviderRuntimeConfig` row, Fernet-encrypting credentials before writing (full mechanism in the Provider/AI Runtime Configuration chapter). The handler is wrapped in its own `try/except Exception`, logging and continuing rather than crashing the process on failure (`main.py:57-60`) -- a failed seed leaves providers reporting as unconfigured rather than blocking startup.

2.4 Uvicorn serving

Once the startup event completes, Uvicorn accepts TCP connections on `0.0.0.0:8000` (mapped to `${HOST_PORT_BACKEND:-8000}` on the host). In development, `--reload` restarts the worker process on any change under the bind-mounted `./backend` directory -- re-running §2.2 and §2.3, but **not** `alembic upgrade head`, which only ran once in the shell wrapper before Uvicorn was first exec'd. In production there is no bind-mount and no `--reload`; picking up new code requires rebuilding the image and recreating the container, re-running the full sequence from scratch.

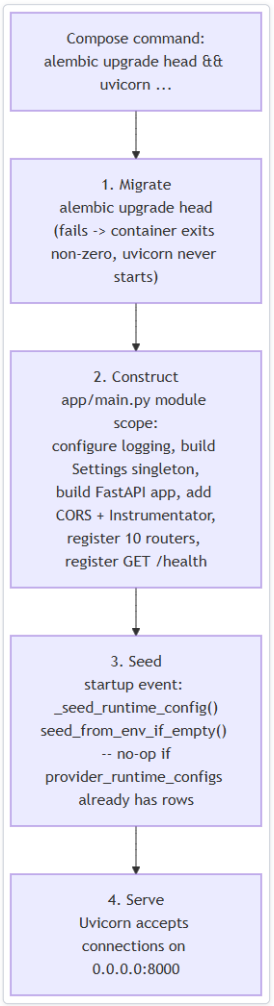

Figure 2 — Diagram: 2.4 Uvicorn serving

Diagram: Startup Sequence -- Migrate, Construct, Seed, Serve. Only step 1 is baked into the Compose `command:`, not the Docker image itself; steps 2-4 all happen inside the single `uvicorn app.main:app` process, in that fixed order, every time it starts.

3. Request Lifecycle: the General Case

Every protected endpoint (everything except `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, and the unauthenticated `GET /health`) follows the same shape: **authenticate -> authorize -> route handler -> persistence**.

1. **Authenticate** -- `Depends(get_current_user)` (`app/auth/rbac.py:28-47`) resolves before the route body runs. It extracts a bearer token via `HTTPBearer(auto_error=False)` (chosen over `OAuth2PasswordBearer` because `/auth/login` takes JSON, not an OAuth2 form-encoded grant); raises `401 Could not validate credentials` if no credentials were supplied, `decode_token()` fails, or the token's `type` claim isn't `"access"`; otherwise looks up the user by the token's `sub` (email) claim with one `SELECT`, raising the same `401` if the user is missing or inactive; and returns a lightweight `CurrentUser(id, email, role)` dataclass, not the full ORM `User` row.
2. **Authorize** -- most routes wrap that dependency in `require_permission(permission: str)` (`app/auth/rbac.py:50-62`), a dependency factory that checks `permission in ROLE_PERMISSIONS.get(user.role, set())` and raises `403 Role '{role}' lacks permission '{permission}'` on failure. The full permission matrix lives in `app/models/user.py` and is covered in the Security Architecture / RBAC chapter -- operationally, this is a single dictionary lookup keyed by role, evaluated fresh on every request with no caching beyond the JWT itself.
3. **Route handler** -- route functions (`app/api/routes/*.py`) are deliberately thin: validate/coerce the request, delegate real logic to a service module (`app/ai/*.py`, `app/providers/*.py`, `app/correlation/engine.py`, `app/evidence/*.py`, or `app/core/runtime_config.py`), and shape the return value. Multi-step logic (provider fan-out, AI calls, correlation) always lives in a dedicated module, never inline in the route.
4. **Persistence** -- most non-streaming routes take a single `AsyncSession` via `Depends(get_db)` (`app/core/db.py:12-14`), opened for the request and torn down automatically on return. `get_db()` and the standalone `new_session()` helper (used outside a request scope -- Celery tasks, and the SSE generator in §4) share one process-wide `async_sessionmaker` / `AsyncEngine` (`create_async_engine(get_settings().database_url, pool_pre_ping=True)`), so both draw from the same connection pool.

Example: `PATCH /api/v1/cases/{case_id}` resolves `require_permission("case:write")`, loads the case via `_load_case()` (`404` if missing), applies only the fields present in the body via `model_dump(exclude_unset=True)`, and returns the full serialized case -- all on the single request-scoped session, committed implicitly when it closes cleanly.

4. Request Lifecycle: the Streamed Lookup (`POST /api/v1/lookup/stream`)

The lookup-creation endpoint is the one exception to "single request-scoped session, return JSON," because it holds one HTTP connection open for the tens of seconds a full investigation takes (provider fan-out, per-provider AI summaries, correlation, final AI assessment) and streams progress as Server-Sent Events instead of blocking until completion. `stream_lookup()`

(`app/api/routes/lookup.py:51-`):

1. **Rate limit** -- a Redis-backed fixed-window `RateLimiter` keyed `lookup_create:{user.id}` (`app/core/cache.py`), bounded by `lookup_rate_limit_max_calls` / `lookup_rate_limit_window_seconds` (10/60s by default); exceeding it raises `429` before any provider or AI call runs.
2. **IOC type detection** -- `detect_ioc_type()` classifies the value unless `ioc_type_hint` was given; an undetectable type raises `422` immediately.
3. **Row creation on the request-scoped session** -- an `IOCLookup` row is inserted with `status=RUNNING` and committed on the normal `Depends(get_db)` session, its `id` captured, before the streaming response is even constructed.
4. **A second, independent session is opened inside the generator itself**, via `new_session()` , not `get_db()` . This is deliberate: a `StreamingResponse` generator body only starts executing *after* the route function returns, by which point the request-scoped session is already torn down -- using it inside the generator would silently drop every later `lookup.status = ...` mutation (a fresh `db.add()` would still appear to work, since SQLAlchemy opens an implicit new transaction for it, which is exactly why provider/correlation rows previously persisted while the lookup's own terminal status did not). The generator therefore owns and commits its own session for its whole run.
5. **Commit after every single provider result**, not once at the end -- necessary because a client disconnecting mid-investigation discarded every uncommitted `ProviderResultRecord` / `AISummaryRecord` in the same transaction, even though those calls had genuinely completed and already been yielded over SSE.
6. **A fixed SSE event sequence**, over one long-lived `text/event-stream` response:

Event	Cardinality	Payload
`detected`	once	`{lookup_id, ioc_value, ioc_type}`
`provider_result`	once per provider run	that provider's `ProviderResult.to_dict()`
`provider_summary`	once per provider with an `OK` result	the per-provider AI summary
`correlation`	once, after all providers finish	`{nodes, edges}`
`final_assessment`	once	the consolidated `FinalAssessment`
`done`	once	`{lookup_id}`

   `provider_result` / `provider_summary` interleave per provider as each finishes, letting the frontend render provider cards incrementally.
7. **Two distinct failure modes.** A regular `Exception` anywhere in the pipeline is caught, logged, flips the lookup to `status=FAILED` , commits, and yields an `error` event in place of `final_assessment` / `done` -- the HTTP response still completes `200` , since the failure is communicated inside the stream. A client disconnecting mid-stream is different: Starlette raises `GeneratorExit` (a `BaseException` , not caught by `except Exception`) into the generator at whichever `yield` is executing; the generator's `finally` block is what still flips the lookup to `FAILED` and commits in that case, so an abandoned lookup never stays stuck in `RUNNING` .

Every other write in the pipeline -- `CorrelationEdgeRecord` rows, the `FinalAssessmentRecord` history row, and the deterministic `EvidenceItem` ledger built by `build_evidence()` -- goes through this same generator-owned session, per-step-then-commit, before its corresponding SSE event is yielded.

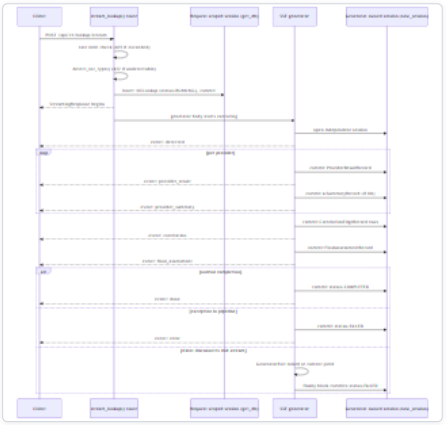

Figure 3 — Diagram: 4. Request Lifecycle: the Streamed Lookup (`POST /api/v1/lookup/stream`)

Diagram: SSE Lookup Request Lifecycle. The generator commits after every single step rather than once at the end, and owns its own database session independent of the request-scoped one -- both choices exist specifically so a disconnect or mid-pipeline exception never leaves already-completed provider/correlation work uncommitted or the lookup stuck in `RUNNING` .

5. Summary

The backend's operational shape: eight Compose services (four datastores, the FastAPI process, two Celery roles that never touch the live lookup path, and the frontend), started with a migrate-then-serve sequence baked into the `backend` / `celery_*` commands rather than the container image, followed by a best-effort runtime-config seed on the FastAPI `startup` event. Every authenticated request follows authenticate -> authorize -> thin route handler -> service module -> persistence via a single request-scoped session -- except `POST /api/v1/lookup/stream` , which trades that simplicity for a self-managed, commit-per-step session and an SSE event stream engineered to survive both mid-pipeline exceptions and mid-stream client disconnects without losing already-completed work.

API Reference

This chapter is the exhaustive endpoint-by-endpoint reference for the HORIZON GRID backend's HTTP API. Every route defined under `backend/app/api/routes/*.py` is documented here — method, path, purpose, authentication/permission requirement, request body, response shape, and error cases — traced directly to the route source. Narrative context (why the pipeline is shaped this way, how the pieces fit together) is covered in the Architecture and Data Flow chapters; this chapter is deliberately just the contract.

All 51 routes share one global prefix, `settings.api_v1_prefix = "/api/v1"` (`backend/app/core/config.py:20`), applied in `backend/app/main.py:38-47` via `app.include_router(..., prefix=settings.api_v1_prefix)` . Routers are registered, and therefore documented below, in this order: **auth → lookup → providers → ai_config → analysis → hunting → pivot → basket → cases → runtime**, plus a newer `dashboard` router documented as an addendum in §11 below. (`app/main.py` also registers `routes/admin.py` and `routes/security_assessment.py` after `runtime` and ahead of `dashboard` in real registration order; both are out of scope for this update and are not yet covered by this reference.)

#	Router file	Base path	Endpoints
1	`routes/auth.py`	`/api/v1/auth`	4
2	`routes/lookup.py`	`/api/v1/lookup`	5
3	`routes/providers.py`	`/api/v1/providers`	2
4	`routes/ai_config.py`	`/api/v1/ai`	2
5	`routes/analysis.py`	`/api/v1/lookup/{lookup_id}/analysis`	10
6	`routes/hunting.py`	`/api/v1/lookup/{lookup_id}`	2
7	`routes/pivot.py`	`/api/v1/lookup/{lookup_id}/pivots`	1
8	`routes/basket.py`	`/api/v1/basket`	5
9	`routes/cases.py`	`/api/v1/cases`	8
10	`routes/runtime.py`	`/api/v1/runtime`	10
11	`routes/dashboard.py`	`/api/v1/dashboard`	2

Total: 51 endpoints. Two additional endpoints are defined directly in `app/main.py` , outside `app/api/routes/` , and are out of scope for this reference but noted for completeness at the end of the chapter: `GET /health` (`main.py:63-65` , no prefix, no auth) and `GET /metrics` (Prometheus instrumentator, `main.py:36`).

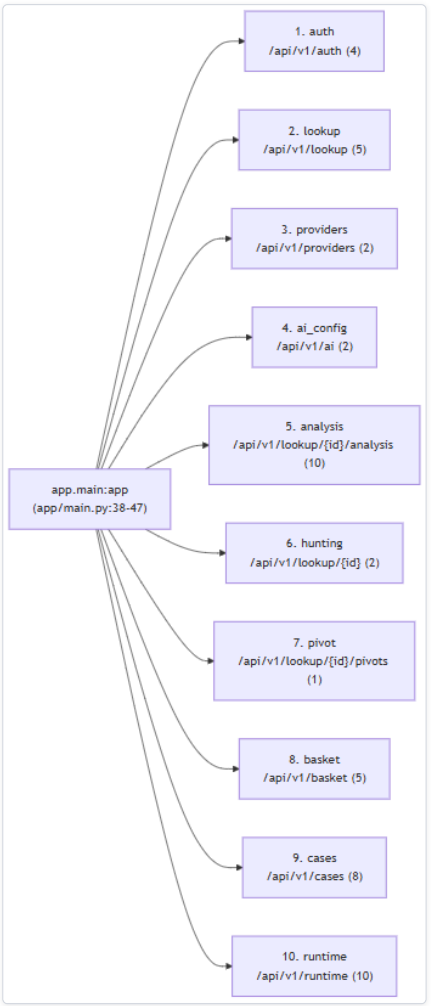

Figure 4 — Diagram: API Reference

Diagram: Router registration order and base paths, all mounted under the global `/api/v1` prefix. Registration order determines only documentation order here -- FastAPI's routing is path-based, not order-sensitive, except where two routers could otherwise share an ambiguous prefix (none do in this codebase).

How to Read This Reference

Every endpoint below follows the same template:

- **Purpose** — what the endpoint does and why it exists.
- **Auth** — the FastAPI dependency gating the route. Most routes call `require_permission("<permission-string>")`; a few require only a valid access token via `Depends(get_current_user)`; the three public auth routes require neither.
- **Path/Query params** — any parameters embedded in the URL or passed as query strings.
- **Request body** — the Pydantic schema fields the client must/may send, or "none" for routes with no body.
- **Response** — the shape of a successful response, including the HTTP status code when it is not the default `200`.
- **Errors** — HTTP status codes and the specific condition that triggers each one, beyond FastAPI's automatic `422` for a request body that fails Pydantic validation (which applies to every typed request body and is not re-stated per endpoint unless it has a distinct, hand-written `422`).

Authentication and permission mechanics

Every `require_permission(...)` dependency resolves through the same two-step chain, defined in `app/auth/rbac.py`:

1. `Depends(get_current_user)` (`rbac.py:32-47`) decodes the request's `Authorization: Bearer <token>` header as a JWT access token. It raises **401** with detail `"Could not validate credentials"` if the header is missing, the token is invalid/expired, the token's type claim is not `"access"` (e.g. a refresh token was used where an access token was expected), or the resolved user's `is_active` flag is false.
2. `require_permission(permission)` (`rbac.py:50-62`) then checks whether `permission in ROLE_PERMISSIONS.get(user.role, set())`. If not, it raises **403** with detail `f"Role '{user.role.value}' lacks permission '{permission}'"`.

There are exactly three roles (`app/models/user.py:11-14`): `admin`, `analyst`, `viewer`. The full permission-to-role matrix (`ROLE_PERMISSIONS`) lives in `app/models/user.py` and is described in the Security Architecture chapter; the permission string required by each route is called out individually below. Fourteen distinct permission strings are checked anywhere in the API: `provider:manage`, `lookup:create`, `lookup:read`, `evidence:read`, `analysis:generate`, `copilot:query`, `hunting:generate`, `basket:manage`, `case:read`, `case:create`, `case:write`, `case:close`,

`audit:read` , `dashboard:read` . `dashboard:read` is the newest of the fourteen — deliberately granted to all three roles (admin, analyst, viewer) alike, since it gates read-only operational-visibility endpoints only, never anything credential-management or write-capable.

---

1. Authentication — `routes/auth.py`

Base path `/api/v1/auth` . All four endpoints are unauthenticated at the route level except `/me` ; `/register` and `/login` are the only ways to obtain a token in the first place.

`POST /api/v1/auth/register`

`app/api/routes/auth.py:20`

- **Purpose:** Register a new user account. **Bootstrap-only:** the very first account ever created on an instance is automatically granted the `admin` role; every registration attempt after that is rejected outright rather than receiving `analyst` .
- **Auth:** None (public), but only succeeds while the `users` table is empty.
- **Request body** (`RegisterRequest` , `app/schemas/auth.py:16-19`):

Field	Type	Notes
`email`	`EmailStr`	required
`password`	`str`	min 8, max 72 characters (72 is bcrypt's real input limit)
`full_name`	`str`	optional, defaults to `""`

- **Response** (`UserResponse` , HTTP **201**, only on an empty `users` table): `id: str` , `email: str` , `full_name: str` , `role: Role` (always `admin` for this route).
- **Errors: 400** `"Email already registered"` if the email already exists (`auth.py:25`); **403** `"Self-registration is closed. Ask an administrator to create your account from the Administration page."` if at least one user already exists (`auth.py:32-35`) — use `POST /api/v1/admin/users` instead.

`POST /api/v1/auth/login`

`app/api/routes/auth.py:42`

- **Purpose:** Authenticate with email and password; issues a JWT access token and refresh token pair.
- **Auth:** None (public).
- **Request body** (`LoginRequest`): `email: EmailStr` , `password: str` (max 72 characters).
- **Response** (`TokenResponse`): `access_token: str` , `refresh_token: str` , `token_type: str` (always `"bearer"`).
- **Errors: 401** `"Invalid email or password"` on bad credentials (`auth.py:47`); **403** `"Account disabled"` if the matched user's `is_active` is false (`auth.py:49`).

`POST /api/v1/auth/refresh`

`app/api/routes/auth.py:56`

- **Purpose:** Exchange a still-valid refresh token for a new access/refresh token pair, without re-sending a password.
- **Auth:** None at the dependency level — validity is enforced by decoding the refresh token itself, not a bearer dependency.
- **Request body** (`RefreshRequest`): `refresh_token: str` .
- **Response** (`TokenResponse`): identical shape to `/login` .
- **Errors: 401** `"Invalid refresh token"` if the token is missing, malformed, or its type claim is not `"refresh"` (`auth.py:60`); **401** (same message) if the user it resolves to is missing or inactive (`auth.py:64`).

`GET /api/v1/auth/me`

`app/api/routes/auth.py:71`

- **Purpose:** Return the identity of the caller associated with the presented access token.
- **Auth:** `Depends(get_current_user)` — a valid access token is required; there is no specific permission string on this route.
- **Request body:** none.
- **Response** (`UserResponse`): `id` , `email` , `full_name` (hard-coded to `""` in this handler — not populated from the database on this particular route), `role` .
- **Errors: 401** `"Could not validate credentials"` for any invalid/missing/expired token or inactive user.

---

2. IOC Lookup — `routes/lookup.py`

Base path `/api/v1/lookup` . This is the core investigation pipeline: creating a lookup, streaming its lifecycle, re-running the AI assessment, and reading back results.

`POST /api/v1/lookup/stream`

`app/api/routes/lookup.py:51`

- **Purpose:** Create a lookup, fan the submitted IOC out to every applicable provider in parallel, and stream the entire investigation lifecycle back to the client as Server-Sent Events (SSE) — detection, each provider's result and AI summary as they complete, the correlation graph, and the final AI assessment.
- **Auth:** `require_permission("lookup:create")` .
- **Request body** (`LookupCreateRequest` , `app/schemas/lookup.py:9-20`):

Field	Type	Notes
`value`	`str`	the raw IOC string
`ioc_type_hint`	`Optional[IOCType]`	forces type classification if auto-detection would be ambiguous
`provider_ids`	`Optional[list[str]]`	restricts the fan-out to specific providers; omit to use all applicable providers
`ai_backend`	`Optional[str]`	overrides the active AI backend for this one lookup

- **Response:** `StreamingResponse`, media type `text/event-stream` — not a single JSON body. Named SSE events, in order: `detected` (`{lookup_id, ioc_value, ioc_type}`), then a `provider_result` (the provider's `to_dict()`) followed by a `provider_summary` for each provider as it finishes, `correlation` (`{nodes, edges}`), `final_assessment` (the final assessment's `model_dump()`), and `done` (`{lookup_id}`). On a mid-stream failure, an `error` event (`{message}`) is emitted in place of `final_assessment` / `done`.
- **Errors: 429** with a message referencing `settings.lookup_rate_limit_max_calls` / `lookup_rate_limit_window_seconds` when the caller's rate limit is exceeded (`lookup.py:68-75`); **422** `"Could not determine IOC type; pass ioc_type_hint."` if automatic type detection fails and no hint was supplied (`lookup.py:80`). Provider or pipeline exceptions raised mid-stream are caught, logged, the lookup is marked `FAILED`, and the failure is surfaced as an `error` SSE event rather than an HTTP error status (`lookup.py:256-260`) — a client must inspect the event stream itself to detect failure, not just the initial HTTP response code.

`POST /api/v1/lookup/{lookup_id}/reanalyze`

`app/api/routes/lookup.py:310`

- **Purpose:** Re-run only the final-assessment AI step against an already-completed lookup's persisted evidence, using an explicitly chosen AI backend, without re-querying providers. The result is stored as an additional, non-primary `FinalAssessmentRecord` — the lookup's original primary verdict is untouched.
- **Auth:** `require_permission("lookup:create")`.
- **Path param:** `lookup_id` (UUID string).
- **Request body** (`ReanalyzeRequest`, inline `lookup.py:306-307`): `ai_backend: str`.
- **Response:** `{id: str, ai_backend: str, ai_model: str | None, assessment: dict}`.
- **Errors: 404** `"Lookup not found"` (`lookup.py:331`); **400** `"Lookup must be completed before it can be re-analyzed."` if the lookup's status is not `COMPLETED` (`lookup.py:333`).

`GET /api/v1/lookup/{lookup_id}/assessments`

`app/api/routes/lookup.py:387`

- **Purpose:** List every final assessment ever generated for a lookup — the original plus every subsequent re-analysis — so backends can be compared side by side against the same evidence.
- **Auth:** `require_permission("lookup:read")`.
- **Path param:** `lookup_id`.
- **Request body:** none.
- **Response:** list of `{id, ai_backend, ai_model, is_primary: bool, assessment: dict, created_at: str}`, ordered ascending by `created_at`.
- **Errors:** none explicit — an unknown or row-less `lookup_id` simply returns an empty list; this route does not 404 on a missing lookup.

`GET /api/v1/lookup/{lookup_id}`

`app/api/routes/lookup.py:416`

- **Purpose:** Fetch full detail for a single lookup — provider results, AI summaries, the primary final assessment, and a correlation graph rebuilt from persisted rows.
- **Auth:** `require_permission("lookup:read")`.
- **Path param:** `lookup_id`.
- **Request body:** none.
- **Response:** `{id, ioc_value, ioc_type, status, final_verdict: str | None, risk_score, confidence_score, final_assessment: dict | None, provider_results: [{provider_id, provider_name, category, status, data, source_url, error_message, latency_ms}], ai_summaries: [dict], created_at, correlation: {nodes: [{node_id, ioc_type, value, labels: []}], edges: [{source, target, relationship, confidence, provenance}]}}`. The correlation payload is rebuilt by `_correlation_payload` (`lookup.py:462-501`) and always includes the seed IOC node even when it has zero edges.
- **Errors: 404** `"Lookup not found"` (`lookup.py:433`).

`GET /api/v1/lookup` **(list)**

`app/api/routes/lookup.py:504`

- **Purpose:** List recent lookups across the whole team — this is a shared, not per-user, view.
- **Auth:** `require_permission("lookup:read")`.
- **Query params:** `limit: int = 50`, clamped server-side to the range `[1, 200]` (`lookup.py:513`).
- **Request body:** none.
- **Response:** list of `{id, ioc_value, ioc_type, status, final_verdict: str | None, risk_score, confidence_score, created_at}`, ordered by `created_at` descending.
- **Errors:** none explicit.

---

3. Providers — `routes/providers.py`

Base path `/api/v1/providers`. Read-only health reporting plus a live credential test used by the setup wizard and the Manage Providers UI.

`GET /api/v1/providers/health`

`app/api/routes/providers.py:11`

- **Purpose:** Report the real, database-backed health of every registered IOC provider across four rolling time windows (1h/24h/7d/30d) — status, success rate, average latency, consecutive-failure count, and rate-limited count per window, computed from actual persisted provider-call outcomes. **This route's behavior changed in this update:** it previously returned only static configuration metadata (`configured` / `requires_key` / `supported_types`) from a stub, `get_provider_health()` (`app/providers/registry.py`), which never touched the database or reported any real status, latency, or failure information — that stub is now dead code. The route now delegates to `get_provider_health_history()` (`app/core/dashboard.py`), covered in the Architecture chapter's Executive Dashboard subsection.
- **Auth:** `require_permission("dashboard:read")` — changed from the prior `lookup:read` . Every role that previously had `lookup:read` also has `dashboard:read` , so this change does not reduce anyone's access.
- **Request body:** none.
- **Response:** a list of per-provider dicts, one for every registered provider (including providers with zero recorded calls ever) — `{provider_id, provider_name, category, configured, requires_key, supported_types: list[str], "1h": {...}, "24h": {...}, "7d": {...}, "30d": {...}}` , where each window object is `{status, success_rate, avg_latency_ms, consecutive_failures, rate_limited_count}` . `status` is one of `"healthy"` / `"degraded"` / `"down"` / `"unknown"` — a window with zero real attempts always reports `"unknown"` , never `"healthy"` ; a provider correctly reporting "nothing found" (`no_data`) or "doesn't apply to this IOC type" (`unsupported_ioc`) for an attempt counts as a healthy outcome, not a failure. `success_rate` and `avg_latency_ms` are `null` (not `0`) when the window has zero qualifying attempts, so "no data" is never rendered as a misleading zero.
- **Errors:** none explicit.

`POST /api/v1/providers/{provider_id}/test`

`app/api/routes/providers.py:20`

- **Purpose:** Live credential check for a single IOC provider — makes one real outbound call using candidate credentials supplied in the request body. The credentials are never persisted by this route; it exists purely to validate a key before it is saved.
- **Auth:** `require_permission("provider:manage")` .
- **Path param:** `provider_id: str` .
- **Request body** (`ProviderTestRequest` , inline `providers.py:16-17`): `credentials: dict[str, str]` .
- **Response:** `{provider_id: str, ok: bool, message: str, latency_ms: <type per test_provider_connection result>}` .
- **Errors:** none raised as `HTTPException` in this route — failures from the underlying connection test surface through the `ok` / `message` fields of a `200` response, not as an HTTP error status.

---

4. AI Configuration — `routes/ai_config.py`

Base path `/api/v1/ai` . Live AI-backend credential testing and model-list discovery for the setup wizard / Manage Providers UI.

`POST /api/v1/ai/test`

`app/api/routes/ai_config.py:39`

- **Purpose:** Live credential check for any of the eleven AI backends (Ollama, Anthropic, Bedrock, Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, OpenRouter) — makes one real, minimal chat request with candidate credentials. Never persists anything.
- **Auth:** `require_permission("provider:manage")` .
- **Request body** (`AITestRequest` , inline `ai_config.py:33-36`): `backend: str` , `credentials: dict[str, str]` (default `{}`), `model: str | None` (default `None`).
- **Response:** `{backend: str, ok: bool, message: str, model: str | None, latency_ms: <type>}` .
- **Errors:** none raised as `HTTPException` — failures surface through `ok` / `message` .

`POST /api/v1/ai/{backend}/models`

`app/api/routes/ai_config.py:63`

- **Purpose:** Return the model list for the AI-backend picker's dropdown. Live discovery for Groq (its `/models` API) and Ollama (`GET {base_url}/api/tags`); static curated lists for Anthropic, Gemini, and Bedrock; an empty list for an unrecognized backend.
- **Auth:** `require_permission("provider:manage")` .
- **Path param:** `backend: str` .
- **Request body** (`ModelListRequest` , inline `ai_config.py:59-60`): `credentials: dict[str, str]` (default `{}`).
- **Response** (shape varies by backend, always includes `backend` and `models`):
  - Groq: `{backend, models: list[str], source: "fallback" | "live", default: str}` — falls back to a hard-coded model list if no `api_key` is supplied or the live call raises `httpx.HTTPError` (`ai_config.py:76-85`).
  - Ollama: `{backend, models: list[str], source: "live" | "fallback", default: str}` — live via `GET {base_url}/api/tags` if a `base_url` is supplied and returns HTTP 200 with model names; otherwise a static two-item fallback list (`ai_config.py:87-99`).
  - Anthropic / Gemini / Bedrock: `{backend, models: list[str], source: "static", default: str}` (`ai_config.py:101-104`).
  - Unknown backend: `{backend, models: [], source: "unknown", default: None}` (`ai_config.py:102-103`).
- **Errors:** none raised as `HTTPException` — provider-side HTTP errors are caught and logged, degrading to the fallback/static response instead of failing the request.

---

5. Lookup Analysis — `routes/analysis.py`

Base path `/api/v1/lookup/{lookup_id}/analysis`. These are the "explain the verdict" endpoints: on-demand AI generations grounded in a *completed* lookup's persisted evidence ledger. Every endpoint in this section shares one loader, `_load_lookup` (`analysis.py:45-51`), which raises **404** `"Lookup not found"` if the lookup does not exist, and — for every endpoint except `/evidence` — also raises **409** `f"Lookup is not completed yet (status={lookup.status.value})"` if the lookup's status is not `COMPLETED`. That shared behavior is stated once here and referenced, not repeated, in each endpoint's Errors line below.

`GET /api/v1/lookup/{lookup_id}/analysis/evidence`

`app/api/routes/analysis.py:70`

- **Purpose:** Return the raw evidence ledger for a lookup (the "Show Receipts" / Evidence tab). Purely deterministic — no AI call is made.
- **Auth:** `require_permission("evidence:read")`.
- **Request body:** none.
- **Response:** list of `{id, evidence_type: str, source_label, provider_id, claim, interpretation, confidence, related_ioc_type, related_ioc_value, source_url, observed_at, created_at: str}`.
- **Errors: 404** `"Lookup not found"` only — this is the one endpoint in this router with no `409` completed-status gate.

`POST /api/v1/lookup/{lookup_id}/analysis/why`

`app/api/routes/analysis.py:99`

- **Purpose:** AI explanation of why the IOC is, or is not, considered malicious, grounded in the evidence ledger.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`WhyMaliciousExplanation`, `app/ai/analysis_schemas.py:41-44`): `verdict_restated: str`, `reasons: list[{reason: str, evidence_ids: list[str]}]`, `caveat: str | None`.
- **Errors: 404** / **409** per the shared loader above.

`POST /api/v1/lookup/{lookup_id}/analysis/what-is-this`

`app/api/routes/analysis.py:110`

- **Purpose:** AI plain-language and technical explanation of what the IOC actually is.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`WhatIsThisIOC`, `analysis_schemas.py:47-52`): `plain_language_summary: str`, `technical_explanation: str`, `evidence_ids: list[str]`, `confidence_narrative: str`, `related_infrastructure: list[str]`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/disagreement`

`app/api/routes/analysis.py:121`

- **Purpose:** AI summary of where the queried providers agree and disagree on this IOC.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`DisagreementSummary`, `analysis_schemas.py:55-60`): `agreement: str`, `conflict: str`, `missing_data: str`, `most_reliable_evidence: str`, `evidence_ids: list[str]`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/false-positive`

`app/api/routes/analysis.py:132`

- **Purpose:** AI assessment of false-positive likelihood — e.g. is the IOC actually a CDN edge, shared-hosting IP, or scanner rather than genuinely malicious infrastructure.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`FalsePositiveAssessment`, `analysis_schemas.py:63-73`): `likely_false_positive: bool`, `candidate_categories: list[Literal["cdn","cloud_provider","shared_hosting","nat","vpn","proxy","security_scanner","search_crawler","monitoring_system","legitimate_business_infrastructure","none"]]`, `explanation: str`, `evidence_ids: list[str]`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/challenge`

`app/api/routes/analysis.py:143`

- **Purpose:** AI "red-team" self-challenge of the current verdict — actively argues against the assessment the platform already produced.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`ChallengeVerdict`, `analysis_schemas.py:76-82`): `supporting_evidence: list[{reason, evidence_ids}]`, `contradictory_evidence: list[{reason, evidence_ids}]`, `missing_evidence: list[str]`, `alternative_explanation: str`, `final_confidence: Literal["low","medium","high"]`, `final_confidence_rationale: str`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/next-actions`

`app/api/routes/analysis.py:154`

- **Purpose:** AI-suggested next investigative actions, informed by the evidence ledger and the correlation graph.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`SmartNextActions`, `analysis_schemas.py:85-94`): `actions: list[{action: str, target_ioc_value: str | None, target_ioc_type: str | None, rationale: str, priority: Literal["low","medium","high"]}]`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/gaps`

`app/api/routes/analysis.py:167`

- **Purpose:** AI-identified intelligence gaps, informed by which providers returned data and which did not.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`IntelligenceGaps`, `analysis_schemas.py:97-103`): `gaps: list[{gap: str, how_to_close: str}]`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/score-explanation`

`app/api/routes/analysis.py:185`

- **Purpose:** AI explanation of how the lookup's numeric risk score was derived.
- **Auth:** `require_permission("analysis:generate")`.
- **Request body:** none.
- **Response** (`ScoreExplanation`, `analysis_schemas.py:106-114`): `components: list[{component: str, contribution: str, evidence_ids: list[str]}]`, `summary: str`.
- **Errors: 404** / **409**.

`POST /api/v1/lookup/{lookup_id}/analysis/copilot`

`app/api/routes/analysis.py:196`

- **Purpose:** Free-form Q&A ("Copilot") over a lookup's evidence, correlation graph, and optional analyst notes.
- **Auth:** `require_permission("copilot:query")` — note this is the one endpoint in this router gated by a different permission string than `analysis:generate`.
- **Request body:** raw, untyped `dict`, documented inline as `{"question": str, "notes": list[str] (optional)}` (`analysis.py:203`).
- **Response** (`CopilotAnswer`, `analysis_schemas.py:153-156`): `answer: str`, `evidence_ids: list[str]`, `suggested_follow_ups: list[str]` (max 4 items).
- **Errors: 422** `"question is required"` if `question` is empty or whitespace-only after stripping (`analysis.py:206`); plus **404** / **409** from the shared loader.

---

6. Threat Hunting — `routes/hunting.py`

Base path `/api/v1/lookup/{lookup_id}`. Both endpoints share a local `_load_lookup` helper (`hunting.py:26-30`) that only checks existence — **404** `"Lookup not found"` — with **no completed-status gate**, unlike `analysis.py`'s helper. A hunting package or detection rule can therefore be requested even for a lookup that is still `RUNNING` or has `FAILED`.

`POST /api/v1/lookup/{lookup_id}/hunt`

`app/api/routes/hunting.py:33`

- **Purpose:** Generate an exact-match plus expansion threat-hunting query package (Sigma, Splunk SPL, Sentinel KQL, etc.) for the lookup's seed IOC and its correlated related indicators.
- **Auth:** `require_permission("hunting:generate")`.
- **Query params:** `formats: list[str]`, default `["sigma","splunk_spl","sentinel_kql","elastic","qradar_aql","chronicle_yara_l","suricata","snort","zeek"]` (`hunting.py:21-23,36`).
- **Request body:** none.
- **Response** (`HuntingPackage`, `analysis_schemas.py:129-138`): `exact_match_queries: list[{format, query, detects}]`, `expansion_targets: list[{related_ioc_value, related_ioc_type, rationale}]`, `broader_queries: list[{format, query, detects}]`.
- **Errors: 404** `"Lookup not found"` (`hunting.py:29`).

`POST /api/v1/lookup/{lookup_id}/detection`

`app/api/routes/hunting.py:48`

- **Purpose:** Generate a single detection-rule draft, in a specified format, for the lookup's IOC and its current verdict/risk score.
- **Auth:** `require_permission("hunting:generate")`.
- **Query params:** `format: str`, required (`hunting.py:51`); valid values are constrained downstream by the `_DetectionRuleFormat` literal — `sigma, yara, splunk_spl, sentinel_kql, elastic, qradar_aql, chronicle_yara_l, suricata, snort, zeek`.
- **Request body:** none.
- **Response** (`DetectionRuleDraft`, `analysis_schemas.py:141-150`): `format`, `title`, `rule`, `detection_objective`, `data_source`, `logic_explanation`, `false_positive_considerations`, `severity: Literal["none","low","medium","high","critical"]`, `mitre_technique_ids: list[str]`.

- **Errors: 404** `"Lookup not found"`.

---

7. Pivoting — `routes/pivot.py`

Base path `/api/v1/lookup/{lookup_id}/pivots`. A single deterministic (non-AI) endpoint.

`GET /api/v1/lookup/{lookup_id}/pivots`

`app/api/routes/pivot.py:23`

- **Purpose:** Deterministic ranking of directly-related IOCs pulled from the correlation graph, ordered by confidence/corroboration, to support one-click pivoting from an investigation.
- **Auth:** `require_permission("lookup:read")`.
- **Query params:** `limit: int = 10`.
- **Request body:** none.
- **Response:** the return value of `rank_pivots(lookup.ioc_value, edge_likes, limit)` (`app/evidence/pivot.py`) — a ranked list; the exact field shape is defined in that module and is deliberately a pure, non-AI sort over correlation edges.
- **Errors: 404** `"Lookup not found"` (`pivot.py:32`).

---

8. IOC Basket — `routes/basket.py`

Base path `/api/v1/basket`. A per-analyst scratch space of saved IOCs, scoped by the caller's own user ID — every endpoint here operates only on the requesting user's own basket items.

`GET /api/v1/basket`

`app/api/routes/basket.py:34`

- **Purpose:** List the caller's own basket items, newest first.
- **Auth:** `require_permission("basket:manage")`.
- **Request body:** none.
- **Response:** list of `{id, ioc_value, ioc_type, note: str | None, latest_lookup_id: str | None, created_at}` (via the `_serialize` helper, `basket.py:23-31`).
- **Errors:** none explicit.

`POST /api/v1/basket`

`app/api/routes/basket.py:45`

- **Purpose:** Add an IOC to the caller's basket. Deduplicates by exact `ioc_value` per owner, and best-effort attaches the most recent *completed* lookup for that value if one exists.
- **Auth:** `require_permission("basket:manage")`.
- **Request body** (`BasketAddRequest`, `app/schemas/basket.py:8-11`): `ioc_value: str`, `ioc_type_hint: Optional[IOCType]`, `note: Optional[str]`.
- **Response** (HTTP **201** for a new item, or the same serializer's output for an item that already existed): `{id, ioc_value, ioc_type, note, latest_lookup_id, created_at}`.
- **Errors: 422** `"Could not determine IOC type; pass ioc_type_hint."` if the type cannot be auto-detected and no hint was supplied (`basket.py:54`).

`DELETE /api/v1/basket/{item_id}`

`app/api/routes/basket.py:89`

- **Purpose:** Remove one basket item owned by the caller.
- **Auth:** `require_permission("basket:manage")`.
- **Path param:** `item_id`.
- **Request body:** none.
- **Response: 204 No Content** (empty body).
- **Errors: 404** `"Basket item not found"` if no such item exists, or it exists but is owned by a different user (`basket.py:98-99`) — ownership mismatch is indistinguishable from non-existence in the response.

`DELETE /api/v1/basket`

`app/api/routes/basket.py:104`

- **Purpose:** Clear all of the caller's own basket items in one call.
- **Auth:** `require_permission("basket:manage")`.
- **Request body:** none.
- **Response: 204 No Content**.
- **Errors:** none explicit.

`POST /api/v1/basket/compare`

`app/api/routes/basket.py:115`

- **Purpose:** Build a deterministic side-by-side comparison table across 2–10 already-looked-up IOCs (referenced by `lookup_id`), then generate an AI narrative comparing them.

- **Auth:** `require_permission("basket:manage")`.
- **Request body:** raw `dict`, documented inline as `{"lookup_ids": list[str]}` (`basket.py:121-122`).
- **Response:** `{rows: list[{ioc_value, ioc_type, verdict: str, risk_score, confidence_score, asn: list[str], malware_families: list[str], threat_actors: list[str], related_domains: list[str], first_seen: str}], narrative: {most_dangerous_ioc_value: str | None, narrative: str, key_differences: list[str]}}` — the narrative fields come from `IOCComparisonNarrative` (`analysis_schemas.py:159-165`).
- **Errors: 422** `"Provide at least 2 lookup_ids to compare"` if fewer than 2 are supplied (`basket.py:128`); **422** `"Cannot compare more than 10 IOCs at once"` if more than 10 are supplied (`basket.py:130`); **422** `"Fewer than 2 of the given lookup_ids resolved to completed lookups"` if resolution yields fewer than 2 valid, completed rows (`basket.py:161`).

---

9. Case Management — `routes/cases.py`

Base path `/api/v1/cases`. Cases are team-shared (not per-analyst) investigation containers grouping IOCs, notes, and reports. Most endpoints share a loader, `_load_case` (`cases.py:68-77`), which raises **404** `"Case not found"` for a missing case; that is stated once here and referenced, not repeated, below.

`GET /api/v1/cases`

`app/api/routes/cases.py:80`

- **Purpose:** List cases across the whole team, optionally filtered by status.
- **Auth:** `require_permission("case:read")`.
- **Query params:** `status_filter: str | None = None`, `limit: int = 50` (clamped to `[1, 200]`, `cases.py:87`).
- **Request body:** none.
- **Response:** list of `{id, title, severity: str, status: str, tags: list[str], created_at}`.
- **Errors:** none explicit.

`POST /api/v1/cases`

`app/api/routes/cases.py:105`

- **Purpose:** Create a new case, owned by the calling analyst.
- **Auth:** `require_permission("case:create")`.
- **Request body** (`CaseCreateRequest`, `app/schemas/case.py:8-12`):

Field	Type	Notes
`title`	`str`	1–255 characters
`description`	`Optional[str]`	—
`severity`	`CaseSeverity`	default `MEDIUM`; one of `low` / `medium` / `high` / `critical`
`tags`	`list[str]`	default `[]`

- **Response** (HTTP **201**): full case object via `_serialize_case` (`cases.py:22-65`) — `{id, title, description, analyst_id, severity, status, tags, created_at, updated_at, iocs: [{id, ioc_value, ioc_type, lookup_id, added_by, created_at}], notes: [{id, author_id, body, anchor_type, anchor_ref, created_at}], reports: [{id, report_type, title, generated_by, created_at}]}`.
- **Errors:** none explicit — invalid field values fail standard FastAPI/Pydantic `422` validation.

`GET /api/v1/cases/{case_id}`

`app/api/routes/cases.py:124`

- **Purpose:** Fetch full detail for one case — its IOCs, notes, and reports.
- **Auth:** `require_permission("case:read")`.
- **Path param:** `case_id`.
- **Request body:** none.
- **Response:** the full case object, as above.
- **Errors: 404** `"Case not found"`.

`PATCH /api/v1/cases/{case_id}`

`app/api/routes/cases.py:133`

- **Purpose:** Partially update a case's mutable fields — only fields explicitly present in the request body are applied.
- **Auth:** `require_permission("case:write")`.
- **Request body** (`CaseUpdateRequest`, `app/schemas/case.py:15-20`, all fields optional, applied via `model_dump(exclude_unset=True)`): `title: Optional[str]`, `description: Optional[str]`, `severity: Optional[CaseSeverity]`, `status: Optional[CaseStatus]` (`open` / `investigating` / `contained` / `resolved` / `false_positive` / `closed`), `tags: Optional[list[str]]`.
- **Response:** the full updated case object.
- **Errors: 404** `"Case not found"`.

`POST /api/v1/cases/{case_id}/close`

`app/api/routes/cases.py:148`

- **Purpose:** Force-close a case — sets `status` to `CLOSED` regardless of its current status.
- **Auth:** `require_permission("case:close")` — note this is a distinct permission string from `case:write`, allowing a role matrix where closing is more restricted than editing.
- **Request body:** none.
- **Response:** the full updated case object.
- **Errors: 404** `"Case not found"`.

`POST /api/v1/cases/{case_id}/iocs`

`app/api/routes/cases.py:162`

- **Purpose:** Attach an IOC to a case, optionally linking it to an existing lookup.
- **Auth:** `require_permission("case:write")`.
- **Request body** (`CaseIOCAddRequest`, `app/schemas/case.py:23-26`): `ioc_value: str`, `ioc_type: str`, `lookup_id: Optional[str]`.
- **Response** (HTTP **201**): the full updated case object.
- **Errors: 404** `"Case not found"` (from `_load_case`). Note: supplying an `lookup_id` that is not a syntactically valid UUID raises an unhandled `ValueError` from `uuid.UUID(...)` (`cases.py:174`) — this surfaces as an unhandled server error rather than a documented `HTTPException` / `422`, and is a real gap rather than an intentional error path.

`DELETE /api/v1/cases/{case_id}/iocs/{ioc_id}`

`app/api/routes/cases.py:182`

- **Purpose:** Remove an IOC from a case.
- **Auth:** `require_permission("case:write")`.
- **Path params:** `case_id`, `ioc_id`.
- **Request body:** none.
- **Response: 204 No Content**.
- **Errors: 404** `"Case IOC not found"` if no `CaseIOC` row matches that `case_id` / `ioc_id` pair (`cases.py:193`).

`POST /api/v1/cases/{case_id}/notes`

`app/api/routes/cases.py:198`

- **Purpose:** Add an analyst note to a case, optionally anchored to a specific artifact (an IOC, an evidence item, a graph node, a timeline event, or a provider result — a free string, not a foreign key).
- **Auth:** `require_permission("case:write")`.
- **Request body** (`CaseNoteCreateRequest`, `app/schemas/case.py:29-32`): `body: str` (min length 1), `anchor_type: Optional[str]`, `anchor_ref: Optional[str]`.
- **Response** (HTTP **201**): the full updated case object.
- **Errors: 404** `"Case not found"`.

---

10. Runtime Configuration — `routes/runtime.py`

Base path `/api/v1/runtime`. This is the API surface behind the DB-backed "Manage Providers" feature: configuring and activating AI backends, configuring and enabling IOC providers, and reading the audit log — all without editing `.env` or restarting a container. Business logic lives in `app/core/runtime_config.py`; every write-oriented endpoint here delegates to it.

`GET /api/v1/runtime/ai-providers`

`app/api/routes/runtime.py:26`

- **Purpose:** List every configured AI-backend runtime row — persisted configuration, masked credentials, active/last-test status.
- **Auth:** `require_permission("provider:manage")`.
- **Request body:** none.
- **Response:** the return value of `svc.list_ai_providers()` (`app/core/runtime_config.py`); row shape is not independently typed by this route.
- **Errors:** none explicit.

`POST /api/v1/runtime/ai-providers/{backend}`

`app/api/routes/runtime.py:38`

- **Purpose:** Persist validated credentials and/or a model choice for one AI backend into the runtime-mutable store. Takes effect on the very next AI call — no restart required.
- **Auth:** `require_permission("provider:manage")`.
- **Path param:** `backend: str`.
- **Request body** (`ConfigureAIProviderRequest`, inline `runtime.py:31-35`): `credentials: dict[str, str]` (default `{}`), `model_id: Optional[str]`.
- **Response:** the return value of `svc.upsert_ai_provider(...)`.
- **Errors: 400** `f"Unknown AI backend {backend!r}"` if `backend` is not in `svc.AI_BACKENDS` (`runtime.py:45`).

`POST /api/v1/runtime/ai-active`

`app/api/routes/runtime.py:55`

- **Purpose:** Switch the platform-wide active AI backend immediately — affects the very next investigation and every subsequent AI call.
- **Auth:** `require_permission("provider:manage")`.
- **Request body** (`ActivateAIRequest`, inline `runtime.py:51-52`): `backend: str`.
- **Response:** `{active_backend: str}`.
- **Errors: 400** with `str(exc)` as the detail message if `svc.set_active_ai_backend` raises `ValueError` — e.g. the backend is unknown or not yet configured (`runtime.py:64-65`).

`GET /api/v1/runtime/ai-active`

`app/api/routes/runtime.py:69`

- **Purpose:** Return the currently active AI backend and model.
- **Auth:** `require_permission("lookup:read")` — note this route uses a different permission string than every other endpoint in this router, since read-only callers (e.g. the home-page AI Quick Switch) do not need `provider:manage`.
- **Request body:** none.
- **Response:** `{backend: str | None, model_id: str | None}` — both `None` if nothing has been configured yet (`runtime.py:72-73`); otherwise `{backend: config["backend"], model_id: config["model_id"]}`.
- **Errors:** none explicit.

`POST /api/v1/runtime/ai-providers/{backend}/record-test`

`app/api/routes/runtime.py:82`

- **Purpose:** Record the outcome of a prior `/api/v1/ai/test` call against a backend's now-saved credential, so the UI can display a "last tested" status without re-running the test.
- **Auth:** `require_permission("provider:manage")`.
- **Path param:** `backend: str`.
- **Request body** (`TestResultRequest`, inline `runtime.py:77-79`): `ok: bool`, `message: str` (default `""`).
- **Response:** `{recorded: true}`.
- **Errors:** none explicit.

`GET /api/v1/runtime/ioc-providers`

`app/api/routes/runtime.py:98`

- **Purpose:** List every known IOC provider merged with its runtime configuration row (or a synthesized default if it has never been configured), together with static metadata (`requires_key`, `credential_fields`, `category`, `supported_types`).
- **Auth:** `require_permission("provider:manage")`.
- **Request body:** none.
- **Response:** a list of dicts, each either the persisted row from `svc.list_ioc_providers()` or a synthesized default — `{provider_id, provider_name, kind: "ioc", enabled: true, is_active: false, configured, model_id: null, extra_config: {}, masked_credentials: {}, last_test_at: null, last_test_ok: null, last_test_message: null, updated_at: null}` — with `requires_key`, `credential_fields: list`, `category: str`, `supported_types: list[str]` appended in either case (`runtime.py:123-126`).
- **Errors:** none explicit.

`POST /api/v1/runtime/ioc-providers/{provider_id}`

`app/api/routes/runtime.py:135`

- **Purpose:** Persist validated credentials and/or extra configuration for one IOC provider.
- **Auth:** `require_permission("provider:manage")`.
- **Path param:** `provider_id: str`.
- **Request body** (`ConfigureIOCProviderRequest`, inline `runtime.py:130-132`): `credentials: dict[str, str]` (default `{}`), `extra_config: Optional[dict]`.
- **Response:** the return value of `svc.upsert_ioc_provider(...)`.
- **Errors: 404** `f"Unknown IOC provider {provider_id!r}"` if the ID is not in the known provider registry (`runtime.py:144`).

`POST /api/v1/runtime/ioc-providers/{provider_id}/enabled`

`app/api/routes/runtime.py:159`

- **Purpose:** Enable or disable an IOC provider at runtime — takes effect on the next investigation, no restart required.
- **Auth:** `require_permission("provider:manage")`.
- **Path param:** `provider_id: str`.
- **Request body** (`EnableRequest`, inline `runtime.py:155-156`): `enabled: bool`.
- **Response:** `{provider_id: str, enabled: bool}`.
- **Errors:** none explicit.

`POST /api/v1/runtime/ioc-providers/{provider_id}/record-test`

`app/api/routes/runtime.py:172`

- **Purpose:** Record the outcome of a prior connection test for an IOC provider.

- **Auth:** `require_permission("provider:manage")`.
- **Path param:** `provider_id: str`.
- **Request body** (`TestResultRequest`): `ok: bool`, `message: str` (default `""`).
- **Response:** `{recorded: true}`.
- **Errors:** none explicit.

`GET /api/v1/runtime/audit-log`

`app/api/routes/runtime.py:185`

- **Purpose:** Retrieve the runtime-configuration audit log — who changed which provider or AI configuration, and when.
- **Auth:** `require_permission("audit:read")` — the only route gated by this permission string.
- **Query params:** `limit: int = 200`.
- **Request body:** none.
- **Response:** the return value of `svc.list_audit_log(limit)`.
- **Errors:** none explicit.

---

11. Dashboard — `routes/dashboard.py`

Base path `/api/v1/dashboard`. Both endpoints back the Executive Dashboard added in this update. Both are read-only aggregations over already-persisted data — neither makes a new provider call, a new AI call outside the one described below, or persists anything.

`GET /api/v1/dashboard/kpis`

`app/api/routes/dashboard.py:20`

- **Purpose:** Return the seven real-time KPI values shown as tiles on the Executive Dashboard — every number a live database aggregate, never hardcoded. See `get_kpis()` (`app/core/dashboard.py`) and the Architecture chapter's Executive Dashboard subsection for exactly how each KPI is defined.
- **Auth:** `require_permission("dashboard:read")` — granted to all three roles (admin, analyst, viewer).
- **Request body:** none.
- **Response:** `{active_investigations: int, critical_high_risk_iocs: int, open_cases: int, open_critical_cases: int, avg_threat_score: float, provider_health_percentage: float, ai_success_rate: float | None}`. `ai_success_rate` is `null` (not `0`) when there is no qualifying AI activity in the lookback window, rather than misrepresenting "no data" as a 0% success rate.
- **Errors:** none explicit.

`GET /api/v1/dashboard/executive-summary`

`app/api/routes/dashboard.py:25`

- **Purpose:** Return an AI-generated (or, on any AI failure, deterministic template-fallback) 2–4 sentence executive narrative, grounded exclusively in the same seven KPI values `GET /api/v1/dashboard/kpis` returns — the AI, or its fallback, is only ever handed those values as given facts and never computes or restates a number independently. See `generate_executive_summary()` (`app/ai/dashboard_summary.py`).
- **Auth:** `require_permission("dashboard:read")` — reuses the same permission as `/dashboard/kpis`; no new permission was introduced for this route.
- **Request body:** none.
- **Response:** `{summary: str, source: "ai" | "template_fallback", kpis: <the same object GET /api/v1/dashboard/kpis returns>}`. `source` honestly reports which path produced the text: `"template_fallback"` on any AI failure that survives one validation retry (an unreachable AI backend, or a structured response that fails schema validation twice) — the fallback sentence is still built directly from the real KPI numbers, never an error message and never fabricated data.
- **Errors:** none explicit — an AI failure is reported via `source: "template_fallback"` in a `200` response, not an HTTP error status.

Out-of-Scope Endpoints

Two endpoints exist in `app/main.py` directly, outside `app/api/routes/`, with no `/api/v1` prefix and no permission check:

- `GET /health` (`main.py:63-65`) — liveness probe, no authentication.
- `GET /metrics` (`main.py:36`) — Prometheus metrics, exposed by `prometheus-fastapi-instrumentator`, no authentication.

Both are excluded from the router/permission inventory above because they are not defined in `app/api/routes/*.py` and carry no RBAC gate, but are noted here so this reference remains a complete map of every HTTP-reachable route in the backend process.

Summary: Endpoint and Permission Counts

Router file	Endpoints	Distinct permission strings used
`auth.py`	4	none (public) / implicit `get_current_user` on `/me`
`lookup.py`	5	`lookup:create`, `lookup:read`
`providers.py`	2	`dashboard:read`, `provider:manage`
`ai_config.py`	2	`provider:manage`
`analysis.py`	10	`evidence:read`, `analysis:generate`, `copilot:query`
`hunting.py`	2	`hunting:generate`
`pivot.py`	1	`lookup:read`
`basket.py`	5	`basket:manage`
`cases.py`	8	`case:read`, `case:create`, `case:write`, `case:close`
`runtime.py`	10	`provider:manage`, `lookup:read`, `audit:read`
`dashboard.py`	2	`dashboard:read`
Total	**51**	**14 distinct permission strings**

Database Reference

This chapter is the exhaustive column-level reference for the platform's PostgreSQL schema: every table, every column with its type/nullability/default, every foreign key, and every native enum. It is a companion to the narrative "Database Architecture" chapter elsewhere in this documentation set, which explains *why* Postgres is the sole system of record and what Redis/Neo4j/OpenSearch do and do not do; this chapter only documents *what the schema is*, table by table, as defined under `backend/app/models/` and applied by the four Alembic migrations in `backend/alembic/versions/`.

The schema was verified by reading every model file and cross-checking it column-by-column against the applied migration DDL. The migration history is a single linear chain with one head — `660d2aa3bc20` → `a6d3ad2bb63c` → `0f2dc283823e` → `2652d888a33f` — and no drift was found between the ORM models and the DDL they generated: every column, enum, foreign key, and index in the migrations matches the current model files exactly. There are **14 tables** in total.

Conventions Used Throughout This Schema

Two shared mixins (`backend/app/models/base.py:10-27`) are applied to almost every table:

- **`UUIDPrimaryKeyMixin`** (`base.py:14-17`) — a single `id` column, type `UUID`, primary key. The value is generated in Python (`uuid.uuid4()`) by the ORM at insert time, not by a Postgres `DEFAULT` — there is no database-side UUID generation function involved.
- **`TimestampMixin`** (`base.py:20-26`) — `created_at` and `updated_at`, both `TIMESTAMPTZ NOT NULL`, both with a genuine database-level `server_default=now()`. `updated_at` additionally has an ORM-side `onupdate=func.now()`, which means the refresh on update is applied by SQLAlchemy in the application layer, not by a Postgres trigger — a row updated by raw SQL outside the ORM would not have `updated_at` bumped automatically.

Every table below carries both mixins **except** **`config_audit_log`**, which uses only `UUIDPrimaryKeyMixin` and defines its own explicit `timestamp` column instead (with no server default — the application must set it).

Two structural facts apply platform-wide and are easy to miss when reading the model files in isolation:

- **No foreign key in any migration specifies `ondelete=`.** At the database level, every FK below is effectively `ON DELETE NO ACTION` (RESTRICT-like behavior). Deleting a row that is still referenced by a child table will be rejected by Postgres unless the application has first deleted or reassigned the children.
- **`cascade="all, delete-orphan"` on `IOCLookup` and `Case` relationships is ORM-only.** SQLAlchemy will delete child rows (provider results, evidence, case notes, etc.) when the parent object is deleted *through the ORM session*, but this cascade is not expressed as a database constraint and has no effect on deletes issued outside the ORM (raw SQL, a different application, a database console).

Python-side `default=` values (as opposed to `server_default=`) are likewise applied by the ORM on `INSERT`, not baked into the column's DDL `DEFAULT` — they are noted as "ORM `<value>`" in the tables below to keep that distinction visible.

Entity-Relationship Overview

No diagram-generation tool was used for this chapter; the description below is a deliberately textual substitute for an ER diagram, built only from the foreign keys and constraints actually present in the models and migrations (no field is inferred or assumed).

The schema has one hub table per functional domain, plus one standalone administrative domain:

- **`users` is the root identity table.** It has no foreign keys of its own and is referenced by nine other tables as the actor, owner, or author of some action: `ioc_lookups.requested_by`, `final_assessment_records.requested_by`, `basket_items.owner_id`, `cases.analyst_id`, `case_iocs.added_by`, `case_notes.author_id`, `case_reports.generated_by`, `provider_runtime_configs.updated_by`, and `config_audit_log.actor_user_id`.
- **`ioc_lookups` is the hub of the investigation domain.** One lookup fans out to child rows in five other tables, all via a `lookup_id` foreign key back to `ioc_lookups.id`: `provider_results` (raw per-provider responses), `ai_summaries` (AI-generated per-provider and consolidated summaries), `correlation_edges` (the relationship graph for that lookup), `evidence_items` (the deterministic evidence ledger), and `final_assessment_records` (the full history of AI final-assessment runs against that lookup, not just the primary one). Two further tables hold a nullable, non-owning reference *into* this hub rather than owning rows within it: `basket_items.latest_lookup_id` and `case_iocs.lookup_id` — a basket item or case IOC can exist and be queried whether or not a completed lookup has ever been linked to it.
- **`cases` is the hub of the case-management domain.** One case owns rows in three child tables via a `case_id` foreign key: `case_iocs`, `case_notes`, and `case_reports`. `case_iocs` additionally, and optionally, points back into the investigation domain via its nullable `lookup_id`.
- **`basket_items` is a leaf table**, owned by exactly one user (`owner_id`) and optionally pointing at one lookup (`latest_lookup_id`); it participates in no other relationship.
- **`provider_runtime_configs` and `config_audit_log` form a self-contained administrative domain.** Both reference `users` (as `updated_by` / `actor_user_id` respectively, both nullable) but are not referenced by, and do not reference, any table in the investigation or case domains. This is the storage layer behind the platform's runtime (no-restart) provider/AI credential configuration and its audit trail.

Put simply: `users` sits at the root of ownership for every domain; `ioc_lookups` is the one-to-many parent for everything a single investigation produces; `cases` is the one-to-many parent for everything an analyst attaches to a multi-IOC case; and the runtime-configuration tables are an independent branch that only touches `users`, never the investigation or case data itself.

Authentication and Identity

`users`

Source: `app/models/user.py:17-27` · Introduced in migration `660d2aa3bc20` (`initial_schema.py:22-33`).

Application accounts and their RBAC role. This is the only root table in the schema — it has no foreign keys of its own.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK, ORM `uuid.uuid4()`	PK
`email`	VARCHAR(255)	NO	—	unique index `ix_users_email`
`hashed_password`	VARCHAR(255)	NO	—	—
`full_name`	VARCHAR(255)	NO	ORM `""`	—
`role`	ENUM `role`	NO	ORM `Role.ANALYST`	—
`is_active`	BOOLEAN	NO	ORM `True`	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

Enum `Role` (`user.py:11-14`): `admin` , `analyst` , `viewer` . The permission matrix keyed off this enum (`ROLE_PERMISSIONS`) lives in the same file and is covered in the Security chapter, not repeated here.

Investigations and AI Output

This group of five tables holds everything produced by a single IOC investigation, from the raw provider responses through to the AI's final verdict, plus a durable history of every AI re-analysis run against that lookup.

`ioc_lookups`

Source: `app/models/lookup.py:37-66` · Introduced in `660d2aa3bc20` (`initial_schema.py:34-50`).

The central record of one investigation: the submitted IOC, its detected type, a lifecycle status, and — once complete — the AI's final verdict, scores, and full structured assessment.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`ioc_value`	VARCHAR(2048)	NO	—	index `ix_ioc_lookups_ioc_value` (non-unique)
`ioc_type`	VARCHAR(64)	NO	—	index `ix_ioc_lookups_ioc_type` (non-unique)
`status`	ENUM `lookupstatus`	NO	ORM `PENDING`	—
`requested_by`	UUID	YES	—	FK → `users.id`
`final_verdict`	ENUM `verdict`	YES	—	—
`risk_score`	FLOAT	YES	—	—
`confidence_score`	FLOAT	YES	—	—
`final_assessment`	JSONB	YES	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

Enum `LookupStatus` (`lookup.py:15-19`): `pending` , `running` , `completed` , `failed` .

Enum `Verdict` (`lookup.py:22-34`), 12 values: `highly_malicious` , `malicious` , `suspicious` , `unknown` , `likely_benign` , `benign` , `scanner` , `tor_exit_node` , `vpn` , `cdn` , `cloud_infrastructure` , `dormant_infrastructure` .

FK: `requested_by` → `users.id` (nullable — a lookup can in principle exist without a resolvable requester). ORM relationships (`lookup.py:55-66` , `cascade="all, delete-orphan"` , ORM-only per the note above): `provider_results` , `ai_summaries` , `correlation_edges` , `evidence_items` . `ioc_lookups` is referenced (via `lookup_id`) by `provider_results` , `ai_summaries` , `correlation_edges` , `final_assessment_records` , and `evidence_items` , and referenced non-owningly by `basket_items.latest_lookup_id` and `case_iocs.lookup_id` .

`provider_results`

Source: `app/models/lookup.py:69-84` · Introduced in `660d2aa3bc20` (`initial_schema.py:79-95`).

One row per provider call per lookup — the raw connector response.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`lookup_id`	UUID	NO	—	FK → `ioc_lookups.id`; index `ix_provider_results_lookup_id`
`provider_id`	VARCHAR(128)	NO	—	index `ix_provider_results_provider_id`
`provider_name`	VARCHAR(255)	NO	—	—
`category`	VARCHAR(64)	NO	—	—
`status`	VARCHAR(32)	NO	—	—
`data`	JSONB	NO	ORM `dict` (`{}`)	—
`source_url`	VARCHAR(2048)	YES	—	—
`error_message`	TEXT	YES	—	—
`latency_ms`	INTEGER	YES	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

No local enums; `status` / `category` are free-form strings mirroring the provider layer's own `ProviderStatus` / `ProviderCategory` Python enums, not database enum types. FK: `lookup_id` → `ioc_lookups.id`, back-populated on `IOCLookup.provider_results` (`lookup.py:84`).

`ai_summaries`

Source: `app/models/lookup.py:87-97` · Introduced in `660d2aa3bc20` (`initial_schema.py:51-62`).

AI-generated summary output. A nullable `provider_id` disambiguates a per-provider summary from the single consolidated summary for the lookup (consolidated rows have `provider_id = NULL`).

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`lookup_id`	UUID	NO	—	FK → `ioc_lookups.id`; index `ix_ai_summaries_lookup_id`
`provider_id`	VARCHAR(128)	YES (NULL = consolidated)	—	index `ix_ai_summaries_provider_id`
`summary`	JSONB	NO	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

FK: `lookup_id` → `ioc_lookups.id`, back-populated on `IOCLookup.ai_summaries` (`lookup.py:97`).

`correlation_edges`

Source: `app/models/lookup.py:100-118` · Introduced in `660d2aa3bc20` (`initial_schema.py:63-78`).

One graph edge produced by the correlation engine for a lookup. As documented in the Database Architecture chapter, this table is, in its entirety, where the "correlation graph" the UI renders actually lives — the Neo4j container referenced elsewhere in configuration is not written to.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`lookup_id`	UUID	NO	—	FK → `ioc_lookups.id`; index `ix_correlation_edges_lookup_id`
`source_type`	VARCHAR(64)	NO	—	—
`source_value`	VARCHAR(2048)	NO	—	—
`target_type`	VARCHAR(64)	NO	—	—
`target_value`	VARCHAR(2048)	NO	—	—
`relationship_type`	VARCHAR(64)	NO	—	—
`confidence`	FLOAT	NO	ORM `1.0`	—
`provenance`	VARCHAR(128)	NO	— (which provider asserted this edge)	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

FK: `lookup_id` → `ioc_lookups.id`, back-populated on `IOCLookup.correlation_edges` (`lookup.py:118`).

`final_assessment_records`

Source: `app/models/lookup.py:121-144` · Introduced in migration `2652d888a33f` (`add_final_assessment_records.py:22-36`), the current migration head.

A durable history of every final-assessment generation run for a lookup, not just the one currently reflected on `ioc_lookups`. This backs the platform's cross-AI-backend comparison feature (e.g. comparing a Groq run against an Ollama re-run over the same evidence); `IOCLookup.final_assessment` / `final_verdict` / `risk_score` remains the single "primary" record surfaced by default.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`lookup_id`	UUID	NO	—	FK → `ioc_lookups.id` ; index `ix_final_assessment_records_lookup_id`
`ai_backend`	VARCHAR(64)	NO	—	—
`ai_model`	VARCHAR(255)	YES	—	—
`is_primary`	BOOLEAN	NO	ORM `False`	—
`assessment`	JSONB	NO	—	—
`requested_by`	UUID	YES	—	FK → `users.id`
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

FKs: `lookup_id` → `ioc_lookups.id` ; `requested_by` → `users.id` . This model declares a bare `relationship()` with no `back_populates` (`lookup.py:144`) — `IOCLookup` does not expose a reverse collection for this table.

The Evidence Ledger

`evidence_items`

Source: `app/models/evidence.py:31-56` · Introduced in migration `a6d3ad2bb63c` (`add_evidence_basket_and_case_management_.py:96-117`).

The deterministic, independently-reproducible evidence ledger. Every AI-generated explanation elsewhere in the platform must cite an `evidence_id` back to one of these rows; rows here are built by `app/evidence/builder.py` from provider results and correlation edges and are never written directly by an AI call — this table is the platform's anti-hallucination grounding mechanism.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`lookup_id`	UUID	NO	—	FK → `ioc_lookups.id` ; index `ix_evidence_items_lookup_id`
`evidence_type`	ENUM `evidencetype`	NO	—	index `ix_evidence_items_evidence_type`
`source_label`	VARCHAR(255)	NO	— (e.g. `"VirusTotal"` , `"Correlation Engine"`)	—
`provider_id`	VARCHAR(128)	YES	— (NULL when the source is the correlation engine itself)	index `ix_evidence_items_provider_id`
`claim`	TEXT	NO	—	—
`interpretation`	TEXT	YES	—	—
`confidence`	FLOAT	NO	— (0-100 scale; no ORM default)	—
`related_ioc_type`	VARCHAR(64)	YES	—	—
`related_ioc_value`	VARCHAR(2048)	YES	—	—
`source_url`	VARCHAR(2048)	YES	—	—
`observed_at`	VARCHAR(64)	YES	— (provider-reported timestamp string, mixed formats, not parsed into a real timestamp type)	—
`raw_data`	JSONB	YES	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

Enum `EvidenceType` (`evidence.py:19-28`), 9 values: `detection` , `reputation` , `relationship` , `malware_association` , `threat_actor_association` , `campaign_association` , `mitre_technique` , `infrastructure` , `other` .

FK: `lookup_id` → `ioc_lookups.id` , back-populated on `IOCLookup.evidence_items` (`evidence.py:56`).

Case Management and the Analyst Workspace

`basket_items`

Source: `app/models/basket.py:16-29` · Introduced in `a6d3ad2bb63c` (lines 37-51).

A per-analyst scratch space of saved IOCs, used ahead of formal case creation. Deliberately carries no status/workflow fields of its own.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`owner_id`	UUID	NO	—	FK → `users.id` ; index `ix_basket_items_owner_id` ; part of composite unique `uq_basket_owner_ioc`
`ioc_value`	VARCHAR(2048)	NO	—	part of composite unique `uq_basket_owner_ioc`
`ioc_type`	VARCHAR(64)	NO	—	—
`latest_lookup_id`	UUID	YES	—	FK → `ioc_lookups.id`
`note`	VARCHAR(1024)	YES	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

Table-level constraint: `UniqueConstraint(owner_id, ioc_value, name="uq_basket_owner_ioc")` (`basket.py:18`) — enforces at most one basket row per owner per exact IOC value; this is a real database-level constraint, not just an application check. FKs: `owner_id` → `users.id` ; `latest_lookup_id` → `ioc_lookups.id` (nullable — an IOC can be basketed before any lookup has run).

`cases`

Source: `app/models/case.py:32-44` · Introduced in `a6d3ad2bb63c` (lines 22-36).

A multi-IOC investigation/incident container with its own status workflow, team-shared rather than per-analyst.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`title`	VARCHAR(255)	NO	—	—
`description`	TEXT	YES	—	—
`analyst_id`	UUID	NO	—	FK → `users.id` ; index `ix_cases_analyst_id`
`severity`	ENUM `caseseverity`	NO	ORM `MEDIUM`	—
`status`	ENUM `casestatus`	NO	ORM `OPEN`	index `ix_cases_status`
`tags`	ARRAY(VARCHAR(64))	NO	ORM `list` (`[]`)	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

Enum `CaseStatus` (`case.py:16-22`), 6 values: `open` , `investigating` , `contained` , `resolved` , `false_positive` , `closed` .

Enum `CaseSeverity` (`case.py:25-29`), 4 values: `low` , `medium` , `high` , `critical` .

FK: `analyst_id` → `users.id` . Relationships (`case.py:42-44` , `cascade="all, delete-orphan"` , ORM-only): `iocs` (`CaseIOC`), `notes` (`CaseNote`), `reports` (`CaseReport`).

`case_iocs`

Source: `app/models/case.py:47-58` · Introduced in `a6d3ad2bb63c` (lines 52-65).

One IOC attached to a case, optionally linked to the lookup that investigated it.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`case_id`	UUID	NO	—	FK → `cases.id` ; index `ix_case_iocs_case_id`
`ioc_value`	VARCHAR(2048)	NO	—	—
`ioc_type`	VARCHAR(64)	NO	—	—
`lookup_id`	UUID	YES	—	FK → `ioc_lookups.id`
`added_by`	UUID	NO	—	FK → `users.id`
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

FKs: `case_id` → `cases.id` ; `lookup_id` → `ioc_lookups.id` (nullable); `added_by` → `users.id` . Back-populated on `Case.iocs` (`case.py:58`).

`case_notes`

Source: `app/models/case.py:61-75` · Introduced in `a6d3ad2bb63c` (lines 67-79).

A free-text analyst note attached to a case, optionally "anchored" to a specific artifact by a loose string pair rather than a real foreign key.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`case_id`	UUID	NO	—	FK → `cases.id`; index `ix_case_notes_case_id`
`author_id`	UUID	NO	—	FK → `users.id`
`body`	TEXT	NO	—	—
`anchor_type`	VARCHAR(64)	YES	— (free string, e.g. `"ioc"`, `"evidence"`, `"graph_node"`, `"timeline_event"`, `"provider_result"` — not an enum, not a foreign key)	—
`anchor_ref`	VARCHAR(2048)	YES	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

FKs: `case_id` → `cases.id`; `author_id` → `users.id`. Back-populated on `Case.notes` (`case.py:75`). Because `anchor_type`/`anchor_ref` are plain strings, the database performs no referential-integrity check on what a note is anchored to — an anchor referencing a since-deleted evidence item, for example, would not be caught by a database constraint.

`case_reports`

Source: `app/models/case.py:78-88` · Introduced in `a6d3ad2bb63c` (lines 81-95).

A generated report artifact attached to a case.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`case_id`	UUID	NO	—	FK → `cases.id`; index `ix_case_reports_case_id`
`report_type`	VARCHAR(64)	NO	— (matches a `ReportType` literal defined in `app/ai/schemas.py`; not a database enum)	—
`title`	VARCHAR(255)	NO	—	—
`content_markdown`	TEXT	NO	—	—
`generated_by`	UUID	NO	—	FK → `users.id`
`context`	JSONB	YES	—	—
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

FKs: `case_id` → `cases.id`; `generated_by` → `users.id`. Back-populated on `Case.reports` (`case.py:88`). The table exists and is fully wired at the schema level; no route or service function in `app/api/routes/cases.py` currently inserts a row here, so it should be read as a defined-but-unpopulated table today (verified against the endpoint inventory: `cases.py` exposes create/list/get/update/close/add-IOC/remove-IOC/add-note, and no report-generation endpoint).

Runtime Configuration and Audit

These two tables are the persistence layer behind the platform's DB-backed, hot-swappable AI/provider credential system, which lets an administrator reconfigure AI backends and IOC providers without editing `.env` or restarting containers.

`provider_runtime_configs`

Source: `app/models/runtime_config.py:30-55` · Introduced in migration `0f2dc283823e` (`add_provider_runtime_config_and_audit_.py:32-51`).

One row per `(kind, provider_id)` pair — every AI backend and every IOC provider the platform knows about. Credentials are Fernet-encrypted before storage; see the Security chapter for the encryption mechanism.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`kind`	ENUM `providerkind`	NO	—	part of composite unique `uq_provider_runtime_kind_id`
`provider_id`	VARCHAR(64)	NO	—	part of composite unique `uq_provider_runtime_kind_id`
`provider_name`	VARCHAR(255)	NO	ORM `""`	—
`enabled`	BOOLEAN	NO	ORM `True`	—
`is_active`	BOOLEAN	NO	ORM `False`	— (meaningful only for `kind=AI`; "exactly one active AI row" is enforced by application code, not a database constraint)
`encrypted_credentials`	TEXT	YES	— (Fernet ciphertext of a JSON object, e.g. `{"api_key": "..."}`)	—
`model_id`	VARCHAR(255)	YES	—	—
`extra_config`	JSONB	YES	— (non-secret extras, e.g. a custom endpoint URL)	—
`last_test_at`	TIMESTAMPTZ	YES	—	—
`last_test_ok`	BOOLEAN	YES	—	—
`last_test_message`	VARCHAR(500)	YES	—	—
`updated_by`	UUID	YES	—	FK → `users.id`
`created_at` / `updated_at`	TIMESTAMPTZ	NO	server `now()`	—

Enum `ProviderKind` (`runtime_config.py:25-27`): `ai`, `ioc`.

Table-level constraint: `UniqueConstraint(kind, provider_id, name="uq_provider_runtime_kind_id")` (`runtime_config.py:32`) — a real database constraint preventing duplicate rows for the same backend/provider. FK: `updated_by` → `users.id` (nullable).

`config_audit_log`

Source: `app/models/runtime_config.py:58-75` · Introduced in `0f2dc283823e` (`add_provider_runtime_config_and_audit_.py:22-31`).

An append-only audit trail of runtime-config changes. This is the one table in the schema that intentionally omits `TimestampMixin` — it uses only `UUIDPrimaryKeyMixin` and defines its own `timestamp` column, with no `created_at` / `updated_at`.

Column	Type	Null	Default	Index / Unique
`id`	UUID	NO	PK	PK
`timestamp`	TIMESTAMPTZ	NO	— (no server default; the application must set this explicitly on insert)	—
`actor_user_id`	UUID	YES	—	FK → `users.id`
`actor_email`	VARCHAR(255)	YES	— (a denormalized copy of the actor's email, kept so the log stays human-readable even if the user account is later deleted)	—
`action`	VARCHAR(100)	NO	—	—
`detail`	VARCHAR(1000)	NO	ORM `""`	—

No enums. FK: `actor_user_id` → `users.id`, but note this model declares no `relationship()` back to `User` — it is an informational foreign key only, not used for ORM-level joins. As documented in the Security chapter, `detail` is restricted by application convention to human-readable descriptive text; it is not a field the write path ever populates with a raw credential value.

Enum Inventory

Every native Postgres `ENUM` type in the schema, named as the lowercase of its Python class (no model overrides this with an explicit `name=`):

DB enum name	Python class	Values
`role`	`Role`	admin, analyst, viewer
`lookupstatus`	`LookupStatus`	pending, running, completed, failed
`verdict`	`Verdict`	highly_malicious, malicious, suspicious, unknown, likely_benign, benign, scanner, tor_exit_node, vpn, cdn, cloud_infrastructure, dormant_infrastructure
`evidencetype`	`EvidenceType`	detection, reputation, relationship, malware_association, threat_actor_association, campaign_association, mitre_technique, infrastructure, other
`casestatus`	`CaseStatus`	open, investigating, contained, resolved, false_positive, closed
`caseseverity`	`CaseSeverity`	low, medium, high, critical
`providerkind`	`ProviderKind`	ai, ioc

Migration Provenance

Migration file	Revision → down_revision	Tables created
`660d2aa3bc20_initial_schema.py`	`660d2aa3bc20` → (root)	`users`, `ioc_lookups`, `ai_summaries`, `correlation_edges`, `provider_results`
`a6d3ad2bb63c_add_evidence_basket_and_case_management_.py`	`a6d3ad2bb63c` → `660d2aa3bc20`	`cases`, `basket_items`, `case_iocs`, `case_notes`, `case_reports`, `evidence_items`
`0f2dc283823e_add_provider_runtime_config_and_audit_.py`	`0f2dc283823e` → `a6d3ad2bb63c`	`config_audit_log`, `provider_runtime_configs`
`2652d888a33f_add_final_assessment_records.py` (head)	`2652d888a33f` → `0f2dc283823e`	`final_assessment_records`

The chain is linear with a single head at `2652d888a33f`. No table or column exists in the ORM models that is missing a corresponding migration, and no migration DDL was found that lacks a corresponding model — the schema described in this chapter is, at the time of writing, exactly the schema Alembic would produce by replaying all four migrations against an empty database.

Provider and AI Backend Integration Reference

This chapter is the exhaustive technical reference for every external integration the backend calls: the 18 registered IOC (Indicator of Compromise) providers in `backend/app/providers/` and the 11 interchangeable AI backends in `backend/app/ai/`. Where the *Provider Architecture* and *AI Architecture* chapters explain how the fan-out, caching, credential-override, and grounding mechanisms work, this chapter documents **what each individual integration actually calls** — real endpoint URLs, auth schemes, credential fields, supported IOC types, rate-limit/error detection, and the shape of the normalized data each one returns. All facts are drawn directly from the connector source files cited inline; nothing below is inferred from documentation or docstrings alone.

1. Shared Provider Contract

Every provider subclasses `BaseProvider` (`app/providers/base.py:80-184`), which wraps `fetch()` with three short-circuits before any outbound call is made — disabled, unsupported IOC type, and not-configured (`base.py:97-146`) — and normalizes every outbound-call exception into one of a fixed set of `ProviderStatus` values. Two mappings are uniform across **all** real HTTP-based providers unless a connector explicitly overrides them:

- `httpx.HTTPStatusError` with status **429, 403, or 509** → `ProviderStatus.RATE_LIMITED` (`base.py:149-155` ; 509 is called out at `base.py:152` as PhishTank's documented over-limit code, since it is not a widely-recognized rate-limit status elsewhere).
- Any other non-2xx status raised via `response.raise_for_status()` → `ProviderStatus.ERROR` .

Retries and timeouts are applied one layer above, in the orchestrator (`app/providers/orchestrator.py:26,47-61`): `asyncio.wait_for` per call, with `tenacity` -based retry limited to `httpx.ConnectError` , `httpx.ReadTimeout` , and `httpx.PoolTimeout` — a bad-status response is never retried, only a genuinely failed connection. Sixteen providers are registered in `app/providers/registry.py:29-46` , including the OSINT crawler described in §3.

2. IOC Providers, One by One

2.1 VirusTotal — `virustotal`

- **File**: `app/providers/virustotal.py` . Category `THREAT_INTEL` . IOC types: `ipv4` , `ipv6` , `domain` , `url` , and every hash type (`md5` / `sha1` / `sha256` / `sha512`) (`:17-20`).
- **Credential**: `requires_key=True` ; field `api_key` → `settings.virustotal_api_key` , sent as header `x-apikey` (`:21,24-26,44-45`).
- **Endpoint**: base `https://www.virustotal.com/api/v3` (`:22`); per-type path — `/files/{hash}` , `/ip_addresses/{ip}` , `/domains/{domain}` , `/urls/{base64url(ioc)}` (`:28-40`).
- **Normalized data**: `verdict` (derived from `last_analysis_stats`), `detection_ratio` , `malicious_count` , `suspicious_count` , `total_engines` , `reputation_score` , `last_analysis_date` , `tags` , plus type-specific fields — hashes get `file_type` / `file_names` / `md5` / `sha1` / `sha256` / `first_seen` / `last_seen` / `malware_families` ; IPs get `asn` / `as_owner` / `country` / `network` ; domains get `resolved_ips` / `registrar` / `creation_date` ; URLs get `final_url` / `title` (`:63-133`).
- **Errors**: HTTP 404 → `NO_DATA` (`:49-58`); everything else falls through to the shared 429/403/509 mapping.

2.2 AbuseIPDB — `abuseipdb`

- **File**: `app/providers/abuseipdb.py` . Category `THREAT_INTEL` . IOC types: `ipv4` , `ipv6` (`:16-19`).
- **Credential**: `requires_key=True` ; field `api_key` → `settings.abuseipdb_api_key` , header `Key` (`:20,25,29-30`).
- **Endpoint**: base `https://api.abuseipdb.com/api/v2` , `GET /check` (`:21,33`).
- **Normalized data**: `verdict` (malicious if abuse-confidence score > 25, else derived from report count), `reputation_score` / `abuse_confidence_score` , `total_reports` , `num_distinct_users` , `is_whitelisted` , `usage_type` , `country_code` , `isp` , `domain` , `hostnames` , `is_tor` , `last_seen` , `reports` (`:60-85`).
- **Errors**: HTTP 404 → `NO_DATA` (`:34-43`); an empty result entry is also mapped to `NO_DATA` (`:48-58`).

2.3 AlienVault OTX — `otx`

- **File**: `app/providers/otx.py` . Category `THREAT_INTEL` . IOC types: `ipv4` , `ipv6` , `domain` , `hostname` , `url` , plus all hash types (`:27-30`).
- **Credential**: `requires_key=True` ; field `api_key` → `settings.otx_api_key` , header `X-OTX-API-KEY` (`:31,36,46-47`).
- **Endpoint**: base `https://otx.alienvault.com/api/v1` ; `GET /indicators/{section}/{indicator}/general` , where `section` is mapped per IOC type (`:17-23,32,38-50`).
- **Normalized data**: `verdict` (malicious if `pulse_count > 0` , else unknown), `pulse_count` , `malware_families` , `threat_actors` , `campaigns` , `tags` , `references` , `reputation` , `first_seen` / `last_seen` , plus type-specific fields (IP: `asn` / `country` ; domain/hostname: `alexa` / `whois` ; URL: `resolved_domains` / `hostname` ; hash: `file_type`) (`:104-164`).
- **Errors**: 404 → `NO_DATA` (`:53-62`); zero pulses is *also* forced to `NO_DATA` rather than a "clean" verdict (`:70-80`).

2.4 URLhaus (abuse.ch) — `urlhaus`

- **File**: `app/providers/urlhaus.py` . Category `THREAT_INTEL` . IOC types: `url` , `domain` , `ipv4` (`:18-21`).
- **Credential**: `requires_key=True` ; field `auth_key` → `settings.abusech_auth_key` , header `Auth-Key` (`:22,27,31`) — this key is shared with ThreatFox and MalwareBazaar below (one abuse.ch key covers all three).
- **Endpoint**: base `https://urlhaus-api.abuse.ch/v1` ; `POST /url/` (form field `url`) for URL lookups, `POST /host/` (form field `host`) for domain/IPv4 lookups (`:23,33-38`).
- **Normalized data**: URL lookup returns `verdict:"malicious"` , `query_status` , `url_status` , `threat` , `tags` , `host` , `first_seen` / `last_seen` , `related_hashes` , `payloads` , `blacklists` (`:84-99`); host lookup returns `verdict` (malicious if any URLs found, else unknown), `query_status` , `url_count` , `first_seen` , `related_urls` , `threat` , `tags` , `blacklists` (`:101-116`).
- **Errors**: 404 → `NO_DATA` ; abuse.ch's own `query_status` field is interpreted by the shared `map_query_status()` helper (`app/providers/abusech.py:15,18-29`): `"ok"` → OK, `{"no_api_key","invalid_api_key","unauthorized"}` → `ERROR` (key rejected), anything else → `NO_DATA` (`urlhaus.py:55-67`).

2.5 ThreatFox (abuse.ch) — `threatfox`

- **File**: `app/providers/threatfox.py` . Category `THREAT_INTEL` . IOC types: `ipv4` , `ipv6` , `domain` , `url` , `md5` , `sha256` (`:18-28`).

- **Credential**: same shared abuse.ch key (`auth_key` , header `Auth-Key`) (`:29,34,38`).
- **Endpoint**: base `https://threatfox-api.abuse.ch/api/v1/` ; `POST` JSON body `{"query":"search_ioc","search_term":...}` (`:30,39-41`).
- **Normalized data**: `verdict:"malicious"` , `ioc_type` , `threat_type` (single value or list), `malware_families` , `confidence_level` (max across matches), `first_seen` / `last_seen` , `tags` , `reference` , `matches` (raw matched entries) (`:88-111`).
- **Errors**: 404 → `NO_DATA` ; `query_status` handled via the same shared `map_query_status()` (`:42-72`).

2.6 MalwareBazaar (abuse.ch) — `malwarebazaar`

- **File**: `app/providers/malwarebazaar.py` . Category `THREAT_INTEL` . IOC types: all hash types — `md5` / `sha1` / `sha256` / `sha512` (`:18-21`).
- **Credential**: same shared abuse.ch key (`auth_key` , header `Auth-Key`) (`:22,27,31`).
- **Endpoint**: base `https://mb-api.abuse.ch/api/v1/` ; `POST` form `{"query":"get_info","hash":...}` (`:23,32-34`).
- **Normalized data**: `verdict:"malicious"` , `file_type` / `file_type_mime` / `file_size` / `file_name` , `malware_families` (from the `signature` field), `tags` , `delivery_method` , `first_seen` / `last_seen` , `md5` / `sha1` / `sha256` , `downloads` / `uploads` (from `intelligence`), `vendor_intel` (`:82-105`).
- **Errors**: 404 → `NO_DATA` ; `query_status` via `map_query_status()` , with an additional rule that "ok" plus zero returned entries is forced to `NO_DATA` (`:48-65`).

2.7 crt.sh — `crtsh`

- **File**: `app/providers/crtsh.py` . Category `CERTIFICATE_INTEL` . IOC types: `domain` , `tls_certificate` (`:27-30`).
- **Credential**: `requires_key=False` , always configured (`:31,34-36`).
- **Endpoint**: base `https://crt.sh` ; `GET /?q={domain}&output=json` (`:32,43-45`).
- **Normalized data**: `certificates` (up to the 25 most recent, each with `issuer_name` , `common_name` , `name_value` — sorted, lowercased names — `not_before` / `not_after` , `serial_number` , `id` , `entry_timestamp`), `certificate_count` (total entries returned), `related_domains` (subdomains extracted from SAN names) (`:100-138`).
- **Errors**: 404 → `NO_DATA` (`:46-55`). Notably, malformed or non-list JSON is caught and degraded to `NO_DATA` rather than raised — the code comment explains crt.sh is known to be slow/flaky and to occasionally emit malformed JSON (`:58-84`).

2.8 NIST NVD — `nvd`

- **File**: `app/providers/nvd.py` . Category `VULNERABILITY` . IOC type: `cve` (`:18-21`).
- **Credential**: `requires_key=False` — works unauthenticated at roughly 5 requests/30s; an optional `api_key` → `settings.nvd_api_key` , sent as header `apiKey` , raises the ceiling to roughly 50 requests/30s (`:22,27-28,32-35`).
- **Endpoint**: base `https://services.nvd.nist.gov/rest/json/cves/2.0` ; `GET` with `params={"cveId":...}` (`:23,37`).
- **Normalized data**: `verdict` (derived from CVSS severity), `cve_id` , `cvss_score` / `cvss_severity` / `cvss_vector` (CVSS v3.1 preferred, then v3.0, then v2; "Primary" scoring source preferred when multiple exist, `:80-93`), `description` , `vuln_status` , `cwes` , `references` , `configurations` , `first_seen` (published)/ `last_seen` (lastModified) (`:106-137`).
- **Errors**: 404 → `NO_DATA` ; an empty `vulnerabilities` list → `NO_DATA` ; other statuses fall through to the shared mapping (`:38-62`).

2.9 CISA Known Exploited Vulnerabilities — `cisa_kev`

- **File**: `app/providers/cisa_kev.py` . Category `VULNERABILITY` . IOC type: `cve` (`:26-29`).
- **Credential**: `requires_key=False` , always configured (`:30,33-35`).
- **Endpoint**: fetches the full catalog JSON once from `https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json` and caches it **in-memory** for up to 3600 seconds, guarded by an `asyncio.Lock` so concurrent lookups don't trigger duplicate fetches (`:17-22,37-55`).
- **Normalized data**: `verdict:"malicious"` (a CVE only appears here if it is confirmed actively exploited), `vulnerability_name` , `vendor_project` , `product` , `date_added` , `short_description` , `required_action` , `due_date` , `known_ransomware_campaign_use` , `notes` , `first_seen` (`:72-84`).
- **Errors**: catalog-fetch failures propagate via `raise_for_status()` ; a CVE absent from the catalog → `NO_DATA` (`:61-70`).

2.10 MITRE ATT&CK — `mitre_attack`

- **File**: `app/providers/mitre_attack.py` . Category `THREAT_INTEL` . IOC type: `mitre_technique` (`:71-74`).
- **Credential**: `requires_key=False` , `configured=True` always (`:75-76`).
- **Endpoint**: downloads the full ~30MB STIX 2.1 enterprise-attack bundle from `https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/enterprise-attack.json` , cached for 3600 seconds behind an `asyncio.Lock` (`:26-31,46-67`).
- **Normalized data**: `technique_id` , `name` , `description` , `tactics` (sorted kill-chain phase names), `mitre_techniques` , `x_mitre_platforms` , `x_mitre_data_sources` , `is_subtechnique` , `revoked` , `deprecated` (`:101-112`).
- **Errors**: a technique ID absent from the bundle's index → `NO_DATA` (`:86-95`); bundle-fetch errors fall through to `raise_for_status()` (`:62`).

2.11 WHOIS / RDAP — `whois_rdap`

- **File**: `app/providers/whois_rdap.py` . Category `WHOIS` . IOC types: `domain` (via WHOIS), `ipv4` / `ipv6` / `asn` (via RDAP) (`:43-46`).
- **Credential**: `requires_key=False` (`:47,50-51`).
- **Mechanism**: domain lookups run the blocking `python-whois` library (`pywhois.whois`) inside `asyncio.to_thread` , with a 10-second socket timeout (`:36,60-67`). IP/ASN lookups use RDAP against `https://rdap.org/ip/{ip}` or `.../autnum/{asn}` , which bootstraps to whichever Regional Internet Registry actually holds the record (`:38,141-153`).
- **Normalized data**: domain — `registrar` , `creation_date` / `expiration_date` / `updated_date` (ISO strings), `name_servers` , `registrant` , `registrant_country` , `status` , `emails` , `first_seen` / `last_seen` (`:113-125`). RDAP — `handle` , `name` , `country` , `start_address` / `end_address` , `start_autnum` / `end_autnum` , `ip_version` , `type` , `status` , `entities` (mapped vCard roles), `registrant` , `asn` (`:176-193`).
- **Errors**: a socket-level failure maps to `TIMEOUT` for `socket.timeout` or `ERROR` for anything else (connection refused/reset, `gaierror`) (`:69-81`); an unparseable TLD WHOIS response (`PywhoisError`) → `NO_DATA` (`:82-94`); a parsed record with no `domain_name` field → `NO_DATA` (`:96-105`); RDAP 404 → `NO_DATA` (`:154-163`).

2.12 Hybrid Analysis (Falcon Sandbox) — `hybrid_analysis`

- **File**: `app/providers/stubs/hybrid_analysis.py`. Category `SANDBOX`. IOC type: **`sha256` only** — per the module docstring, MD5/SHA1 are unsupported because the replacement endpoint accepts only SHA256, and URL support was removed because the correct request shape for it could not be confirmed (`:1-17,28-31`).
- **Credential**: `requires_key=True`; field `api_key` → `settings.hybrid_analysis_api_key`, header `api-key`; also always sends `user-agent: "Falcon Sandbox"` because Hybrid Analysis rejects generic user agents (`:32,40,44-49`).
- **Endpoint**: base `https://hybrid-analysis.com/api/v2` (bare domain, confirmed live — `www.` 301-redirects) (`:33-36`); `GET /overview/{sha256}/summary` (`:52`).
- **Normalized data**: `verdict`, `threat_score`, `av_detect_pct` (from `multiscan_result`), `sha256`, `submitted_at`, `first_seen` (= submitted_at) / `last_seen` (= last_multi_scan) (`:80-90`).
- **Errors**: 400 or 404 → `NO_DATA` (`:53-62`); other statuses fall through to the shared mapping.

2.13 Spamhaus DBL/ZEN — `spamhaus`

- **File**: `app/providers/stubs/spamhaus.py`. Category `THREAT_INTEL`. IOC types: `ipv4`, `domain` (`:54-57`).
- **Credential**: `requires_key=False`, `configured=True` (`:58-59`).
- **Mechanism**: this connector makes **no HTTP call at all** — it performs plain DNS A-record lookups via `loop.getaddrinfo()` against `*.zen.spamhaus.org` (IP, octet-reversed) or `*.dbl.spamhaus.org` (domain) (`:61-76`).
- **Normalized data**: `verdict` ("clean" if NXDOMAIN/no answer, "malicious" if any A-record hit that isn't a query-error code, "unknown" if every returned code is a query-error code), `listed` (bool), `list` (`"ZEN"` or `"DBL"`), `query`, and on a real listing, `return_codes` plus `listing_reason` (decoded from the published Spamhaus ZEN/DBL return-code tables) (`:81-131`).
- **"Errors"**: `socket.gaierror` (NXDOMAIN) is caught and treated as "not listed" — a normal clean result, not an error. Because there is no HTTP call, the 429/403/509 rate-limit mapping described in §1 does not apply to this connector.
- **Real bug fixed (v0.2.2)**: Spamhaus's two shared query-error codes (`127.255.255.254` "public/open resolver not permitted", `127.255.255.255` "excessive queries, temporarily blocked") were only documented in `_ZEN_CODES`, and — more importantly — were never actually distinguished from a real listing in the classification logic itself: ANY non-empty DNS answer, including these, was reported as `verdict: "malicious"`. Confirmed live against a real Docker container (a common deployment shape, not unique to any one environment): every domain lookup got `127.255.255.254` back (Spamhaus rejecting the container's default DNS resolver as public/shared), so every domain — including `example.org`, IANA's reserved, universally-benign example domain — was reported malicious with zero real finding behind it. Fixed with a `_QUERY_ERROR_CODES` set checked before classification: a response containing ONLY error codes now reports `status=ERROR`, `verdict="unknown"`, `listed=False`, with a clear `error_message` explaining the query was rejected; a real listing found alongside an error code still correctly reports `malicious` (the real evidence isn't discarded just because an unrelated error code was also present). `_DBL_CODES` also gained the two error codes for documentation completeness, confirmed live to appear on DBL (domain) lookups too, not only ZEN (IP) lookups as previously assumed.

2.14 PhishTank — `phishtank`

- **File**: `app/providers/stubs/phishtank.py`. Category `THREAT_INTEL`. IOC type: `url` (`:19-22`).
- **Credential**: `requires_key=False` — the key is optional and only raises the rate limit; field `api_key` → `settings.phishtank_api_key`, sent as form field `app_key` when present (`:23-24,28-32`).
- **Endpoint**: base `https://checkurl.phishtank.com/checkurl/`; `POST` form `{"url":..., "format":"json"}` (`:25,34`).
- **Normalized data**: `verdict` (malicious if `valid`, else suspicious), `in_database`, `valid`, `verified`, `verified_at`, `submission_time`, `phish_id`, `phish_detail_page` (`:64-73`).
- **Errors**: 404 → `NO_DATA`; `results.in_database == False` → `NO_DATA`; other non-2xx statuses fall through to the shared mapping — this is the connector that motivated including HTTP **509** in the shared rate-limit set, since it is PhishTank's documented over-limit status code (`:35-59`, `base.py:152`).

2.15 Censys — `censys`

- **File**: `app/providers/stubs/censys.py`. Category `PASSIVE_DNS`. IOC types: `ipv4`, `ipv6` (`:24-27`).
- **Credential**: `requires_key=True`; **both** a Personal Access Token (Bearer auth) and an Organization ID (header `X-Organization-ID`) are required for `configured=True` — either alone is not sufficient (`:28,33-36,40-46`).
- **Endpoint**: base `https://platform.censys.io`; `GET /v3/global/asset/host/{ip}` (`:29,49-51`).
- **Normalized data**: `verdict:"unknown"` (Censys is a passive-DNS/asset-inventory source, not a verdict provider), `services` (list of `port` / `protocol` / `service_name`), `location` (`city` / `province` / `country` / `country_code`), `autonomous_system`, `asn`, `as_owner`, `last_seen` (`:78-91`).
- **Errors**: 404 → `NO_DATA`; other statuses fall through to the shared mapping.

2.16 urlscan.io — `urlscan`

- **File**: `app/providers/urlscan_io.py`. Category `SANDBOX`. IOC types: `url`, `domain` (`:39-41`).
- **Credential**: `requires_key=True`; field `api_key`, header `API-Key` (`:41,45`). Not offered on either installer's setup wizard (§ below) — configured after install from the app's own Providers page.
- **Endpoint**: `POST https://urlscan.io/api/v1/scan/` to submit, then `GET https://urlscan.io/api/v1/result/{uuid}/` polled every 3 seconds until ready or a 60-second wall-clock timeout (`_TIMEOUT_SECONDS` / `_POLL_INTERVAL_SECONDS`) elapses — a real, live sandbox run per lookup, not a cached-database check like most other providers in this chapter.
- **Errors**: the poll endpoint returns HTTP 404 while the scan is still processing (expected, not an error) and HTTP 200 with the full result once ready; 401/403 on either the submit or poll request are treated as "bad API key," not rate limiting — deliberately not routed through `BaseProvider.run()`'s generic mapping (which treats 403 as `RATE_LIMITED` for every other connector, matching PhishTank's documented behavior) because urlscan.io's own 403 semantics are different. A scan not ready by the timeout → `ProviderStatus.TIMEOUT`, never a fabricated result from an incomplete scan.

2.17 Google Safe Browsing — `google_safe_browsing`

- **File**: `app/providers/google_safe_browsing.py`. Category `THREAT_INTEL`. IOC types: `url`, `domain` (`:31-33`).
- **Credential**: `requires_key=True`; field `api_key`, sent as the `key` query parameter on the Lookup API v4 call (`:33`). Not offered on either installer's setup wizard (§ below) — configured after install from the app's own Providers page.
- **Endpoint**: `POST https://safebrowsing.googleapis.com/v4/threatMatches:find?key=<API_KEY>`, checking against `MALWARE`, `SOCIAL_ENGINEERING`, `UNWANTED_SOFTWARE`, and `POTENTIALLY_HARMFUL_APPLICATION` threat types (`_THREAT_TYPES`).

- **Normalized data**: a "clean" verdict only when the response is a genuine HTTP 200 with an empty/missing `matches` field — this is the *only* input path that can produce a safe result. Every other outcome (non-200 status, network error/timeout, or a response that doesn't parse the way the API contract promises) returns early via the module's own `_error()` helper, which always sets `data={"verdict": "unknown", ...}` — there is no code path from "the request failed" to a result that looks clean, a deliberately pinned invariant (module docstring; covered by `test_google_safe_browsing.py`'s `test_*_never_looks_like_safe` tests).

3. Internet Intelligence Collector (OSINT Crawler-as-Provider) — `internet_intelligence`

- **File**: `app/crawler/collector.py`. Category `OSINT`. IOC types: `domain`, `ipv4`, `malware_family`, `threat_actor`, `campaign`, `cve`, `file_name` — deliberately limited to free-text-searchable types; the module docstring notes that "raw network atoms like ja3 hashes or mutexes are excluded" (`:36-44,50-52`).
- **Credential**: `requires_key=False` — every underlying source is unauthenticated (`:54,58-59`).
- **Mechanism**: registered as an ordinary `BaseProvider`, but instead of one `base_url` it fans out **concurrently** to four independent sub-source modules under `app/crawler/sources/` (`:28,46,65-66`):

Sub-source	Endpoint	Auth	Rate-limit detection
`github.py`	`GET api.github.com/search/repositories` and `/search/code`	none (optional unwired `GITHUB_TOKEN` env var would raise the ceiling 10→30 req/min, but is not read by `app/core/config.py`)	HTTP 403 or 429 → `SourceRateLimitedError` (GitHub's Search API uses 403 for rate limiting, not only auth); throttled client-side via `AsyncMinIntervalLimiter` (6s)
`reddit.py`	`GET www.reddit.com/search.json`	none, but requires a real `User-Agent` (`settings.crawler_user_agent`) or Reddit blocks the request	HTTP 429 → `SourceRateLimitedError`; throttled client-side (1.1s min interval)
`rss_news.py`	8 hardcoded public security-news RSS feeds (BleepingComputer, The Hacker News, Krebs on Security, Talos, GCP TI blog, Microsoft Security blog, CrowdStrike blog, Unit42), fetched concurrently, parsed with `feedparser`, keyword-filtered	none	none — any per-feed exception is swallowed to `[]`, never raised
`pastebin_search.py`	`GET psbdmp.ws/api/v3/search/{query}` (unofficial, unauthenticated)	none	none — non-200 or any exception → `[]`, never raised

- **Normalized data**: `osint_findings` (a deduplicated-by-URL merged list of `{title, url, snippet, published_at, source}`, capped at `crawler_max_results_per_source * 4`), `source_count` (per-source hit counts), `total_findings`, `rate_limited_sources` (`collector.py:68-110`).
- **Status resolution**: the collector maps "all four sources came back empty and at least one was rate-limited" → `ProviderStatus.RATE_LIMITED`; "all empty, none rate-limited" → `NO_DATA`; any findings at all → `OK` (`:112-133`).

4. AI Backends

All eleven backend clients expose an identical async method, `call_claude_json(system_prompt, user_prompt, json_schema, tool_name="emit_result", max_tokens=None) -> dict`, which is what lets `app/ai/service.py` swap backends with zero branching logic (`service.py:43-53,56-99`). Defaults for every credential/model/URL below live in `app/core/config.py:44-80`; at call time, `_build_client()` (`service.py:56-99`) constructs a **fresh** client per call using whichever credentials the active runtime-config row (or an explicit override) supplies — proving these constructor parameters are live override points a caller genuinely exercises, not dead code.

4.1 Ollama — `ollama_client.py`

- **Endpoint**: `POST {base_url}/api/chat`; default `base_url` = `http://host.docker.internal:11434` (`:15-17`, `config.py:76`, used at `:91`).
- **Auth**: none — local server.
- **Default model**: `settings.ollama_model = "llama3.2:3b"` (`config.py:77`).
- **Structured output**: the request's `format` field is set to the JSON Schema itself (grammar-constrained decoding, not tool-calling), after flattening `$ref`/`$defs` via `inline_refs()` since Ollama's grammar compiler doesn't reliably resolve refs (`:6-10,30,76`).
- **Constructor overrides**: `OllamaClient(base_url=None, model=None, max_tokens=None)` — all three fall back to `get_settings()` values when omitted (`:39-52`).
- **Errors**: `httpx.ConnectError` → `RuntimeError("Could not reach Ollama...")`; `httpx.TimeoutException` after 120s → `RuntimeError`; HTTP status ≥400 → `RuntimeError` with status and body; a non-JSON or non-dict response body → `RuntimeError` (`:89-126`).

4.2 Anthropic — `anthropic_client.py`

- **Endpoint**: `POST https://api.anthropic.com/v1/messages` (`_API_BASE`, `:21`).
- **Auth**: header `x-api-key`, plus `anthropic-version: 2023-06-01` (`:22,54-57`).
- **Default model**: `settings.anthropic_model_id = "claude-sonnet-4-5-20250929"` (`config.py:62`).
- **Structured output**: a forced single tool call — `tools:[{name, description, input_schema: json_schema}]` with `tool_choice:{"type":"tool","name":tool_name}`; the response is parsed from the `tool_use` content block's `input` field. No schema flattening is needed — Anthropic resolves `$ref`/`$defs` natively (`:49-53,65-86`).
- **Constructor overrides**: `AnthropicClient(api_key=None, model_id=None, max_tokens=None)` (`:26-35`).
- **Errors**: HTTP status ≥400 → `RuntimeError`; a missing `tool_use` block in the response → `RuntimeError` (`:78-87`).

4.3 AWS Bedrock — `bedrock_client.py`

- **Endpoint**: boto3's `bedrock-runtime` client `.converse()` API — an SDK call, not raw HTTP (`:62-66,109-115`); the synchronous boto3 call is run via `asyncio.to_thread` (`:84-86`).
- **Auth**: two schemes — a bearer token via the `AWS_BEARER_TOKEN_BEDROCK` environment variable (set from `bedrock_api_key`/`settings.bedrock_api_key`, `:46,53-58`), or classic SigV4 IAM access-key/secret (`settings.aws_access_key_id`/`aws_secret_access_key`, `:47-48,59-61`). `is_configured` requires the bearer token **or** both IAM values (`:50,68-70`).
- **Default model / region**: `settings.bedrock_model_id = "global.anthropic.claude-sonnet-4-5-20250929-v1:0"` (`config.py:52`); default region `settings.aws_region = "us-east-1"` (`config.py:48`).

- **Structured output**: forced tool call via the Converse API's `toolConfig` (`toolSpec.inputSchema.json = json_schema` , `toolChoice:{"tool":{"name":tool_name}}`); the response is parsed from `output.message.content[].toolUse.input` (`:96-124`).
- **Constructor overrides**: `BedrockClaudeClient(bedrock_api_key=None, aws_access_key_id=None, aws_secret_access_key=None, aws_region=None, model_id=None, max_tokens=None)` (`:34-42`).
- **Errors**: `ClientError` / `BotoCoreError` → `RuntimeError` ; no matching `toolUse` block in the response → `RuntimeError` (`:116-124`).

4.4 Google Gemini — `gemini_client.py`

- **Endpoint**: `POST {_API_BASE}/models/{model_id}:generateContent` , `_API_BASE = "https://generativelanguage.googleapis.com/v1beta"` (`:29,77`).
- **Auth**: the API key is passed via the `x-goog-api-key` **header** (`:80`), deliberately not the `?key=...` query-string form Google's docs also support — httpx logs the full request URL at INFO level, which would otherwise put the real key in plain text in every application log line (`:72-76`).
- **Default model**: `settings.gemini_model_id = "gemini-2.0-flash"` (`config.py:57`).
- **Structured output**: `generationConfig.responseMimeType = "application/json"` plus an OpenAPI-3.0-style `responseSchema` , ref-flattened via `inline_refs()` because Gemini's schema dialect does not resolve JSON-Schema `$ref` / `$defs` (`:10-15,60-69`); the response text is parsed with `json.loads()` (`:91-94`).
- **Constructor overrides**: `GeminiClient(api_key=None, model_id=None, max_tokens=None)` (`:33-42`).
- **Errors**: HTTP status ≥400 → `RuntimeError` ; no `candidates` or no text in the response → `RuntimeError` ; invalid JSON text → `RuntimeError` (`:77-94`).

4.5 Groq — `groq_client.py`

- **Endpoint**: `POST {_API_BASE}/chat/completions` , `_API_BASE = "https://api.groq.com/openai/v1"` — an OpenAI-compatible chat-completions API (`:44,112`); model discovery via `GET {_API_BASE}/models` (`:152-156`).
- **Auth**: header `Authorization: Bearer <GROQ_API_KEY>` (`:108`).
- **Default model**: `settings.groq_model_id = "llama-3.3-70b-versatile"` (`config.py:69` ; also the module-level `DEFAULT_MODEL` at `:60`). A separate `FALLBACK_MODELS` list (`:55-59`) exists **only** to populate the setup wizard's model dropdown when live discovery fails — it is never used to validate a real inference call.
- **Structured output**: forced tool-calling (`tools:[{"type":"function","function":{name, description, parameters: flat_schema}}]` , `tool_choice:{"type":"function","function":{"name":tool_name}}`) rather than the OpenAI `response_format` JSON mode — the module docstring's stated rationale is that `response_format` isn't guaranteed to be honored by every hosted model (`:20-31,96-107`). The schema is ref-flattened defensively via `inline_refs()` (`:39,87`); the response is parsed from `choices[0].message.tool_calls[].function.arguments` (a JSON string) (`:128-137`).
- **Constructor overrides**: `GroqClient(api_key=None, model_id=None, max_tokens=None)` (`:64-73`).
- **Errors**: `httpx.TimeoutException` after 60s → `RuntimeError` ; HTTP status ≥400 → `RuntimeError` ; no `choices` in the response → `RuntimeError` ; no matching tool call → `RuntimeError` ; invalid JSON in `arguments` → `RuntimeError` (`:110-137`).

4.6 OpenAI — `openai_client.py`

- **Endpoint**: `POST https://api.openai.com/v1/chat/completions` ; model discovery `GET https://api.openai.com/v1/models` (`_API_BASE`).
- **Auth**: header `Authorization: Bearer <OPENAI_API_KEY>` .
- **Default model**: `settings.openai_model_id` , `DEFAULT_MODEL = "gpt-4o-mini"` — the smaller/cheaper model, matching this codebase's convention of defaulting to the fast tier rather than the largest available model.
- **Structured output**: forced tool-calling (`tool_choice` naming a specific function) rather than `response_format` JSON mode — the same "name/shape exactly one tool call" approach `groq_client.py` uses, chosen for the same precision reason. `inline_refs()` flattens `$ref` / `$defs` defensively, same as every other client in this module.
- **Model discovery filtering**: OpenAI's `/models` endpoint lists every model visible to the account, including embedding/moderation/image/audio models that can't do chat completions at all — `list_models()` filters to ids starting with `gpt-` / `o1` / `o3` / `o4` / `chatgpt-` and excluding audio/realtime/transcribe/tts/embedding matches, so the wizard's dropdown isn't cluttered with entries that would fail immediately if selected.
- **Errors**: `httpx.TimeoutException` after 60s → `RuntimeError` ; HTTP status ≥400 → `RuntimeError` ; no `choices` in the response → `RuntimeError` ; no matching tool call → `RuntimeError` ; invalid JSON in `arguments` → `RuntimeError` .

4.7 Kimi (Moonshot AI) — `kimi_client.py`

- **Endpoint**: `POST https://api.moonshot.ai/v1/chat/completions` , an OpenAI-compatible surface; model discovery `GET https://api.moonshot.ai/v1/models` .
- **Auth**: header `Authorization: Bearer <KIMI_API_KEY>` .
- **Default model**: `settings.kimi_model_id` , `DEFAULT_MODEL = "kimi-k2.5"` — deliberately not Moonshot's newest model. Several current Moonshot models (`kimi-k3` , `kimi-k2.7-code`) always run in "thinking" mode with no way to disable it, and a forced `tool_choice` call is documented as incompatible with thinking mode on those models (would `400`). `kimi-k2.5` and the `moonshot-v1-*` family have no thinking parameter and work with forced tool-calling with no special handling, so the default is pinned there.
- **Structured output**: OpenAI-style forced tool-calling, same approach as `openai_client.py` / `groq_client.py` .
- **Errors**: same shape as OpenAI — timeout after 60s, HTTP ≥400, missing choices, missing matching tool call, or invalid JSON in `arguments` all raise `RuntimeError` .

4.8 DeepSeek — `deepseek_client.py`

- **Endpoint**: `POST https://api.deepseek.com/chat/completions` — note the base URL has **no** `/v1` path segment, unlike most other OpenAI-compatible clients in this module; model discovery `GET https://api.deepseek.com/models` .
- **Auth**: header `Authorization: Bearer <DEEPSEEK_API_KEY>` .
- **Default model**: `settings.deepseek_model_id` , `DEFAULT_MODEL = "deepseek-v4-flash"` .
- **Structured output**: OpenAI-style forced tool-calling, same approach as every other OpenAI-compatible client in this module.
- **Errors**: same RuntimeError shape as OpenAI/Kimi. DeepSeek's API also returns a provider-specific HTTP 402 ("Insufficient Balance") in addition to the standard 400/401/429/500/503 codes — that per-status distinction is handled in `app/ai/connection_test.py` , not in the client itself.

4.9 xAI (Grok) — `xai_client.py`

- **Endpoint**: `POST https://api.x.ai/v1/chat/completions`, an OpenAI-compatible surface; model discovery `GET https://api.x.ai/v1/models`.
- **Auth**: header `Authorization: Bearer <XAI_API_KEY>`.
- **Default model**: `settings.xai_model_id`, `DEFAULT_MODEL = "grok-4.6"`. The module's own docstring flags that `"grok-3"` is retired (as of 2026-05-15) and deliberately excluded from `FALLBACK_MODELS`.
- **Structured output**: forced tool-calling via the standard OpenAI-shaped `tool_choice` object. Caveat noted in the module docstring: xAI's own documentation states `tools[].function.parameters` "should" be followed by the model but is "not enforced at the moment" — there is no guaranteed strict schema adherence on xAI's end, so the JSON-decode error handling is left exactly as defensive as every other client's, not loosened or tightened for this one.
- **Errors**: same RuntimeError shape as the other OpenAI-compatible clients. xAI's error body is flat (`{"code", "error"}` as a bare string), not nested like OpenAI's — this client doesn't parse error bodies itself (it surfaces `response.text` verbatim), so the shape difference doesn't affect anything here.

4.10 Mistral AI — `mistral_client.py`

- **Endpoint**: `POST https://api.mistral.ai/v1/chat/completions`; model discovery `GET https://api.mistral.ai/v1/models`.
- **Auth**: header `Authorization: Bearer <MISTRAL_API_KEY>` (standard OpenAI-compatible bearer scheme, not Anthropic's `x-api-key` or Gemini's query-param key).
- **Default model**: `settings.mistral_model_id`, `DEFAULT_MODEL = "mistral-small-2506"` — the smaller/faster/cheaper model, matching this codebase's default-to-fast-tier convention. Mistral versions its models with dated suffixes rather than a single rolling name, though rolling aliases (e.g. `mistral-large-latest`) also exist.
- **Structured output**: forced tool-calling via the standard OpenAI-shaped `tool_choice` object, per Mistral's own live API reference.
- **Errors**: same RuntimeError shape as the other OpenAI-compatible clients. Mistral's error-response body shape is unconfirmed (no dedicated error-schema documentation was found), so this client deliberately doesn't attempt to parse it — same as `openai_client.py`'s own template, which only ever surfaces `response.text` on a non-2xx status.

4.11 OpenRouter — `openrouter_client.py`

- **Endpoint**: `POST https://openrouter.ai/api/v1/chat/completions`; model discovery `GET https://openrouter.ai/api/v1/models`. OpenRouter is a meta-router, not a model provider of its own — it gives access to hundreds of underlying models from many companies (OpenAI, Anthropic, Google, Meta, DeepSeek, and others) through one OpenAI-compatible API surface.
- **Auth**: header `Authorization: Bearer <OPENROUTER_API_KEY>`.
- **Default model**: `settings.openrouter_model_id`, `DEFAULT_MODEL = "openai/gpt-4o"`.
- **Structured output**: forced tool-calling via the standard OpenAI-shaped `tool_choice` object, confirmed directly against OpenRouter's own OpenAPI spec (schema `ChatNamedToolChoice`).
- **Model discovery filtering, a real constraint specific to this backend**: because OpenRouter fans out to hundreds of underlying models, many of which do not support forced tool-calling at all, `list_models()` filters the live `/models` response down to ids whose `supported_parameters` array contains `tool_choice` (confirmed live: 342 of 413 models declared tool_choice support at the time this was written) — picking a model without it would `400` against this client's forced-tool-calling `call_claude_json()`. Falls back to returning every listed model id if the response doesn't include `supported_parameters` for any model.
- **Errors**: same RuntimeError shape as the other OpenAI-compatible clients. OpenRouter's error body is `{"error": {"code", "message", "metadata"?}}` — similar to OpenAI's nested shape but without an `error.type` / `error.param` field; doesn't change handling since only `response.text` is surfaced either way.

5. Cross-Cutting Mechanisms

- **Per-investigation credential overrides (IOC providers)**: `app/core/runtime_context.py:38-47` — `get_credential(provider_id, field, fallback)` returns a per-investigation `ContextVar` override when one has been set (via `set_provider_overrides()` at `orchestrator.py:113`), else falls back to the `.env`-derived `Settings` value. Every real provider's `fetch()` calls this — e.g. `abuseipdb.py:29`, `virustotal.py:44`, `otx.py:46`, `nvd.py:32`, `censys.py:40-41`, `hybrid_analysis.py:45`, `phishtank.py:29`, `urlhaus.py:31`, `threatfox.py:38`, `malwarebazaar.py:31`.
- **AI backend resolution order**: `app/ai/service.py:_get_ai_client` (`:102-145`) resolves the active backend in this order on **every call**: an explicit `backend_override` (used by the reanalyze/comparison feature) → the DB-backed active runtime config (`app/core/runtime_config.py`) → the legacy `settings.ai_backend` default of `"ollama"` (`config.py:80`).
- **Live connection tests** (candidate credentials only, never `get_settings()`): IOC providers are tested in `app/providers/connection_test.py:51-208` (VirusTotal, AbuseIPDB, OTX, the abuse.ch family via ThreatFox's `query_status`, NVD, Hybrid Analysis, Censys, PhishTank); urlscan.io and Google Safe Browsing are tested the same way despite having no wizard entry. AI backends are tested in `app/ai/connection_test.py` — a dedicated `_check_*` function exists for all eleven (Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, OpenRouter, Anthropic, Gemini, Ollama, Bedrock), not just the original five; the module's own docstring records the investigation finding that motivated dedicated per-backend checks in the first place: none of the original five backends' generic error handling reliably distinguished "bad key" from "bad request" from "service down" without one. Both endpoints — `POST /api/v1/providers/{id}/test` and `POST /api/v1/ai/test` — make one real, minimal outbound call with the credentials from the request body and never persist them; see the *Runtime Configuration and Credential Lifecycle* chapter for how this relates to the separate, saved runtime-config path.

6. Quick-Reference Table

Provider ID	Category	Auth mechanism	IOC types	Rate-limit signal
`virustotal`	threat_intel	header `x-apikey`	ipv4/ipv6/domain/url/hashes	429/403/509
`abuseipdb`	threat_intel	header `Key`	ipv4/ipv6	429/403/509
`otx`	threat_intel	header `X-OTX-API-KEY`	ipv4/ipv6/domain/hostname/url/hashes	429/403/509
`urlhaus`	threat_intel	header `Auth-Key` (shared)	url/domain/ipv4	`query_status` mapping
`threatfox`	threat_intel	header `Auth-Key` (shared)	ipv4/ipv6/domain/url/md5/sha256	`query_status` mapping
`malwarebazaar`	threat_intel	header `Auth-Key` (shared)	md5/sha1/sha256/sha512	`query_status` mapping
`crtsh`	certificate_intel	none	domain/tls_certificate	429/403/509
`nvd`	vulnerability	optional header `apiKey`	cve	429/403/509
`cisa_kev`	vulnerability	none	cve	429/403/509
`mitre_attack`	threat_intel	none	mitre_technique	429/403/509
`whois_rdap`	whois	none	domain/ipv4/ipv6/asn	socket timeout/error
`hybrid_analysis`	sandbox	header `api-key`	sha256 only	429/403/509
`spamhaus`	threat_intel	none (DNS only)	ipv4/domain	none (no HTTP call)
`phishtank`	threat_intel	optional form `app_key`	url	429/403/**509**
`censys`	passive_dns	Bearer token + `X-Organization-ID`	ipv4/ipv6	429/403/509
`internet_intelligence`	osint	none	domain/ipv4/malware_family/threat_actor/campaign/cve/file_name	per-sub-source (see §3)
`urlscan`	sandbox	header `API-Key`	url/domain	401/403 = bad key, not rate limit; else `TIMEOUT` at 60s
`google_safe_browsing`	threat_intel	query param `key`	url/domain	any non-200/error → `unknown`, never `RATE_LIMITED` - mapped as "safe"

AI backend	Endpoint style	Auth	Structured-output technique
Ollama	Local REST (`/api/chat`)	none	`format` = flattened JSON Schema (grammar-constrained)
Anthropic	Messages API	header `x-api-key`	forced tool call
Bedrock	boto3 `.converse()` SDK call	bearer token or IAM SigV4	forced tool call via `toolConfig`
Gemini	`generateContent` REST	header `x-goog-api-key` (deliberately not the `?key=` query-string form, to keep the key out of httpx's INFO-level URL logging)	`responseSchema` JSON mode (flattened)
Groq	OpenAI-compatible chat-completions	header `Authorization: Bearer`	forced tool call (not `response_format`)
OpenAI	OpenAI chat-completions	header `Authorization: Bearer`	forced tool call
Kimi (Moonshot AI)	OpenAI-compatible chat-completions	header `Authorization: Bearer`	forced tool call
DeepSeek	OpenAI-compatible chat-completions (no `/v1` in base URL)	header `Authorization: Bearer`	forced tool call
xAI (Grok)	OpenAI-compatible chat-completions	header `Authorization: Bearer`	forced tool call (best-effort — xAI docs note `parameters` isn't strictly enforced)
Mistral AI	OpenAI-compatible chat-completions	header `Authorization: Bearer`	forced tool call
OpenRouter	OpenAI-compatible chat-completions (meta-router over many providers)	header `Authorization: Bearer`	forced tool call (model list filtered to `tool_choice` -capable ids)

Runtime Configuration Architecture

Every provider credential, every AI backend key, and the choice of which AI backend is active right now are all governed by one subsystem: the runtime configuration layer built around `backend/app/core/runtime_config.py`, `backend/app/core/runtime_context.py`, and the `provider_runtime_configs` / `config_audit_log` tables. This chapter is the authoritative reference for that subsystem because nearly every operational question a backend engineer or security reviewer will eventually ask — where does this API key live, will changing a key require a restart, can two investigations racing at once see two different credentials, what happens to configuration if the container is destroyed and recreated — is answered here.

The short version, expanded in full below: configuration used to be a frozen, `.env`-derived, process-lifetime singleton (`app/core/config.py`'s `@lru_cache`'d `get_settings()`). It has been replaced, without removing that legacy path, by a database-backed store that is read fresh on every relevant call, encrypted at rest, and audit-logged. The legacy path still exists as a fallback for installs that have never touched the new system.

Two ways configuration enters the system

Entry point	Where it writes	Who uses it	Restart required?
Windows Setup Wizard (`windows/wizard/Setup-Wizard.ps1`)	`.env` file, via `Write-PlatformEnvFile` (`windows/scripts/Write-EnvFile.ps1:51`)	First-time installers, and anyone re-running the wizard	Yes, implicitly — `.env` is only read once, at process start, by `get_settings()`
Manage Providers UI → Runtime API (`POST /api/v1/runtime/ai-providers/{backend}`, `POST /api/v1/runtime/ioc-providers/{provider_id}`)	`provider_runtime_configs` table (Postgres), encrypted	Any admin/analyst with `provider:manage`, at any time after first boot	No

The wizard path is the one-time bootstrap: `Setup-Wizard.ps1` collects values across its AI Configuration and Provider Configuration pages into `$State.Settings`, then on the Summary page's "Start Installation" click writes them all to `.env` in one pass (`Setup-Wizard.ps1:998-1002`). `Write-EnvFile.ps1:15-49` (`ConvertTo-SafeEnvValue`) strips CR/LF and rejects any value containing `#` before writing, since `#` opens a comment in `.env` syntax and would silently truncate everything after it (`Write-EnvFile.ps1:62-66`). This is the *only* code path that populates `.env` with provider/AI secrets; the backend loads that file once, at import time, through `app/core/config.py`'s `Settings(BaseSettings)` class and its `@lru_cache`'d `get_settings()` accessor (`config.py:110-111`).

The runtime-API path is what makes the platform's "no restart" claim true. The `/providers` page in the frontend (`frontend/app/providers/page.tsx:106-174`) calls `configureAIProvider` / `configureIOCProvider` (`frontend/lib/api.ts:418-423`, `:482-487`), which hit `backend/app/api/routes/runtime.py:38-48` (`configure_ai_provider`) and `runtime.py:135-152` (`configure_ioc_provider`), both gated by `require_permission("provider:manage")` and both delegating to `runtime_config.py`'s `upsert_ai_provider()` / `upsert_ioc_provider()`.

A third, deliberately separate action exists at each layer: **Test Connection** (`POST /api/v1/ai/test`, `POST /api/v1/providers/{id}/test`). It never writes anywhere — it makes one live call using whatever credentials are currently sitting in the form, before the analyst has clicked Save. This "test" vs. "save" distinction is structural, not cosmetic, and is the key to the historical bug discussed later in this chapter.

Where configuration is stored

`provider_runtime_configs` (`backend/app/models/runtime_config.py:30-55`) holds one row per `(kind, provider_id)` pair — `kind` is `ai` or `ioc` (`ProviderKind` enum), and the pair is enforced unique by `UniqueConstraint(kind, provider_id, name="uq_provider_runtime_kind_id")` (`runtime_config.py:32`). Relevant columns:

Column	Purpose
`enabled`	IOC providers only, in practice — whether this provider participates in the next fan-out.
`is_active`	AI backends only, in practice — exactly one AI row is meant to be active at a time; enforced in application code, not by a DB constraint.
`encrypted_credentials`	Fernet ciphertext of a JSON object (e.g. `{"api_key": "..."}`). Never plaintext, never returned by any API response.
`model_id`	The selected model for an AI backend, or provider-specific model/tier info.
`extra_config`	Non-secret extras (e.g. a custom endpoint URL) as JSONB.
`last_test_at` / `last_test_ok` / `last_test_message`	Recorded separately, by the `/record-test` endpoints, after a Test Connection call — not derived automatically from the test itself.
`updated_by`	FK to `users.id`.

A second table, `config_audit_log` (`runtime_config.py:58-75`), is append-only and intentionally has no `created_at` / `updated_at` — it carries its own explicit `timestamp` column, plus `actor_user_id` (FK, nullable), a denormalized `actor_email` (so the log stays readable even if the account is later deleted), `action`, and `detail`. `detail` only ever accepts descriptive text — e.g. "configured AI backend anthropic" — never a credential value; verified by exercising the configure/enable/disable/test/activate flow and grepping the resulting audit rows and backend logs for any real secret, with no hits (see Audit trail below).

Protecting the secrets themselves

Credentials submitted through the runtime API are encrypted before the `upsert_ai_provider()` / `upsert_ioc_provider()` calls persist them: `row.encrypted_credentials = encrypt_secret(json.dumps(credentials))` (`runtime_config.py:234`, `:339`). `encrypt_secret()` / `decrypt_secret()` (`backend/app/core/crypto.py:49-54`, `:57-67`) use `cryptography.fernet.Fernet` — symmetric, authenticated encryption. `decrypt_secret()` returns `""` on any `InvalidToken` rather than raising, so a corrupted or key-mismatched row degrades to "no credential" instead of crashing a request.

The Fernet key itself comes from one of two sources, resolved in `crypto.py:39-46` (`_fernet()`):

1. `settings.encryption_master_key`, if the operator has explicitly set it (`config.py:30`), or
2. an HKDF-derived key from `settings.jwt_secret_key` (`config.py:23`) if no master key is configured (`crypto.py:33-36`, `_derive_key_from_jwt_secret`).

This fallback is a real, worth-stating tradeoff rather than a hidden detail: it means every existing installation already has a usable encryption key with no migration step, but it is defense-in-depth, not independent key separation — compromise of `jwt_secret_key` also exposes the derived encryption key. An operator who wants the two secrets to be independently rotatable and independently

compromised should set `encryption_master_key` explicitly rather than rely on the derived default. `mask_secret()` (`crypto.py:70-77`) is the only thing the UI is ever shown for an already-saved credential — a `****...`-style preview, used in `_row_to_public_dict` (`runtime_config.py:148`); the plaintext or ciphertext is never returned by any API response.

Bridging existing `.env` installs: `seed_from_env_if_empty()`

An install that has been running purely on `.env` since before this table existed does not start with an empty configuration UI. `seed_from_env_if_empty()` (`runtime_config.py:398-448`) checks whether `provider_runtime_configs` has any rows at all; if it does, it is a strict no-op (`:408-410`). If the table is empty, it walks the eleven AI backends (`_AI_ENV_SEED_MAP` , `:62-76`) and all eighteen registered IOC providers (`_ENV_SEED_MAP` , `:49-60` , sourced from `app.providers.registry.get_all_providers()`), reads each one's corresponding field off the frozen `get_settings()` singleton, and inserts a pre-encrypted row for each (`:417-427` , `:435-443`). It runs once at process startup (`backend/app/main.py:50-58` , the `_seed_runtime_config` startup event), wrapped in a try/except so a seeding failure never blocks boot (`main.py:59-60`), and is explicitly documented as idempotent — safe to call on every restart because it only acts on an empty table (`runtime_config.py:404`).

The practical effect: an operator who has only ever used `.env` gets a populated, editable Manage Providers screen the first time they upgrade to a build that has this table, with zero manual data entry — and every subsequent edit through the UI takes over from `.env` for that specific provider/backend from that point forward.

How configuration is loaded — the part that actually matters

This is the section that determines whether "I changed a key" and "the platform used the new key" are the same statement. The platform resolves configuration completely differently depending on whether the caller is an IOC provider or an AI backend, but both share one property: **neither path trusts a value cached in process memory beyond the scope of a single investigation or a single AI call.**

IOC providers: a `ContextVar` snapshot taken once per investigation

IOC provider connectors (`VirusTotal` , `AbuseIPDB` , etc.) are module-level singletons — one Python object per provider, reused for the life of the backend process (`backend/app/providers/registry.py`). Mutating a shared attribute on that singleton the moment a request needs it would be unsafe under concurrency: two investigations could be in flight at once, one started right after an administrator changed a provider's credentials, one already mid-flight — a naive shared-state write could let them see each other's configuration.

Instead, `run_all_providers()` (`backend/app/providers/orchestrator.py:112-113`) takes one fresh read from the database — `snapshot = await get_ioc_provider_snapshot()` (`runtime_config.py:336-354`) — at the very start of a single investigation's fan-out, and stores it via `set_provider_overrides(snapshot)` into a module-level `contextvars.ContextVar` (`runtime_context.py:19-21`). Because `asyncio.create_task()` copies the current `contextvars.Context` into every task it spawns (`orchestrator.py:120`), every concurrent per-provider task belonging to that one investigation inherits the identical, frozen snapshot — regardless of what happens to the underlying table while those tasks are still running. Two investigations running concurrently, each configured differently, therefore never cross-contaminate.

Each provider's `BaseProvider.run()` reads that snapshot before doing anything else (`backend/app/providers/base.py:104-111`): `override = get_provider_override(self.provider_id)` , and `effective_configured` prefers the override's `configured` flag over the object's own frozen `self.configured` . If the effective state is "not configured," the call short-circuits with `ProviderStatus.NOT_CONFIGURED` and the message `f"{self.provider_name} is not configured (missing API key/credentials)."` (`base.py:143`) — no HTTP call is attempted. If configured, the provider's `fetch()` pulls the actual credential value through `get_credential(provider_id, field, fallback)` (`runtime_context.py:38-47`), which returns the per-investigation override if one exists, otherwise falls back to whatever `.env` provided via `get_settings()` — for example `virustotal.py:44` : `api_key = get_credential("virustotal", "api_key", settings.virustotal_api_key)` .

AI backends: a fresh client built on every single call, not a snapshot

AI backend resolution goes further than a per-investigation snapshot — it is refreshed on every individual AI call, not once per investigation. `_get_ai_client()` (`backend/app/ai/service.py:102-145`) resolves the backend in this order:

1. an explicit `backend_override` parameter, used only by the AI-comparison / re-analyze feature (`service.py:125-126`);
2. `app.core.runtime_config.get_active_ai_config()` — a fresh database read, every call, no caching (`service.py:128` ; the read itself is `runtime_config.py:200-215`);
3. the legacy `Settings` -derived value, only if no runtime config row exists yet (`service.py:134-138`).

Whichever backend wins, `_build_client()` (`service.py:56-99`) constructs a **new client instance** for that call — e.g. `AnthropicClient(api_key=..., model_id=...)` (`anthropic_client.py:25-39`) — using the credentials the DB read just returned, rather than reusing the process-lifetime singleton each client module also exposes (e.g. `get_anthropic_client()` , `anthropic_client.py:93-97`). This is the single design decision that makes "switch the active AI backend, no restart" true in practice: there is no cached client object anywhere holding a stale key or a stale backend choice. If the resolved client's `is_configured` still comes back false, the call raises `RuntimeError(f"AI backend '{backend}' is not configured (missing API key/URL/model) -- configure it from the AI Providers panel or check .env")` (`service.py:140-144`) rather than silently proceeding.

How a runtime update actually propagates, step by step

1. An admin submits new credentials for, say, Anthropic on the Manage Providers screen.
2. `POST /api/v1/runtime/ai-providers/anthropic` runs; `upsert_ai_provider()` encrypts and writes the row, and `config_audit_log` gets an entry describing the action (never the value).
3. Nothing currently running is affected — no in-flight investigation's `ContextVar` snapshot changes, and no cached AI client is invalidated, because none exists to invalidate.
4. The *next* IOC lookup's `run_all_providers()` call takes a brand-new `get_ioc_provider_snapshot()` read, and the *next* AI call (in that same lookup or an unrelated one) calls `get_active_ai_config()` fresh. Both see the row written in step 2 immediately.

No cache invalidation, no signal, and no process-wide lock is needed anywhere in this sequence — correctness comes entirely from *never caching* the value past the lifetime of one investigation (providers) or one call (AI backends).

Why this replaced a frozen-singleton design, and the exact bug that motivated it

Before this system existed, every provider's `configured` flag was computed exactly once, at module-import time, from the frozen `get_settings()` singleton — for example `virustotal.py:24-26` originally read `self.configured = bool(get_settings().virustotal_api_key)` at construction. `get_settings()` is `@lru_cache` 'd (`config.py:110-111`), meaning it evaluates `.env` exactly once per process and returns the identical object for the rest of that process's life. That is fine for values that genuinely never change, but it is the wrong model for a value an operator expects to change without restarting a container.

This produced a specific, git-independently-documented divergence: **Test Connection would report success, and the very next real investigation would still report the provider as not configured.** Confirmed from `connection_test.py` 's own module docstring plus three independent write-ups (`docs/PROVIDERS.md:375-378` , `docs/TROUBLESHOOTING.md:96-134` , `RUNTIME_PROVIDER_IMPLEMENTATION_REPORT.md:11-12`): `POST /api/v1/providers/{id}/test` makes one live HTTP call using the *candidate* key from the request body, never reading `get_settings()` at all, so it always correctly reflects whatever key was just typed in. A real investigation, however, went through `BaseProvider.run()` , which — before this system existed — had nothing else to check but `self.configured` , frozen at process start. If the key was added to `.env` *after* the backend process had already started — the normal sequence, since Test Connection happens before any save/restart — the real investigation still saw `configured=False` and short-circuited to `NOT_CONFIGURED` , immediately after a Test Connection that had just reported success for the same credential.

The fix is exactly the mechanism described above: the frozen `self.configured` check was replaced by the `ContextVar` snapshot fed from a fresh per-investigation database read (`runtime_context.py` , `orchestrator.py:112-113`), and the AI side's cached client singleton was replaced by `_get_ai_client()` 's fresh-DB-read-plus-fresh-construction pattern (`service.py:56-63` , `:102-138`). A credential saved through the Manage Providers UI is now visible to the very next investigation or AI call, closing the exact gap that used to let Test Connection and a live investigation disagree.

Two things are worth being precise about, so this section doesn't overstate what was found. First, the literal phrase "API key not provided" does not appear anywhere in this codebase's source or documentation — the real, current error strings are `f"{self.provider_name} is not configured (missing API key/credentials)."` (`base.py:143` , also asserted against in `backend/app/tests/unit/test_provider_base.py:55-61`) for providers, and the `RuntimeError` text quoted above for AI backends. Second, a separate, unrelated bug with the *opposite* symptom — Test Connection *failing* despite a valid key — is self-documented in `Setup-Wizard.ps1:211-236` (an in-code "ROOT CAUSE" comment) and `API_CONFIGURATION_FIX_REPORT.md:15-30` : every Test Connection button required a wizard session token (`$State.AccessToken`) that was only ever set during the Summary page's "Start Installation" step, so re-running the wizard to reconfigure an already-installed platform (without re-logging-in) made every Test Connection click fail with a misleading "create the administrator account first" message, even though the account already existed. This is a wizard-session bug, not a configuration-freshness bug, and should not be conflated with the frozen-singleton issue above — different symptom, different fix (`Get-OrCreateWizardSession` , `Setup-Wizard.ps1:237-253`).

Surviving a restart

Because runtime-configured credentials live in `provider_runtime_configs` in Postgres — a container that, in the Docker Compose and Kubernetes topologies alike, persists its data volume independently of the `backend` container's lifecycle — stopping, recreating, or upgrading the `backend` container does **not** lose or reset any runtime-configured provider or AI credential. This is the inverse of the pre-existing `.env` behavior, where a value only ever took effect after a restart; here, a restart is *not required* for a change to take effect, and *is safe* in the sense that it does not roll configuration back to whatever `.env` says. On a fresh backend process start, `seed_from_env_if_empty()` runs again but is a no-op the moment any row already exists (`runtime_config.py:408-410`), so an established runtime configuration is never overwritten by `.env` on restart. Only a genuinely empty `provider_runtime_configs` table — a brand-new database volume — would cause `.env` values to be (re-)seeded.

Audit trail

Every configure, enable/disable, activate, and record-test action against a provider or AI backend is written to `config_audit_log` with a timestamp, the acting user's ID and email, an `action` string, and a `detail` string, retrievable via `GET /api/v1/runtime/audit-log` (gated by `require_permission("audit:read")`). The table's write path accepts only descriptive text for `detail` — there is no code path that interpolates a credential value into it, and this was checked empirically (not just read from source) by exercising the full configure/test/enable/activate sequence and grepping both the audit rows and the backend logs for any real secret, with no hits either place.

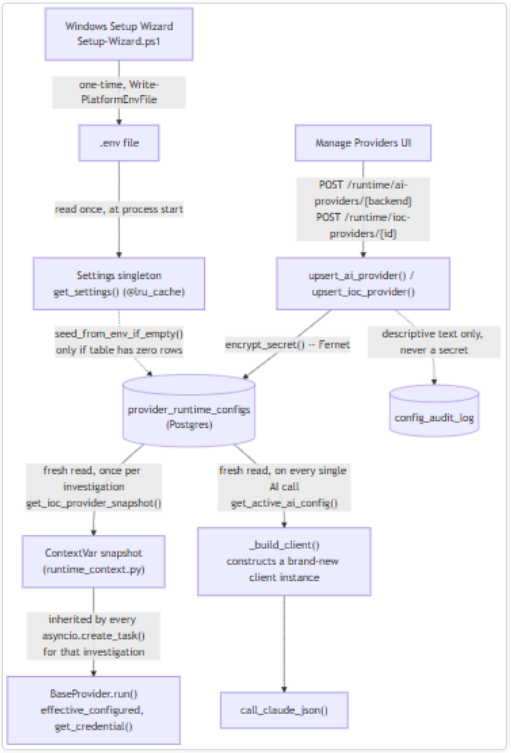

Figure 5 — Diagram: Audit trail

Diagram: Credential lifecycle -- wizard/ `.env` and runtime-API entry points, encrypted storage, the per-investigation `ContextVar` snapshot for IOC providers, and the per-call fresh client construction for AI backends. The two right-hand branches are why a saved credential change needs no restart: neither path ever reads a value cached longer than one investigation (providers) or one call (AI backends).

Summary

Configuration enters the platform two ways — a one-time `.env` write from the Windows wizard, or a live, encrypted database write from the Manage Providers UI — and the database path is the one that matters for "no restart" claims. Credentials are Fernet-encrypted before they are stored, using either an explicitly configured master key or one derived from the JWT signing secret as a fallback with a stated tradeoff. Every read that actually gates behavior — a provider's `configured` check, an AI call's backend/client resolution — is a fresh read scoped to the current investigation (providers, via a `ContextVar` snapshot) or the current call (AI backends, via fresh client construction), never a value cached for the life of the process. That design directly closes a real, documented historical bug in which Test Connection and a live investigation could disagree about whether a provider was configured, and it is also why configuration now survives a backend restart intact rather than requiring one to take effect.

Backend Security Architecture

This chapter documents the four security-relevant subsystems of the FastAPI backend that a backend engineer, DevOps operator, or security reviewer needs to reason about before deploying or auditing HORIZON GRID: **authentication**, **authorization (RBAC)**, **credential/secrets management**, and **audit logging**. Every claim below is traceable to a specific file and, where useful, a line reference, obtained by direct inspection of `backend/app/` and `windows/` in this repository.

Each section ends with two explicit lists:

- **Implemented** — mechanisms that exist in the running code today, verified by reading the implementation (not a comment, docstring, or marketing description of intent).
- **Recommended (not yet implemented)** — controls a production-security review would normally expect, confirmed absent by searching the same code paths (e.g. no revocation table, no rate limiter on a given route). These are documented as honest gaps, not hidden. One item, the Fernet key's default derivation, is called out separately below because it is a documented design tradeoff rather than a missing feature.

1. Authentication

Authentication is JWT-based. `backend/app/auth/security.py` issues tokens with `python-jose`, signing algorithm `HS256` (`jwt_algorithm` in `app/core/config.py:24`), and hashes passwords with `passlib`'s `bcrypt` scheme (`security.py:10`) — plaintext passwords are never persisted, only the bcrypt hash on `users.hashed_password`.

Two token types are minted together at `/auth/login` and `/auth/refresh`, both carrying `sub` (email), `role`, `token_version`, `type`, `iat`, and `exp` claims (`security.py`'s `_create_token`):

Token	`type` claim	Default lifetime	Setting
Access token	`"access"`	30 minutes	`access_token_expire_minutes` (`config.py:25`)
Refresh token	`"refresh"`	7 days	`refresh_token_expire_days` (`config.py:26`)

`GET /api/v1/auth/refresh` (`app/api/routes/auth.py:56-68`) only accepts a token whose decoded `type` claim is `"refresh"`; `get_current_user` (`app/auth/rbac.py:28-47`), used by every protected route, only accepts `type == "access"` — a refresh token cannot be used directly as a bearer credential, and vice versa. `get_current_user` re-checks `user.is_active` against the database on every request (`rbac.py:45`), so an account disabled after a token was issued is rejected on its very next request, without waiting for the token to expire.

The dependency is wired via FastAPI's `HTTPBearer(auto_error=False)` rather than `OAuth2PasswordBearer`, because `/auth/login` takes a JSON body, not an OAuth2 form-encoded grant (`rbac.py:14-18`).

A deactivated account now gets a distinct `403 "Account disabled"`, not the generic `401 "Could not validate credentials"` (v0.2.2). Previously `get_current_user` raised the same generic 401 for both "no such user" and "user exists but is inactive." Found via adversarial concurrency testing of the last-active-admin protection invariant (§ below): two admins simultaneously trying to disable each other is correctly resolved so the system never reaches zero active admins, but the losing caller's OWN account is what gets deactivated by the other request that won the race — so their own next request (still in flight) hit this exact check and got a 401 that read like a broken session, never even reaching the business-rule check that would have explained what actually happened. The distinction matters here specifically because the caller and the affected account are the same person in this race; a 403 with the existing "Account disabled" message (already used at `/auth/login` for the same underlying condition) is accurate and far less confusing than a generic credentials failure.

First-user-becomes-admin bootstrap

`POST /auth/register` (`app/api/routes/auth.py:21-45`) contains a specific rule: the very first user ever created on an instance is granted `Role.ADMIN`; every registration attempt after that is rejected outright with `HTTP 403` and `detail="Self-registration is closed. Ask an administrator to create your account from the Administration page."` (`auth.py:27-35`, checked via `SELECT User.id` against an empty table) — it no longer falls back to creating a `Role.ANALYST` account. This is the platform's only way to obtain an initial administrator — there is no seed script or default account — and it is exactly the mechanism the Windows Setup Wizard drives when it creates the administrator account at install time. Every account created after that first admin must go through an existing admin, via `POST /admin/users` (§2) or the `/admin` frontend page, which lets the admin pick the new account's role up front.

Password validation

Field	Constraint	Where enforced
`RegisterRequest.password`	`min_length=8`, `max_length=72`	Pydantic `Field()`, `app/schemas/auth.py:18`
`LoginRequest.password`	`max_length=72` only (no minimum)	`app/schemas/auth.py:24`

The 72-byte cap is bcrypt's own effective limit (bytes beyond 72 are silently ignored); a code comment records that an unconstrained password previously produced an *unhandled 500* from `passlib`'s `PasswordSizeError` on inputs over 4096 bytes before this constraint existed (`schemas/auth.py:5-13`). There is no complexity rule beyond the 8-character minimum.

Implemented:

- Bearer JWT access/refresh tokens (HS256), bcrypt password hashing, DB-backed `is_active` re-check on every authenticated request.
- Server-side minimum password length (8 characters) enforced by Pydantic on registration.
- First-user-becomes-admin bootstrap, removing any need for a manual database edit to obtain an administrator, with self-registration closing itself (`403`) the instant that first admin exists.

Recommended (not yet implemented):

- **No self-service token revocation.** `token_version` (`app/models/user.py`) gives one specific, admin-driven path to invalidate a user's existing tokens: `POST /api/v1/admin/users/{id}/reset-password` (§2's user-management subsection) increments it, and both `get_current_user` and `POST /auth/refresh` reject any token whose embedded `token_version` no longer matches the database value — closing the gap a stateless JWT would otherwise have, where an already-issued token stays valid under the OLD password until it naturally expires. What's still genuinely missing: a user cannot trigger this for their *own* sessions without changing their password (no self-service "log out everywhere"), and there is still no revocation for reasons other than a password reset short of rotating `jwt_secret_key`, which invalidates *every* session platform-wide.
- **`/auth/login` now has a real rate limiter, added in a later mission-critical-reliability review (v0.2.3):** a per-account Redis fixed-window limiter (`app/core/cache.py::RateLimiter`, keyed `login:{email}` rather than by source IP, since registration/login lookups are already keyed by email throughout the codebase) rejects further attempts against the same account with `429` once

`login_rate_limit_max_attempts` (10 by default) is exceeded within `login_rate_limit_window_seconds` (60 by default), both configurable. **`/auth/register` still has no rate limiting** — the platform's other rate limiter (`POST /lookup/stream`) is unrelated and does not cover it, and the bootstrap-only self-registration gate (first bullet above) is not a throttle.

- **No account lockout after repeated failed logins**, and no multi-factor authentication — neither concept exists anywhere in `app/auth/` or `app/models/user.py`.
- **No password complexity rule beyond length** (no digit/symbol/case requirement).

2. Authorization: Role-Based Access Control (RBAC)

The platform has exactly three roles, `Role.ADMIN`, `Role.ANALYST`, `Role.VIEWER` (`app/models/user.py:11-14`), stored on `users.role`. Authorization is a flat permission-string matrix, `ROLE_PERMISSIONS` (`app/models/user.py:31-44`), checked per-route by the `require_permission(permission: str)` dependency factory (`app/auth/rbac.py:50-62`): a route either declares a required permission string as a FastAPI dependency or it enforces none — there is no implicit default-deny/default-allow fallback beyond "no `require_permission` call means no permission check."

Permission string	admin	analyst	viewer	Enforced by
`lookup:create`	✓	✓		`POST /lookup/stream`, `POST /lookup/{id}/reanalyze`
`lookup:read`	✓	✓	✓	`GET /lookup/{id}`, `GET /lookup`, `GET /lookup/{id}/assessments`, `GET /providers/health`, `GET /lookup/{id}/pivots`, `GET /runtime/ai-active`
`lookup:export`	✓	✓		**Defined in the matrix; no route calls `require_permission("lookup:export")`.**
`evidence:read`	✓	✓	✓	`GET /lookup/{id}/analysis/evidence`
`analysis:generate`	✓	✓		all nine AI explanation endpoints under `/lookup/{id}/analysis/*` (why, what-is-this, disagreement, false-positive, challenge, next-actions, gaps, score-explanation)
`copilot:query`	✓	✓		`POST /lookup/{id}/analysis/copilot`
`hunting:generate`	✓	✓		`POST /lookup/{id}/hunt`, `POST /lookup/{id}/detection`
`basket:manage`	✓	✓		all five `/basket*` routes
`case:create` / `case:read` / `case:write` / `case:close`	✓	✓	`case:read` only	the eight `/cases*` routes
`provider:manage`	✓			`POST /providers/{id}/test`, `POST /ai/test`, `POST /ai/{backend}/models`, and every `/runtime/ai-providers*` / `/runtime/ioc-providers*` route
`audit:read`	✓			`GET /runtime/audit-log`
`user:manage`	✓			Every route in `app/api/routes/admin.py`: `GET /admin/users`, `GET /admin/users/stats`, `GET /admin/roles`, `POST /admin/users`, `PATCH /admin/users/{id}`, `POST /admin/users/{id}/active`, `POST /admin/users/{id}/reset-password`.

A permission failure raises **HTTP 403** with `detail=f"Role '{user.role.value}' lacks permission '{permission}'"` (`rbac.py:56-59`) — naming both the caller's own role and the missing permission string, which is convenient for debugging and not a material information leak, since the caller already knows its own role from its own JWT.

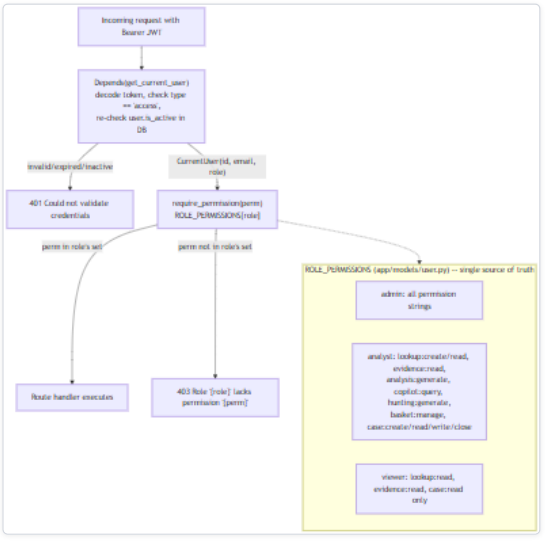

Figure 6 — Diagram: 2. Authorization: Role-Based Access Control (RBAC)

Diagram: RBAC permission resolution -- role to permission-string to route. `require_permission` is the single dependency every gated route shares; a route with no `require_permission` call enforces no permission at all, and `viewer`'s permission set contains no create/write/manage/generate string anywhere in the matrix.

Implemented:

- Three fixed roles with a centralized, single-source-of-truth permission matrix (`app/models/user.py`), checked by one shared dependency (`require_permission`) rather than ad hoc per-route logic.
- `viewer` is provably read-only: its permission set (`lookup:read`, `evidence:read`, `case:read`) contains no create/write/manage/generate permission anywhere in the matrix.
- Every route that mutates state (auth aside) requires either `analyst`-or-above or `admin`-only, per the table above.

Multi-admin user management (`app/core/users.py` , `app/api/routes/admin.py`)

Administrators are ordinary `User` rows with `role=ADMIN` — there is no separate hardcoded admin account or username. Beyond the `/auth/register` bootstrap (§1), any existing admin can create another admin directly (`POST /admin/users`), so there is no ceiling on how many active admins can exist and no single-admin bottleneck.

Two protections stop an admin action from leaving the platform with zero administrators, both in `update_user()` and `set_user_active()` :

- **Self-role-change is rejected** (`400` , `SelfRoleChangeError`) — `if user.id == actor_user_id` in `update_user()` — purely to prevent an accidental mid-session self-lockout, since role changes take effect immediately (below), not as an independent security boundary.
- **Last-admin protection is race-safe, not just check-then-act.** `_lock_all_admin_rows()` (`users.py:73-79`) takes a `SELECT ... FOR UPDATE` lock on **every** `ADMIN` -role row before `_remaining_active_admins()` (`users.py:81-82`) counts how many would stay active. A lock scoped to only the request's own target row would let two concurrent requests — each disabling a *different* one of exactly two remaining active admins — both succeed (each observes the other still active) and leave zero; locking the whole admin set serializes the second request behind the first's commit, so it re-evaluates against the post-commit state and correctly raises `LastAdminError` (`409`).

No hard delete exists, by necessity rather than by choice: `app/models/case.py` 's `analyst_id` / `added_by` / `author_id` / `generated_by` and `app/models/basket.py` 's `owner_id` are all `NOT NULL` foreign keys to `users.id` . Deleting a user who has ever created a case, added evidence, or owned a basket would either violate those constraints or require cascading away real investigation data; `set_user_active(..., False)` (disable) is the only account-removal mechanism, which also preserves the audit trail's own actor references (`config_audit_log.actor_user_id` is likewise a plain FK — even a disposable test account can't be hard-deleted once it has acted).

Immediate effect, no session-cache to invalidate. `get_current_user` already re-reads `role` and `is_active` from the database on every request rather than trusting the JWT (§1) — so a role change or a disable takes effect on the user's very next request with their existing token, with zero new machinery required for either. Password resets are the one exception genuinely needing new machinery (`token_version` , §1), since changing `hashed_password` alone does not invalidate an already-issued JWT.

Implemented (this subsection):

- Full CRUD for user accounts via `app/api/routes/admin.py` , all gated by `user:manage` .
- Race-safe last-administrator protection and self-role-change prevention.
- Immediate effect of role/enable-disable changes; `token_version` -based immediate effect for password resets.
- No hard delete — disable-only, consistent with the same FK constraints that make hard delete unsafe for real investigation data.

Recommended (not yet implemented):

- **No row-level/ownership authorization beyond the basket.** `basket_items` is scoped to `owner_id` at the query level, but `cases` and `ioc_lookups` are visible to any authenticated user holding the relevant `*:read` / `*:write` permission — there is no per-case or per-lookup ACL, only the coarse role check. This is a documented design choice (cases/lookups are team-shared, not per-analyst), but worth stating plainly: any `analyst` can read and write any other analyst's cases and lookups.

3. Credential and Secrets Management

There are two architecturally distinct places a secret can live, and a backend engineer needs to know which applies to a given credential before reasoning about exposure.

Path A — `.env` (legacy, process-frozen). `Settings` (`app/core/config.py:13-112`) is a `pydantic-settings` model loaded once from `.env` and cached for the process lifetime via `@lru_cache def get_settings()` . Every provider/AI credential field here (`virustotal_api_key` , `anthropic_api_key` , `jwt_secret_key` , etc.) is fixed at process start; changing `.env` requires a container restart to take effect for anything still reading `get_settings()` directly.

Path B — DB-backed runtime configuration (encrypted). The `provider_runtime_configs` table (`app/models/runtime_config.py:30-55`) lets an administrator add/change AI-backend and IOC-provider credentials through the Manage Providers UI while the platform is running, with no restart. Because this moves secret entry out of the git-ignored `.env` file into a database row, the credential dict is Fernet-encrypted before it is ever written: `app/core/crypto.py:49-54` (`encrypt_secret`), called from `runtime_config.py:234` / `:339` , storing ciphertext in `encrypted_credentials` (`Text` , nullable). Decryption (`crypto.py:57-67`) returns `""` on any `InvalidToken` rather than raising — a corrupted row or rotated key degrades to "not configured," not a 500.

Key material and the HKDF fallback (a documented tradeoff, not a gap)

The Fernet key is resolved by `_fernet()` (`crypto.py:39-46`): if `settings.encryption_master_key` is explicitly set, that value is the key; otherwise a key is deterministically derived via HKDF-SHA256 from `settings.jwt_secret_key` (`crypto.py:33-36`), domain-separated by a fixed `info` string so it cannot collide with any other derivation of that same secret. The module's own docstring states the intent plainly (`crypto.py:1-20`): this fallback means every existing install already has a usable encryption key with zero migration effort, at the cost of key *independence* — compromising `jwt_secret_key` also exposes the derived Fernet key. That is defense-in-depth against a raw database leak, not HSM-grade key separation, and it is a documented tradeoff rather than an oversight: an operator wanting true separation sets `encryption_master_key` explicitly (`config.py:27-30`).

Masked display — the API never returns a raw runtime-configured secret

Every read path for a runtime-configured provider (`GET /runtime/ai-providers` , `GET /runtime/ioc-providers`) goes through `_row_to_public_dict()` (`app/core/runtime_config.py:144-163`), which decrypts internally only to immediately re-mask: `masked_credentials = {k: mask_secret(v) for k, v in creds.items()}` . `mask_secret()` (`crypto.py:70-77`) replaces every character except the last four with `*` and is documented as intentionally non-reversible ("never round-trippable back to the real value, unlike returning a truncated real prefix"). No route in `app/api/routes/runtime.py` returns a decrypted credential to a client.

Windows installer: file-level protection for `.env`

For Path A, the Windows Setup Wizard does two things a plain `.env.example` -copy would not. It generates `jwt_secret_key` (and the Postgres/Neo4j passwords) with .NET's cryptographically secure `RandomNumberGenerator` , not PowerShell's `Get-Random` (`windows/scripts/Write-EnvFile.ps1` 's `New-RandomSecret` / `New-DefaultPlatformSettings` , lines 133-176). It then restricts the written file's ACL immediately: `icacls.exe $Path /inheritance:r /grant:r "*S-1-5-32-544:F" "*S-1-5-18:F"` (`Write-EnvFile.ps1:130`) — breaking inheritance and granting Full Control only to Administrators (`S-1-5-32-544`) and SYSTEM (`S-1-5-18`); the same pattern is reused for the data directory in `windows/scripts/Common.ps1:95,233` . A separate fix, `ConvertTo-SafeEnvValue` (`Write-EnvFile.ps1:15-49`), strips CR/LF from every value and rejects any value containing `#` outright, since `#` starts a comment in `.env` syntax and would otherwise silently truncate a credential with no error downstream.

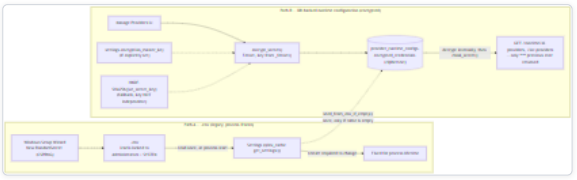

Figure 7 — Diagram: Windows installer: file-level protection for `.env`

Diagram: Credential lifecycle -- `.env` vs. the encrypted runtime store. Path A requires a restart to change and is protected only by filesystem ACLs; Path B is Fernet-encrypted at rest, never returns a decrypted value over the API, and is the only path that supports a live, no-restart credential change.

Implemented:

- Fernet symmetric encryption at rest for every runtime-configured (Path B) credential; ciphertext-only in the database, confirmed by direct model inspection (`encrypted_credentials: Text`).
- Masked-only credential display (`mask_secret`) — no API route returns a decrypted runtime-configured credential.
- Cryptographically secure secret generation and `icacls` -based NTFS permission lockdown (Administrators + SYSTEM only) for `.env` on Windows installs.
- `.env` value sanitization (`ConvertTo-SafeEnvValue`) preventing silent truncation/corruption of a written credential.
- `seed_from_env_if_empty()` (`app/core/runtime_config.py:398-448`) migrates existing `.env` credentials into the encrypted store exactly once, idempotently, on first startup of this feature — an upgrade never silently drops a previously configured key.

Recommended (not yet implemented):

- **`jwt_secret_key` defaults to the literal string `"change-me-in-production"`** if never set (`config.py:23`). Because the Fernet key for Path B falls back to a derivation of this same value, an unedited default does not just weaken JWT signing — by the tradeoff above, it also weakens runtime-credential encryption for any deployment that never sets `encryption_master_key` . (The Windows installer avoids this by always generating a real value; a bare `docker compose up` against `.env.example` without wizard involvement does not.)
- **No key-rotation procedure.** Neither `jwt_secret_key` nor `encryption_master_key` has a documented or coded rotation path; rotating either without a migration step would immediately invalidate all existing sessions and/or make every previously encrypted runtime credential undecryptable.
- **No secrets-manager integration** (Vault, AWS Secrets Manager, etc.) — both paths described above are self-contained (file-based or database-based), with no external secret store as an option.
- **No Linux/macOS equivalent of the `icacls` file-permission lockdown.** The ACL hardening described above is Windows-installer-specific; a `.env` file created by `docker compose` directly on Linux relies on the operator's own umask/filesystem permissions, which this platform does not set or verify.

4. Audit Logging

`config_audit_log` (`app/models/runtime_config.py:58-75`) is an append-only table — it uses only `UUIDPrimaryKeyMixin` , deliberately not the shared `TimestampMixin` , because it has its own explicit `timestamp` column set by application code rather than a DB `server_default` . Every row carries `actor_user_id` (nullable FK to `users.id`), a denormalized `actor_email` (so history stays readable if the account is later deleted), an `action` string, and a free-text `detail` (`String(1000)`).

All writes go through one function, `record_audit()` — originally defined inside `app/core/runtime_config.py` , extracted into a dedicated `app/core/audit.py` (`record_audit()` at line 20, `list_audit_log()` at line 39) once user-management code (`app/core/users.py`) needed the same append-only sink without importing the entire provider-config service module for it. The model itself (`ConfigAuditLog`) stays in `app/models/runtime_config.py` unchanged — only the two functions moved; `app/core/runtime_config.py` re-exports both so `app/api/routes/runtime.py` 's existing `svc.list_audit_log(...)` call needed no change. The docstring states the rule directly: `detail` "must be human-readable description text ONLY — never a credential/password/token value." This is a **coding convention enforced by review, not by a type or runtime check** — nothing in `record_audit()` inspects `detail` for secret-shaped content. Every current call site passes a hardcoded, credential-free f-string (e.g. `f"Configured AI provider '{backend}'."`); a future call site that interpolated an untrusted value would not be caught automatically.

`action` value	Emitted from	Actor recorded?
`ai_provider.configure`	`upsert_ai_provider` (`runtime_config.py`)	Yes
`ai_provider.activate`	`set_active_ai_backend` (`runtime_config.py`)	Yes
`ai_provider.test`	`record_ai_test_result` (`runtime_config.py`)	**No** — called without `actor_user_id` / `actor_email` , even though the calling route (`runtime.py`) has an authenticated `user` in scope
`ioc_provider.configure`	`upsert_ioc_provider` (`runtime_config.py`)	Yes
`ioc_provider.enable` / `.disable`	`set_ioc_provider_enabled` (`runtime_config.py`)	Yes
`ioc_provider.test`	`record_ioc_test_result` (`runtime_config.py`)	**No**, same gap
`system.seed`	`seed_from_env_if_empty` (`runtime_config.py`)	No (system-initiated)
`user.create`	`create_user` (`users.py:164-182`)	Yes
`user.update`	`update_user` (`users.py:185-224`) — only written if `full_name` / `role` actually changed	Yes
`user.enable` / `.disable`	`set_user_active` (`users.py:227-250`)	Yes
`user.password_reset`	`reset_password` (`users.py:254-272`)	Yes (the administrator performing the reset, never the affected user)
`auth.login`	`record_login_success` , called from `POST /auth/login` (`users.py:281-286`)	Yes — the logging-in user is recorded as their own actor
`auth.login_failed`	`record_login_failure` , called from `POST /auth/login` on bad credentials (`users.py:289-290`)	No (`actor_user_id=None`) — the email that was attempted is in `detail` , but there is no authenticated identity to attribute it to; the function's signature (`email: str` only) makes it structurally impossible to pass a password into `detail`

The log is exposed read-only via `GET /api/v1/runtime/audit-log?limit=200` (`app/api/routes/runtime.py`), gated by `require_permission("audit:read")` — only `admin` holds this permission per the RBAC matrix in Section 2. This single endpoint and table now serve both the Providers page's audit tab and the Administration console's audit tab (`frontend/app/admin/page.tsx`) — user-management events were deliberately routed into the *existing* audit sink rather than a second, parallel table.

Implemented:

- A dedicated, queryable audit table for every AI-backend/IOC-provider configuration change **and** every user-management/authentication event (configure, enable/disable, activate, record-test, create/update/enable/disable/password-reset, login/login-failed), including the one-time `.env` -to-database migration event.
- Actor attribution (user id + denormalized email) recorded for every configure/activate/enable/disable/user-management action, and for successful logins (self-attributed).
- `admin` -only read access to the audit trail via a dedicated permission (`audit:read`), independent from `provider:manage` / `user:manage` .
- A documented, code-level convention against ever writing a credential or password value into `detail` , reinforced for login failures by a function signature (`record_login_failure(email: str)`) that has no password parameter to leak in the first place.

Recommended (not yet implemented):

- **Test-result audit rows have no actor.** `ai_provider.test` and `ioc_provider.test` entries record *what* happened but not *who* triggered it, unlike every other action type in the table above — a straightforward fix already available since both calling routes have the current user in scope.
- **No audit coverage for case, basket, or lookup activity.** Creating/closing a case, adding/removing a basket item, or running a lookup produces no row in any audit table — only the runtime-configuration subsystem is audited.
- **No tamper-evidence or immutability guarantee.** `config_audit_log` is "append-only" purely by convention (no code path updates or deletes a row); there is no database-level `REVOKE UPDATE/DELETE` , no cryptographic chaining/hashing between rows, and no export/retention policy.
- **No enforced redaction of `detail` .** As noted above, the "never a credential" rule is a comment and a code-review norm, not a validated constraint.

5. CORS and Network Reachability

`backend/app/main.py` gates every browser-originated cross-origin request through `CORSMiddleware` , configured as a private-network-shaped regex rather than a fixed origin list:

```
PRIVATE_NETWORK_ORIGIN_REGEX = (
    r"^http://("
    r"localhost"
    r"|127\.0\.0\.1"
    r"|10\.\d{1,3}\.\d{1,3}\.\d{1,3}"
    r"|172\.(?:1[6-9]|2\d|3[01])\.\d{1,3}\.\d{1,3}"
    r"|192\.168\.\d{1,3}\.\d{1,3}"
    r")(:\d+)?$"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=[],
    allow_origin_regex=PRIVATE_NETWORK_ORIGIN_REGEX,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Why a regex over a static list. The platform is designed to be reachable from any device on the operator's local network, not just the machine it's installed on -- and the exact LAN address varies per install and per network (DHCP-assigned, never hardcoded; see the Runtime Configuration chapter's discussion of why a container can't discover the host's real LAN IP itself). A static origin list would either have to be re-edited per deployment or fall back to `allow_origins=["*"]` , which this codebase deliberately does not do. Matching by IP-shape instead answers the actual question CORS needs answered -- "is this origin on a private network" -- without needing to know the specific address in advance. `settings.debug` (`config.py:19`) is no longer read anywhere in the backend as a result of this design (confirmed via `grep -rn "settings\.debug" backend/app`) -- it previously was the sole gate on a single-origin, dev-only CORS policy.

This is a same-origin-policy relaxation, not the platform's authorization boundary. A request from a matching origin still must carry a valid bearer token for every protected route (`app/auth/rbac.py`); CORS only controls whether a browser's JavaScript is allowed to *read* the response, not whether the request reaches the API. Matching by IP-shape only, not a specific port, is a deliberate simplification: restricting to a specific configured port was evaluated and would have been nearly free to add (`docker-compose.yml` 's `env_file: .env` already injects `HOST_PORT_FRONTEND` into the backend process with no additional wiring), but was judged to add plumbing without a real security improvement, since the actual boundary is authentication, not origin matching.

`GET /network-info` (`main.py` , declared directly on `app` , no `api_v1_prefix` , no auth dependency -- the same unauthenticated pattern as `/health`) echoes three `Settings` fields for the frontend's Network Access panel:

```
@app.get("/network-info")
async def network_info():
    return {
        "detected_lan_ip": settings.detected_lan_ip,
        "frontend_port": settings.host_port_frontend,
        "backend_port": settings.host_port_backend,
    }
```

`detected_lan_ip` (`DETECTED_LAN_IP` env var) is written once, at install/reconfigure time, by the Windows wizard's `Get-LanIpAddress` (`windows/scripts/Common.ps1`) -- confirmed empirically that the backend container cannot determine this itself: self-detecting the container's own outbound-route IP from inside a running instance returned Docker's bridge-network address (`172.21.0.7` in the environment this was verified against), and resolving `host.docker.internal` returned Docker Desktop's internal VM gateway address, neither of which is the host's real LAN-facing interface. `host_port_frontend` / `host_port_backend` (`HOST_PORT_FRONTEND` / `HOST_PORT_BACKEND`) populate from the same `.env` mechanism with no additional `docker-compose.yml` wiring. None of these three values are secret; the endpoint is unauthenticated by the same reasoning as `/health` -- a LAN IP and two port numbers reveal nothing an inbound port scan on the same network wouldn't already show.

Implemented:

- CORS restricted to loopback and RFC 1918 private-IP origins only, over `http://` only -- never `allow_origins=["*"]`.
- A Windows Firewall rule (`windows/scripts/Common.ps1` 's `New-AppFirewallRule`) scoped to the platform's two ports and the **Private** network profile only, created at install/reconfigure time and removed on uninstall (`windows/installer.iss` 's `InitializeUninstall`) -- never Domain or Public profiles.
- The frontend resolves the backend's address dynamically from the browser's own request (`frontend/lib/api.ts` 's `getApiUrl()`), rather than a value baked in at build or install time -- eliminating an entire class of "works on the install machine, not from another device" bugs without needing to know any LAN address in advance.
- No credential or secret is ever transmitted to or exposed by the LAN-access mechanism itself -- `/network-info` returns only a non-secret IP address and two port numbers.

Recommended (not yet implemented):

- **No port-specific CORS restriction.** As described above, matching is by IP-shape only, not a specific configured port -- a deliberate simplification, but a stricter deployment could add it with the `HOST_PORT_FRONTEND` value already available.
- **No HTTPS/TLS anywhere in this architecture.** LAN access is `http://` only; the platform is designed for a trusted local network, not a hostile one, and has no certificate provisioning story of its own.
- `Get-LanIpAddress` **is a point-in-time snapshot, not live.** A LAN IP change (common after a router restart) is not detected automatically; the operator must re-run the wizard's Configuration option. There is no background poller or scheduled re-detection.
- **The firewall rule creation is best-effort, not verified post-install.** `New-AppFirewallRule` is wrapped in try/catch and logs a warning on failure rather than blocking setup (a locked-down/GPO-managed machine that rejects `New-NetFirewallRule` should not prevent the rest of installation from completing) -- but nothing in the wizard re-checks afterward that the rule actually exists.

Summary: Implemented vs. Recommended

Area	Strongest implemented control	Most significant open recommendation
Authentication	JWT (HS256) + bcrypt, DB-checked `is_active` on every request; `token_version` -based immediate invalidation on admin-initiated password reset; per-account rate limiter on `/auth/login` (v0.2.3)	No self-service "log out everywhere"; no rate limiting or lockout on `/auth/register` ; no account lockout on login
Authorization (RBAC)	Centralized `ROLE_PERMISSIONS` matrix, one shared `require_permission` dependency; full multi-admin user-management API (`admin.py`) with race-safe last-administrator protection; `lookup:export` enforced on the export route, distinct from `lookup:read`	No per-case/per-lookup ACL
Credentials/Secrets	Fernet-at-rest encryption for DB-backed provider/AI credentials; masked-only display; `icacls` -restricted `.env` on Windows	Default `jwt_secret_key` literal risk; encryption key derivation shares root secret with JWT signing unless `encryption_master_key` is set explicitly; no rotation procedure
Audit Logging	Actor-attributed log of every provider/AI configuration change **and** every user-management/login event, `admin` -only read	No coverage of case/lookup events; test-result entries lack actor attribution; append-only by convention only
CORS/Network	Private-network-only CORS regex + Private-profile-only Windows Firewall rule; never `allow_origins=["*"]`	No port-specific CORS restriction; no HTTPS/TLS; LAN-IP detection is a point-in-time snapshot with no live re-detection

None of the "Recommended" items are claimed vulnerabilities being actively exploited in a deployed instance — they are the concrete, code-verified list of controls a production hardening pass would add next.

Background Processing and Caching

This chapter covers two backend subsystems that operate outside the hot path of an interactive lookup: the **Celery worker/beat stack** (`backend/app/workers/`) and **Redis** (`backend/app/core/cache.py`), which serves as the provider-result cache, the rate limiter, and the Celery message transport. They are covered together because they share one physical dependency — a single Redis container backs the cache, the rate limiter, and Celery's broker/result-backend, split across three logical database indexes.

The fact to internalize first: `POST /api/v1/lookup/stream` **— the core investigation endpoint — never touches Celery.** Provider fan-out, correlation, AI summarization, and the final assessment run entirely in-process inside the FastAPI request/response lifecycle and stream back as Server-Sent Events. Celery exists for exactly one job, described below, with no code path that can block or delay a live investigation.

Celery: what actually runs

`backend/app/workers/celery_app.py` defines the Celery application:

```
celery_app = Celery(
    "ioc_intel_platform",
    broker=settings.celery_broker_url,
    backend=settings.celery_result_backend,
    include=["app.workers.tasks"],
)
```

Its module docstring describes the intended scope as "scheduled crawler runs, PDF/report export rendering, and any lookup re-processing that outlives an HTTP request lifecycle." Checked against the actual code in `app/workers/tasks.py` , two of those three named use cases — PDF/report export rendering and lookup re-processing — do not exist as implemented tasks. There is exactly **one** registered task in the codebase, `run_osint_crawl` , wired into a `beat_schedule` that fires it once an hour:

```
celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="UTC",
    enable_utc=True,
    task_track_started=True,
    beat_schedule={
        "crawl-osint-sources-hourly": {
            "task": "app.workers.tasks.run_osint_crawl",
            "schedule": 3600.0,
        },
    },
)
```

Stated plainly for capacity planning: **the Celery worker and beat containers exist and are actively used, but for a single low-volume hourly maintenance job, not a general-purpose task queue backing multiple product features.** Report generation (`case_reports` exists as a table — see the database chapter) and asynchronous lookup re-processing are not implemented as Celery tasks despite being named in the module's own docstring as in-scope; if either is added later, this is the natural place, but neither exists today.

`run_osint_crawl` — the one scheduled job

Defined in `backend/app/workers/tasks.py` . Its own module docstring explains the design rationale directly: rather than crawl an arbitrary hardcoded list of search terms, the task re-runs the OSINT crawler (`app/crawler/collector.py` 's `internet_intelligence_provider` , the same "Internet Intelligence Collector" provider documented in the Providers chapter) against IOCs that were **actually looked up recently** by real users, so crawler capacity is spent on what analysts are investigating rather than on speculative targets.

Mechanics, in order, on each hourly tick:

1. `_recent_crawlable_iocs()` queries `ioc_lookups` for rows where `ioc_type` is one of the crawler-eligible types — `domain` , `ipv4` , `malware_family` , `threat_actor` , `campaign` , `cve` , `file_name` (must stay in sync with `app/crawler/collector.py` 's `_SUPPORTED_TYPES` , since free-text OSINT search on a raw hash or IP is treated as too noisy to be useful) — and `created_at` within the last **24 hours** (`_LOOKBACK_HOURS`). It over-fetches (`_MAX_IOCS_PER_RUN * 10` rows) and de-duplicates `(ioc_value, ioc_type)` pairs in Python (Postgres rejects `SELECT DISTINCT ... ORDER BY created_at` when `created_at` isn't in the select list), then caps the result at **25 IOCs per run** (`_MAX_IOCS_PER_RUN`) — a fixed bound so one beat tick never scales with lookup volume.
2. No crawlable IOCs in the window → logs `"no recently-investigated crawlable IOCs, nothing to do"` and returns `0` . This is the expected outcome on a lightly-used instance, not an error.
3. Otherwise it opens one shared `httpx.AsyncClient` (30s timeout, redirects followed) and calls `_crawl_one()` per target sequentially, each wrapped in its own `try/except` so one bad target cannot abort the run — logged as a warning and skipped.
4. `_crawl_one()` calls `internet_intelligence_provider.run(...)` — the same `BaseProvider.run()` entry point live investigations use — and, only if the result status is `ProviderStatus.OK` , writes it into the same Redis cache via `set_cached_result(...)` with `settings.provider_cache_ttl_seconds` (default 3600s) as TTL. Non- `OK` results are discarded, not cached.

Net effect: OSINT crawler results stay warm in the shared cache for IOCs users are actively investigating, so a second lookup within the TTL gets a cache hit instead of a fresh, rate-limit-sensitive crawl across GitHub/Reddit/RSS/Pastebin.

Concurrency note: the task function itself is synchronous (`def run_osint_crawl() -> int`), but its body is `async` . Since a Celery worker process has no running event loop to attach to, the task wraps its body in `asyncio.run(_run_osint_crawl_async())` — a fresh event loop per execution, per the module's own comment. It therefore cannot share a connection pool or event loop with the FastAPI process; it opens its own DB session (`new_session()`) and its own `httpx.AsyncClient` each run.

Task registry — the full picture

Task name	Trigger	Frequency	File
`app.workers.tasks.run_osint_crawl`	Celery Beat (`crawl-osint-sources-hourly` schedule entry)	Every 3600 seconds (1 hour)	`backend/app/workers/tasks.py:119-124`

That is the entire task registry. No other `@celery_app.task` decorator exists anywhere in the backend codebase — confirmed by the fact that `celery_app.py` 's `include=["app.workers.tasks"]` names the only module Celery is told to import tasks from, and that module defines exactly one.

How the worker and beat containers are wired

Both `celery_worker` and `celery_beat` in `docker-compose.yml` build from the **same backend image** as the FastAPI `backend` service (`build: {context: ./backend}`), and differ only in their `command:` :

Container	Command	Role
`celery_worker`	`celery -A app.workers.celery_app worker --loglevel=info`	Executes tasks pulled from the broker queue.
`celery_beat`	`celery -A app.workers.celery_app beat --loglevel=info`	Emits the scheduled `run_osint_crawl` message onto the broker every hour; does not execute tasks itself.

Both are started with `restart: unless-stopped` , and both depend on Redis being healthy (`depends_on: redis: condition: service_healthy`) before starting; `celery_worker` additionally depends on Postgres, since its one task reads `ioc_lookups` . Neither container runs `alembic upgrade head` — only the `backend` service's startup command does that, so the workers assume the schema is already current by the time they start.

The Kubernetes manifests (`k8s/base/celery-worker-deployment.yaml` , `k8s/base/celery-beat-deployment.yaml`) mirror this: same `ioc-intel-platform/backend:latest` image, same `celery ... worker` / `celery ... beat` commands, same `ioc-intel-config` / `ioc-intel-secrets` ConfigMap/Secret as the backend Deployment. Two operational details: **`celery-worker` is scaled to 2 replicas** (250m/512Mi requested, 1000m/1Gi limit) since multiple workers can safely share one queue; **`celery-beat` is pinned to 1 replica**, with a manifest comment explaining why — "celery beat is a singleton scheduler and running more than one replica would produce duplicate scheduled task dispatches."

Production Compose (`docker-compose.prod.yml`) leaves both commands unchanged and only strips bind-mounted `volumes:` (`volumes: !reset []`), the same treatment given to `backend` / `frontend` . Its header comment notes why: WSL2 bind-mount permissions under Program Files caused `celery_beat` 's schedule-file writes to fail with `PermissionError` in dev, motivating removing volumes across the board.

Redis: the shared dependency

Redis (`redis:7-alpine`) is the one piece of infrastructure the two subsystems in this chapter share, partitioned into three logical database indexes on a single Redis instance/container rather than three separate processes:

Redis URL (as set in **`docker-compose.yml`** / **`config.py`**)	Logical DB	Consumer	Purpose
`redis://redis:6379/0` (`settings.redis_url`)	`/0`	`app/core/cache.py` , read directly by the FastAPI backend process and by the `run_osint_crawl` Celery task	Provider-result cache + rate-limiter counters
`redis://redis:6379/1` (`settings.celery_broker_url`)	`/1`	Celery worker + beat	Message broker — task dispatch queue
`redis://redis:6379/2` (`settings.celery_result_backend`)	`/2`	Celery worker + beat	Result backend — stores task return values/state (`task_track_started=True` is set, so state transitions are recorded too)

All three URLs are injected identically into `backend` , `celery_worker` , and `celery_beat` in `docker-compose.yml` 's `environment:` block — the FastAPI process and the Celery containers reach the same cache/broker/backend with no cross-container RPC, just three processes pointed at the same Redis host with three different `/N` suffixes. This separation means a `FLUSHDB` or eviction event against the cache (`/0`) cannot corrupt in-flight Celery task state (`/1` , `/2`), even though all three share one Redis process and memory budget.

Client construction

`app/core/cache.py` builds its Redis connection lazily, once per process, via a module-level `_pool` global populated on first call to `get_redis()` : `aioredis.from_url(get_settings().redis_url, decode_responses=True)` . This is the async `redis.asyncio` client (package `redis==5.0.8` per `backend/requirements.txt`); `decode_responses=True` means every value this module handles is a Python `str` , not `bytes` . This client always talks to `/0` — it is unrelated to, and shares no connection with, Celery's own broker/backend clients (which Celery manages internally against `/1` / `/2`). There is exactly one `aioredis.Redis` client per backend/worker process, reused for the life of that process; it is never explicitly closed or recycled.

Provider-result cache

`cache_key(provider_id, ioc_type, ioc_value)` builds keys of the form:

```
provider_cache:{provider_id}:{ioc_type}:{sha256(ioc_value)}
```

The IOC value itself is SHA-256-hashed before being embedded in the key — `hashlib.sha256(ioc_value.encode("utf-8")).hexdigest()` — rather than stored in the clear as part of the Redis key name. `get_cached_result()` / `set_cached_result()` are thin `GET` / `SET ... EX <ttl>` wrappers around that key, serializing the cached payload with `json.dumps(payload, default=str)` (the `default=str` handles any non-JSON-native Python value a provider's `ProviderResult.to_dict()` might contain, e.g. a `datetime`) and deserializing with `json.loads()` on read.

The cache is consulted from exactly two call sites, both of which use the identical key scheme so a write from one is visible to a read from the other:

1. **`app/providers/orchestrator.py` 's `_run_with_policy()`** — checked *before* any live HTTP call is attempted, for every applicable provider, on every investigation. A hit short-circuits the entire fetch/retry/timeout pipeline for that provider and is marked `result.from_cache = True` so the UI can distinguish a fresh answer from a cached one. A write only happens `if result.status == ProviderStatus.OK` — results with any other status (`NO_DATA` , `ERROR` , `TIMEOUT` , `RATE_LIMITED` , `NOT_CONFIGURED` , `UNSUPPORTED`) are never cached. This is a deliberate asymmetry worth knowing operationally: a provider that currently has no data for an IOC (`NO_DATA`) will be queried again, in full, on every subsequent lookup of that same IOC until it eventually returns `OK` — there is no negative-result caching.

2. `app/workers/tasks.py`'s `_crawl_one()` (the hourly OSINT job) — writes only, using the same `set_cached_result()` function and the same TTL setting, refreshing the crawler's entry in this same cache so a subsequent interactive lookup of that IOC can hit it.

TTL for both call sites is `settings.provider_cache_ttl_seconds`, defaulting to **3600 seconds (1 hour)** (`app/core/config.py:103`). There is no cache-busting endpoint or manual invalidation path anywhere in the API surface — the only way a stale cached provider result is refreshed before its TTL expires is if a provider's underlying data changes and someone explicitly needs a fresh read, in which case the only lever available today is waiting out the TTL (there is no "force refresh" flag on `POST /api/v1/lookup/stream`).

Rate limiting

The same module defines `RateLimiter`, a Redis-backed fixed-window counter:

```
class RateLimiter:
    def __init__(self, provider_id: str, max_calls: int, window_seconds: int) -> None:
        self._key = f"rate_limit:{provider_id}"
        self._max_calls = max_calls
        self._window_seconds = window_seconds

    async def allow(self) -> bool:
        r = get_redis()
        current = await r.incr(self._key)
        if current == 1:
            await r.expire(self._key, self._window_seconds)
        return current <= self._max_calls
```

The constructor parameter is named `provider_id`, but this is a generic identifier for whatever the caller wants to key the window by — it is not restricted to provider identifiers. The one place this class is actually instantiated in the codebase is `app/api/routes/lookup.py`'s `POST /api/v1/lookup/stream` handler, which keys it **per authenticated user**, not per provider:

```
limiter = RateLimiter(
    f"lookup_create:{user.id}",
    max_calls=settings.lookup_rate_limit_max_calls,
    window_seconds=settings.lookup_rate_limit_window_seconds,
)
```

Defaults are `lookup_rate_limit_max_calls = 10` and `lookup_rate_limit_window_seconds = 60` (`config.py:106-107`) — at most 10 new lookups per user per rolling 60-second fixed window. `INCR` on a key that doesn't exist initializes it to `1`; the code sets the key's expiry only on that first increment (`if current == 1: await r.expire(...)`), the standard fixed-window pattern — a burst straddling a window boundary can momentarily exceed the nominal limit, a known fixed-window tradeoff the code does not correct for. Exceeding the limit returns `HTTP 429` with a detail message explaining why the limit exists: a single lookup fans out to every provider plus the crawler plus multiple AI calls, so the per-user cap bounds aggregate downstream cost/load, not just request count.

Being Redis-backed, this limiter stays correct if the FastAPI backend is horizontally scaled to multiple replicas — the module's stated reason for not using an in-process counter. Contrast the crawler's own **separate** in-process limiter, `AsyncMinIntervalLimiter` (`app/crawler/sources/rate_limit.py`), which spaces out GitHub (6.0s) and Reddit (1.1s) search calls within the crawler's OSINT sub-sources using a plain `asyncio.Lock` + monotonic clock. It is intentionally not Redis-backed and not shared with `RateLimiter`, since it only needs to bound one process's own outbound rate to a third-party API, not enforce a global cross-replica cap.

Operational notes and gaps

- **No dedicated automated tests target `app/core/cache.py` or `app/workers/tasks.py` directly.** No `test_cache*.py` or `test_workers*.py` / `test_tasks*.py` file exists under `backend/app/tests/`. Cache coverage is indirect, via provider/orchestrator tests exercising cache-hit/miss branches as a side effect; the hourly task has no unit test of its own.
- **No cache metrics or hit/miss counters are exposed.** The only observability into cache behavior is the `from_cache` boolean on each `ProviderResult`, visible per-lookup — there is no aggregate hit-rate metric on the Prometheus `/metrics` endpoint.
- **No Flower (or equivalent) Celery monitoring UI exists anywhere in this codebase** — task history/state is inspectable only via the result backend (`/2`) directly or worker/beat container logs.
- **A Redis outage has two blast radii at once.** Because the cache/rate-limiter (`/0`) and the Celery broker/backend (`/1`, `/2`) share one physical Redis process, an outage fails both simultaneously — interactive lookups fail at `RateLimiter.allow()` before the cache is even consulted, and the hourly job fails to dispatch. No fallback path (fail-open limiter, in-memory cache) exists for a Redis-unavailable condition; Redis should be treated as a hard dependency of the lookup-creation endpoint, not merely a performance optimization.

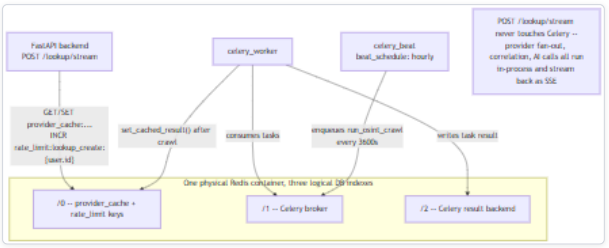

Figure 8 — Diagram: Operational notes and gaps

Diagram: Celery worker/beat topology and the three logical Redis database indexes (`/0` cache + rate-limit, `/1` broker, `/2` result backend) shared between the FastAPI backend, the Celery worker, and the Celery beat scheduler. A single Redis outage takes down both the cache/rate-limiter and the Celery broker/backend at once, since all three indexes live in one physical process.

Summary

Celery is real, running infrastructure in every deployment of this platform (Compose and Kubernetes alike), but currently backs exactly one job — an hourly re-crawl of OSINT sources for recently-investigated, crawler-eligible IOCs, capped at 25 per run — with no involvement in the interactive, SSE-streamed lookup pipeline, which runs entirely in-process. Redis is the shared substrate for both subsystems, split by logical index into a provider-result cache plus fixed-window rate limiter (`/0`) and a Celery broker/result-backend pair (`/1` , `/2`). The cache is positive-only (no `NO_DATA` /error caching) with a one-hour default TTL and no manual invalidation path; the rate limiter is per authenticated user on the lookup-creation endpoint, chosen specifically because a Redis-backed counter stays correct under horizontal scaling, unlike the crawler's separate, process-local `AsyncMinIntervalLimiter` . Redis should be treated as a hard dependency of the live lookup path via the rate limiter, not just a cache, and Celery should not be assumed to be doing more than this one documented hourly job.

Deployment and Troubleshooting

This chapter covers the two deployment paths that exist in this repository -- Docker Compose (development and production) and the Windows Inno Setup installer that wraps it -- how environment variables reach the running process, and a symptom-first troubleshooting reference keyed to real error strings and enum values. Performance/scaling claims are limited to what is actually configured in code or measured in the project's own QA pass; see the Performance chapter for the full measured-value table.

All facts below are drawn from `docker-compose.yml`, `docker-compose.prod.yml`, `backend/Dockerfile`, `backend/app/core/config.py`, `backend/app/core/db.py`, `backend/app/workers/celery_app.py`, `backend/app/providers/base.py`, `backend/app/providers/orchestrator.py`, `backend/app/main.py`, `k8s/base/*.yaml`, `windows/installer.iss`, `windows/wizard/Setup-Wizard.ps1`, `windows/scripts/*.ps1`, cross-checked against `docs/DEPLOYMENT.md` / `docs/TROUBLESHOOTING.md`, and this documentation set's Runtime Configuration Architecture and System Overview chapters, which this chapter does not re-derive.

1. Docker Compose

Compose is the only deployment path fully wired end-to-end and exercised by this project's own test suite. `docker-compose.yml` defines eight services: four datastores (`postgres`, `redis`, `neo4j`, `opensearch`, all published to `127.0.0.1` only -- an explicit fix after confirming each was reachable unauthenticated from other LAN devices under the old `0.0.0.0` shorthand), `backend`, two Celery roles (`celery_worker`, `celery_beat`), and `frontend`. `backend` and `celery_worker` wait on Postgres/Redis health checks (`pg_isready` / `redis-cli ping`, 5s interval, 10 retries); `celery_beat` waits only on Redis. Only the two Celery services carry `restart: unless-stopped` -- `backend`, the datastores, and `frontend` have no restart policy, so a crashed container stays down until manually recreated.

1.1 Dev vs. production: what `docker-compose.prod.yml` changes

Applied as an override, not a replacement: `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`. It touches only `backend`, `celery_worker`, `celery_beat`, and `frontend`:

Change	Dev (base)	Prod (override)	Why
`backend` volumes	`./backend:/app` bind-mount	`!reset []` (none)	Runs entirely from the built image
`backend` command	`alembic upgrade head && uvicorn ... --reload`	same, minus `--reload`	No hot reload in production
`celery_worker` / `celery_beat` volumes	bind-mount	`!reset []`	Windows/WSL2 bind-mount permissions broke `celery_beat`'s schedule-file write (`PermissionError`) under an installed copy
`frontend` volumes	source + anonymous `node_modules` / `.next` volumes	`!reset []`	The anonymous volumes failed to mount at all under an installed copy -- confirmed live: container stuck at `Created`, never started
`frontend` command	`npm run dev`	`npm run build && ... && node .next/standalone/server.js`	Real production build, not the dev server

The production frontend command works around two real gotchas: `next.config.js` sets `output: "standalone"`, which plain `next start` doesn't support (confirmed live: `"next start" does not work with "output: standalone" configuration`), so it runs `node .next/standalone/server.js` directly; and since Next's standalone build excludes `public/` / `.next/static/`, the command copies them in by hand after building.

No datastore credential or networking behavior differs between the two files -- Postgres/Neo4j/OpenSearch still run with default/hardcoded credentials (`ioc` / `ioc`, `neo4j` / `changeme-neo4j`) and OpenSearch's security plugin is disabled in both; hardening those is an operator responsibility neither file addresses.

`backend/Dockerfile`'s baked-in `CMD` is plain `uvicorn app.main:app --host 0.0.0.0 --port 8000` -- it does **not** run `alembic upgrade head`; migration is a Compose-layer `command:` override, not an image-layer step, in both dev and prod. Running the built image directly (`docker run`, bypassing Compose) starts the API without applying pending migrations. `celery_worker` / `celery_beat` never run migrations themselves; they rely on `backend` to bring the schema current, since Alembic migrations are idempotent.

1.2 Kubernetes (`k8s/base/`)

A plain-YAML, kustomize-compatible, hand-translated mirror of the Compose services also exists, with no Helm templating and no CI/CD wiring anywhere in the repo. `backend` / `frontend` run 2 replicas; `celery-beat` is pinned to 1 (avoids duplicate scheduled dispatches). `backend` probes `GET /health` for readiness and liveness; resource requests/limits (`backend` / `celery-worker`: `250m` / `512Mi` request, `1000m` / `1Gi` limit; `frontend`: `100m` / `256Mi` request, `500m` / `512Mi` limit) are static manifest values, not load-test-derived. TLS is commented out in `ingress.yaml` and not implemented, and `frontend/Dockerfile` as committed is dev-mode only -- it cannot satisfy the Deployment's `npm start` without a separate production Dockerfile that does not exist in this repo.

Migrations in this topology. `k8s/base/backend-deployment.yaml:31-34` sets the same `command: sh -c "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000"` used by Compose (minus `--reload`, per the manifest's own header comment, lines 1-5) -- there is no separate Kubernetes Job or init container that runs the migration once; each of the Deployment's individual pods runs `alembic upgrade head` itself, every time that pod starts. With `replicas: 2`, a rollout can start both pods within the same window, and nothing in the manifest serializes their two `alembic upgrade head` invocations against each other -- Alembic has no built-in cross-process lock, so two pods racing to apply the same not-yet-applied revision concurrently is a real, unmitigated possibility in this manifest as written, not merely a Compose-vs-K8s documentation gap. `celery-worker`'s Deployment does not run `alembic upgrade head` at all (same division of responsibility as Compose), so it depends on one of the two `backend` pods having already brought the schema current.

2. The Windows Inno Setup Installer

Its scope is narrow: **the installer copies files and collects configuration; it does not itself start Docker.** `installer.iss` runs prerequisite checks (`Check-Prerequisites.ps1`: 64-bit Windows 10+, admin privileges, 8GB+ RAM soft check, disk space, Docker Desktop running, Compose v2, free ports -- failures prompt "Continue anyway?" rather than hard-aborting), copies `backend/`, `frontend/`, both compose files, `docs/`, `README.md`, and the wizard/scripts into `%ProgramFiles%\IOC Intelligence Platform\app\`, sets up Start Menu shortcuts, then launches the Setup Wizard (a WinForms app) -- the component that actually configures and starts anything.

On "Start Installation" the wizard runs, in order: (1) writes `.env` via `Write-PlatformEnvFile`, using collected values plus CSPRNG-generated secrets (`ConvertTo-SafeEnvValue` strips CR/LF and rejects any value containing `#`, since `#` opens a `.env` comment and would silently truncate the rest); (2) runs `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` --

this is the moment Docker actually starts, nothing before it brings up a single container; (3) polls `GET /health/detailed` for up to 3 minutes (updated in v0.2.3 -- the wizard needs to know Postgres is genuinely reachable before the very next step writes to it, not just that the process has started; see the System Health chapter for the plain-`/health`-vs-`/health/detailed` split); (4) registers the very first administrator account via `POST /api/v1/auth/register` and logs in -- self-registration is bootstrap-only: it succeeds only while the users table is empty, and every subsequent call (including a second install attempt against an already-configured database) is rejected with `403`, directing the caller to have an existing administrator create their account instead. Re-running the wizard via the **Configuration** shortcut on an existing install pre-populates fields from the existing `.env` and takes a `pg_dump` backup first (both platforms, as of v0.2.3 -- previously Windows-only).

Real secrets live at `%ProgramData%\IOC Intelligence Platform\config\.env`, locked to Administrators+SYSTEM via `icacls`. Because Compose's `env_file: .env` resolves relative to the project directory (`{app}\app\`), a helper copies that file into `{app}\app\.env` on every Compose invocation and re-applies the same ACL -- two on-disk `.env` files with real secrets, both locked down.

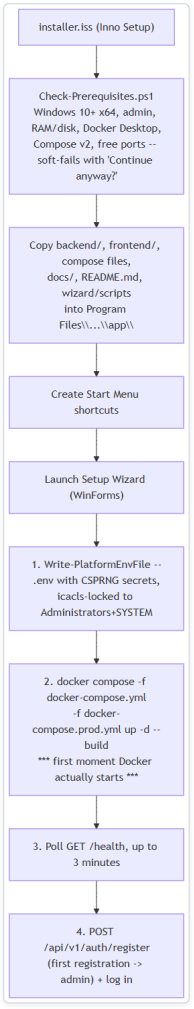

Figure 9 — Diagram: 2. The Windows Inno Setup Installer

Diagram: Installer file-copy vs. Setup Wizard Docker/registration steps. Everything before step G is file staging and prerequisite checking only -- the installer itself never starts a container; that happens exclusively inside the wizard's second step.

3. Environment Variable Configuration

`backend/app/core/config.py` 's `Settings` class (`pydantic-settings` , `env_file=".env"`) is the exhaustive list the legacy `.env` path supports; `.env.example` is the operator-facing template.

Group	Representative fields	Notable default
App	`debug`	`True` -- undeclared `DEBUG=false` means CORS is restricted to only `http://localhost:3000` regardless of the real frontend host
Security	`jwt_secret_key`, `encryption_master_key`	`jwt_secret_key` defaults to the placeholder `"change-me-in-production"`; `encryption_master_key` falls back to an HKDF-derived key from the JWT secret if unset
Datastores/Celery	`database_url`, `redis_url`, `neo4j_*`, `opensearch_url`, `celery_broker_url` / `celery_result_backend`	Redis DB `1` / `2` for broker/result, DB `0` for app cache/rate-limiter
AI backends	`ai_backend` + per-backend key/model fields (Ollama/Anthropic/Bedrock/Gemini/Groq/OpenAI/Kimi/DeepSeek/xAI/Mistral/OpenRouter)	`ai_backend` defaults to `"ollama"`
IOC providers	one `..._api_key` per keyed provider (Censys needs a token + org ID pair)	All unset by default -- keyed providers report `not_configured` until set
Provider execution	`provider_timeout_seconds`, `provider_max_retries`, `provider_cache_ttl_seconds`	`20`, `2`, `3600`
Rate limiting	`lookup_rate_limit_max_calls`, `lookup_rate_limit_window_seconds`	`10`, `60`

Two mechanisms govern whether a change actually takes effect, and they differ sharply. First, **a Compose `environment:` entry always wins over `env_file:`** -- `backend`'s `OLLAMA_BASE_URL` is set directly in `docker-compose.yml`'s `environment:` block rather than left to `.env` alone, so it silently overrides whatever `.env` says (it still respects a real *shell/host* `OLLAMA_BASE_URL`, just not one written only into `.env`). Second, **`Settings` is a process-lifetime, `@lru_cache`'d singleton**, read once at import time and never re-read: `docker compose restart backend` re-reads nothing, since Compose only re-reads `.env` at container *creation*; use `docker compose up -d backend` to force recreation.

This frozen-`.env` behavior is exactly the legacy path superseded, for provider/AI credentials specifically, by the database-backed runtime configuration system (`provider_runtime_configs`, the Manage Providers UI) covered in full in the Runtime Configuration Architecture chapter -- credentials saved there take effect on the next investigation or AI call, no restart. `.env` remains authoritative for everything that system doesn't cover (JWT key, datastore URLs, Celery broker/result, rate limits, provider timeouts/retries/cache TTL).

4. Troubleshooting

4.1 Where to look first

1. **`docker compose ps`** -- which containers are actually running vs. exited (no `restart` policy on `backend`/datastores/`frontend`).
2. **`docker compose logs backend`** (or `celery_worker` / `celery_beat` / `frontend`) -- structured JSON (`structlog`, configured before anything else in `app/main.py`), so early startup failures land in the same format as runtime logs.
3. **`GET /health`** (no auth, outside `/api/v1`) -- `{"status": "ok", "service": "HORIZON GRID"}` the moment Uvicorn accepts connections; it does **not** check Postgres/Redis/Neo4j/OpenSearch, only that the process itself is up. This is what the Setup Wizard and the Kubernetes readiness/liveness probes poll.
4. **`GET /api/v1/providers/health`** -- which IOC providers are currently `configured`, distinct from container health.
5. **`GET /api/v1/runtime/audit-log`** -- every configure/enable/disable/activate/record-test action against a provider or AI backend, with actor/timestamp/descriptive detail (never a credential value). Fastest way to see whether "was working yesterday" correlates with a configuration change.
6. **`GET /metrics`** -- Prometheus-format, but no Prometheus/Grafana is bundled; useful only with your own scraper.
7. **Windows Diagnostics bundle** (`windows/scripts/Diagnostics.ps1`) -- zips `docker compose ps` / `logs`, the setup log, and a *redacted* `.env` (`***REDACTED***(set, N chars)` or `(not set)`, variable names only) to the Desktop; the real `.env` is never included.

4.2 Startup failures

Symptom	Cause	Fix
`backend` exits immediately, non-zero, before Uvicorn logs anything	`alembic upgrade head` failed inside the `sh -c "alembic upgrade head && uvicorn ..."` wrapper -- migrations run synchronously before `uvicorn` even execs, so a failed migration means no API start at all (not a stale-schema start)	Check `docker compose logs backend` for the Alembic traceback; confirm `postgres` healthy first, then `docker compose up -d backend`
Missing/incomplete `.env`	Every `Settings` field has a code-level default, so a missing `.env` does **not** crash startup -- it starts with insecure defaults (placeholder JWT key) or providers reporting `not_configured`	Copy `.env.example` to `.env`, fill in real values, and recreate (not restart) `backend`
A `.env` edit has no visible effect	`docker compose restart` doesn't re-read `.env`; `Settings` is cached for the process's life regardless	`docker compose up -d backend`
`frontend` / `celery_beat` stuck at `Created` or `PermissionError` on an installed (Program Files) copy	Bind-mount/anonymous-volume permissions break under Windows/WSL2 outside a real git tree	Use `docker-compose.prod.yml` (what the wizard already does)

4.3 Provider and AI failures

Every IOC provider call normalizes to one `ProviderStatus` enum value (`backend/app/providers/base.py`) -- the authoritative vocabulary for any provider symptom, visible in both the SSE `provider_result` payload and `GET /api/v1/lookup/{id}`:

`ProviderStatus`	Meaning / typical trigger
`ok`	Usable data returned
`no_data`	Reached successfully, nothing on this IOC (404 or explicit "not found" field)
`error`	Non-2xx not classified as rate-limited, or unhandled exception (e.g. abuse.ch's auth-rejected statuses map here)
`timeout`	Exceeded `provider_timeout_seconds` (default `20`), enforced uniformly via `asyncio.wait_for`
`rate_limited`	Vendor returned `429`, `403`, or `509` (509 is PhishTank's documented over-limit code)
`not_configured`	`requires_key=True` with no effective credential -- key never set, or set in `.env` but the container hasn't been recreated yet
`unsupported_ioc`	Provider doesn't support the detected IOC type
`disabled`	Administrator turned it off at runtime (`enabled=false`) -- distinct from `not_configured`: a valid key can still be intentionally disabled

Only connection-level failures (`httpx.ConnectError` / `ReadTimeout` / `PoolTimeout`) are retried, up to `provider_max_retries` additional attempts (default `2`, exponential backoff `wait_exponential(multiplier=0.5, max=4)`). A `429` / `403` / `509` is reported as `rate_limited` immediately, never retried.

Test Connection succeeding while a real investigation still reports `not_configured` is a specific, previously-real gap, now closed by the runtime configuration system (full root cause and fix in the Runtime Configuration Architecture chapter). Short version: a credential saved via the Manage Providers UI is visible to the next investigation immediately; one only ever written to `.env` by hand still needs `docker compose up -d backend`, not just a restart.

AI backend failures raise `RuntimeError` from the relevant client, caught by `app/ai/service.py` and converted to a degraded static result (verdict `unknown`, scores `0`) rather than failing the lookup. Only Bedrock retries (`botocore`-level, `BotoConfig(retries={"max_attempts": 3, "mode": "adaptive"})`); every other backend makes exactly one attempt. Common real messages: `"Could not reach Ollama at http://host.docker.internal:11434 -- is it running?"` (Ollama not running on the host -- it's deliberately never containerized, since a large model's mmap load was observed dramatically slower through Docker Desktop's WSL2 volume layer); `"Ollama did not respond within 120s (model=llama3.2:3b) -- likely too slow for available hardware"` (hardcoded `_TIMEOUT_SECONDS`, not a `.env` setting -- fix with a smaller model, not a bigger timeout); and, for any backend, `"AI backend '{backend}' is not configured (missing API key/URL/model) -- ..."` if `is_configured` is still false after resolution.

`429` on `POST /api/v1/lookup/stream` is the platform's only rate limiter -- a Redis-backed fixed-window `RateLimiter` keyed `lookup_create:{user.id}`, per user across all workers, bounded by `lookup_rate_limit_max_calls` / `_window_seconds` (`10` / `60`). Exists because one lookup fans out to every provider plus the crawler plus multiple AI calls. No other endpoint (including login/register) has any rate limiting.

5. Performance and Scaling Notes (evidenced only)

Only what is actually configured in code or measured in the project's own QA pass -- see the Performance chapter for the full measured-value table and its single-machine, single-pass caveats.

- **DB connection pool**: `backend/app/core/db.py` creates one process-wide `AsyncEngine` (`create_async_engine(..., pool_pre_ping=True, echo=False)`), shared by `get_db()` and `new_session()`. No `pool_size` / `max_overflow` override is set -- SQLAlchemy's own library defaults govern the ceiling.
- **Provider fan-out**: `run_all_providers()` opens one `httpx.AsyncClient` per investigation with `httpx.Limits(max_connections=50, max_keepalive_connections=20)`, dispatching every applicable provider as a concurrent `asyncio.create_task` collected via `asyncio.as_completed` -- results stream as each finishes rather than waiting for the slowest.
- **Per-provider timeout/retry/cache**: `provider_timeout_seconds=20`, `provider_max_retries=2` (connection errors only), `provider_cache_ttl_seconds=3600` (Redis-cached `ok` results skip the outbound call entirely on a repeat lookup within the hour).
- **Celery concurrency**: no `--concurrency` flag is set anywhere in either compose file or `celery_app.py` -- Celery's own default (one prefork process per detected CPU core) governs it, not an application setting. The interactive SSE lookup path never goes through Celery at all, so this has no bearing on lookup latency.
- **Kubernetes-only figures**: `backend` / `celery-worker` request `250m` / `512Mi` (limit `1000m` / `1Gi`); `frontend` requests `100m` / `256Mi` (limit `500m` / `512Mi`); `backend` / `frontend` run 2 replicas, `celery-beat` is a pinned singleton. Static manifest values, not load-derived.
- **No load-testing infrastructure exists in this repository** -- no benchmark harness, no CI/CD to run one. Any broader concurrent-load characterization would not be traceable to evidence here and is intentionally omitted.

6. Summary

Two working deployment paths exist: Docker Compose (a small, surgical production override that drops dev bind-mounts and swaps in production start commands) and the Windows installer, which only copies files and launches the Setup Wizard -- the wizard, not the installer, writes `.env` and runs `docker compose up`. `Settings` is a process-lifetime singleton read once from `.env`, superseded for provider/AI credentials by the database-backed runtime configuration system covered elsewhere in this set. Troubleshoot in order: `docker compose ps` / `logs`, `/health`, `GET /api/v1/providers/health`, the runtime audit log -- and reason about provider failures via the eight `ProviderStatus` values, not raw HTTP codes. Performance/scaling statements here are limited to what is actually configured (pool defaults, a 20s/2-retry provider policy, a 1-hour result cache, per-core Celery concurrency, static Kubernetes resource requests) -- there is no load-testing harness in this repository to support any broader claim.

Security Assessment Toolkit Architecture

This chapter documents the Security Assessment Toolkit (`backend/app/security_assessment/` , `backend/app/core/security_assessment.py` , `backend/app/api/routes/security_assessment.py`) — the platform's one and only subsystem that sends **active** traffic to a target, as opposed to every provider under `backend/app/providers/` , which only ever queries a third party's already-collected data. It was built under an explicit, non-negotiable constraint: this must never become an exploitation framework. Every architectural decision below is traceable to that constraint.

For the user-facing tool/profile/severity reference, see SECURITY_ASSESSMENT_TOOLKIT.md in the repo root `docs/` folder — this chapter covers the *implementation*, that one covers *behavior*.

1. Why Tools Are Not Providers

`app/providers/base.py` 's `BaseProvider` / `ProviderResult` / `ProviderCategory` / `ProviderStatus` contract was confirmed, by direct research before any of this was written, to be fully reusable: `app/ai/service.py` 's `_provider_result_to_prompt()` / `_prune_for_prompt()` / `generate_final_assessment()` are field-name-generic and need zero changes to consume a new category of data, provided it arrives as a genuine `ProviderResult` . So every tool in this package *produces* one. But tools are **not** `BaseProvider` subclasses, and are **not** registered in `app/providers/registry.py` 's `_ALL_PROVIDERS` :

- `BaseProvider.run()` is built around the automatic, always-on orchestrator fan-out (`app/providers/orchestrator.py`) — every registered provider runs on every investigation whose IOC type it supports, with no per-call authorization. A tool that sends real traffic to a target must never run that way.
- Tools are often materially slower than an API call (a real port scan, not a single HTTP request) and need their own timeout/subprocess-lifecycle handling (`nmap_tool.py` 's `asyncio.wait_for` / `proc.kill()`), which doesn't fit `BaseProvider` 's existing wrapper.

Instead, `app/security_assessment/base.py` 's `SecurityAssessmentTool` is a separate, minimal base class: `tool_id` , `tool_name` , `supported_types` , a `profiles: dict[str, ScanProfile]` , and one abstract `async def run(self, target, ioc_type, profile_id) -> ToolRunResult` . `ToolRunResult` bundles the genuine `ProviderResult` (for the AI/correlation pipeline) alongside a `list[Finding]` (for persistence and UI drill-down) — the two representations a tool's work needs to feed.

2. The Authorization Gate

`app/core/security_assessment.py::_validate_scope()` is the single choke point every request passes through before a `SecurityAssessmentRun` row is even created:

```
def _validate_scope(lookup, target_confirmation, authorization_confirmed) -> IOCType:
    if not authorization_confirmed:
        raise AuthorizationNotConfirmedError(...)
    if target_confirmation != lookup.ioc_value:
        raise TargetMismatchError(...)
    if lookup.ioc_type not in SCANNABLE_TYPES:
        raise UnscannableIOCTypeError(...)
    ioc_type = IOCType(lookup.ioc_type)
    if ioc_type == IOCType.CIDR:
        network = ipaddress.ip_network(lookup.ioc_value, strict=False)
        if network.num_addresses > _MAX_CIDR_ADDRESSES:  # 16, i.e. /28
            raise CIDRTooLargeError(...)
    return ioc_type
```

Order matters: authorization is checked *before* target matching, so an unconfirmed request gets a clear "you must confirm authorization" error rather than a possibly-true-or-false target-mismatch claim. Every branch here is a plain `ValueError` -style exception the route layer maps to `400` — none of it reaches a tool, a subprocess, or a network call. This is deliberately the *only* place scope is checked; there is no second, looser path into `app/security_assessment/registry.py` 's tools from anywhere else in the codebase.

3. Structured Process Execution (Nmap)

`nmap_tool.py` is the one tool that shells out to an external binary, and is held to the strictest standard: `PROFILES` / `_PROFILE_ARGS` are hardcoded Python dicts mapping a profile *id* (never accepted from a caller as raw text beyond that id) to a fixed `list[str]` of arguments. The subprocess is launched via:

```
argv = ["nmap", "-oX", "-", *_PROFILE_ARGS[profile_id], target]
proc = await asyncio.create_subprocess_exec(*argv, stdout=..., stderr=...)
```

`create_subprocess_exec` (never `create_subprocess_shell`) takes the argument list directly — there is no shell interpolation step where `target` (attacker- or accident-controlled string) could be parsed as anything other than one literal argv element. This is what "structured process execution" means concretely: the string `"; rm -rf /"` as a `target` value is passed to `nmap` as a single, literal, meaningless hostname argument, never concatenated into a shell command line.

Output is parsed via `xml.etree.ElementTree` against Nmap's own `-oX -` XML format, never by regex-scraping human-readable text output (which Nmap's own docs warn is unstable across versions).

4. Provenance Categories

`app/core/provenance.py` defines four category constants — `THREAT_INTEL` , `SECURITY_ASSESSMENT` , `LOCAL_OBSERVATION` , `AI_INTERPRETATION` — and one mapping function, `category_for_provider_category()` , used by both `app/correlation/engine.py` (tagging each `GraphEdge.provenance_category` at creation time) and `app/evidence/builder.py` (tagging each `EvidenceRecord.provenance_category`). Only the first two are populated by any code today; `LOCAL_OBSERVATION` / `AI_INTERPRETATION` are modeled and reserved for future use (e.g. an analyst's own manually-asserted annotation; an AI-inferred-but-unverified relationship), documented here rather than silently omitted.

This is a genuinely different axis from the pre-existing `CorrelationEdgeRecord.provenance` / `GraphEdge.provenance` string, which names *which* provider or tool asserted a fact (and, for corroborated facts, is a comma-joined list of them — see `engine.py` 's merge logic). Adding a second axis required two migrations (`5c8e1f3a9b2d` , `6d2f4b8e1a7c`), not an overload of the existing field:

`evidence/builder.py` and `ai/service.py` already parse `provenance` by splitting on `,` and treating each token as a provider id, so smuggling a category tag into that same string would have silently corrupted both call sites.

`engine.py`'s new `merge_correlation_results()` combines an already-persisted correlation (rebuilt via `app/evidence/loaders.py::correlation_from_records()`) with a freshly computed one from a security-assessment run's new results, for a single `generate_final_assessment()` call — a deliberate, disclosed simplification: it does not re-run `correlate()`'s own cross-provider dedup/corroboration-boost logic across the two sets, so an identical fact asserted by both an original provider and a new tool run would appear as two edges rather than one boosted-confidence edge. Judged unnecessary complexity for what is, in practice, a rare overlap.

5. Result Integration and the "Refresh" vs. "Reanalyze" Distinction

`app/api/routes/lookup.py::reanalyze_lookup()` (the existing "compare with a different AI backend" feature) always writes its new `FinalAssessmentRecord` with `is_primary=False` — it's an alternate *opinion* on the same evidence, never promoted. `app/core/security_assessment.py:: _refresh_lookup_assessment()`, called automatically after a run completes, does the opposite: it demotes the previous primary record (`UPDATE ... SET is_primary=False WHERE is_primary=True`) and inserts the new one as `is_primary=True`, updating `IOCLookup.final_verdict` / `risk_score` / `final_assessment` in place. The distinction is deliberate: new active-scan findings are genuine new evidence about the investigation, not merely a different model's take on unchanged evidence.

6. Background Execution and Task Lifetime

`start_run()` returns `{"run_id", "status": "pending"}` immediately and spawns `_execute_run()` via `_spawn_background()`, which adds the `asyncio.Task` to a module-level `_background_tasks: set` with a `done_callback` that discards it on completion — the standard workaround for `asyncio.create_task`'s own documented gotcha that a task with no other strong reference can be garbage-collected mid-flight. A test-only helper, `wait_for_background_runs()`, awaits every currently in-flight task; this was added after a real test-isolation bug surfaced during development, where a test polled `SecurityAssessmentRun.status` (which flips to `COMPLETED` *before* the post-completion AI-refresh step runs) and returned while that refresh step was still executing in the background, corrupting whichever test ran next by disposing the shared DB engine out from under it.

7. Cancellation

Cancellation was added after a forensic audit of the port-scanning pipeline found no way to stop a run once started — the run tracking in §6 only prevented Python from garbage-collecting an in-flight task; nothing let a caller reach in and stop one.

Run-id-keyed task tracking. `_spawn_background()` now takes the `run_id` as well as the coroutine, and populates a second module-level map, `_run_tasks: dict[uuid.UUID, asyncio.Task]`, alongside the existing GC-prevention set — the same `done_callback` clears both on completion:

```
def _spawn_background(coro, run_id: uuid.UUID) -> asyncio.Task:
    task = asyncio.create_task(coro)
    _background_tasks.add(task)
    _run_tasks[run_id] = task

    def _discard(t: asyncio.Task) -> None:
        _background_tasks.discard(t)
        if _run_tasks.get(run_id) is t:
            del _run_tasks[run_id]

    task.add_done_callback(_discard)
    return task
```

`cancel_run(run_id, actor_user_id, actor_email)` is the new entry point (called from `POST /api/v1/security-assessment/runs/{run_id}/cancel`, gated by the same `security_assessment:create` permission as starting a run — this is a shared-team resource, per §5's existing no-per-user-ownership model, so any analyst/admin can cancel any run, not only their own). It loads the run, rejects with `RunNotCancellableError` if it's already `COMPLETED` / `FAILED` / `CANCELLED`, and then branches on whether this *process* has a live task for it:

- If `_run_tasks` has a live entry, it calls `task.cancel()` — this raises `asyncio.CancelledError` inside `_execute_run()` at its next `await` point, which is handled as described below.
- If not — the run exists and is `PENDING` / `RUNNING` in the database, but this process has no task for it, which is exactly what happens after a backend restart — it writes the `CANCELLED` status directly. Without this branch, a run orphaned by a restart could never be cancelled at all, since there would be no live task to `.cancel()`.

Exactly one audit record per cancellation, even under a real race. `cancel_run()`'s initial status check and its `task.cancel()` call are not atomic, so two concurrent cancel requests for the same run can both read `RUNNING` and both proceed — confirmed live via a genuine concurrent-request test. This is harmless on the `asyncio.Task` side (`.cancel()` on an already-cancelling task is a no-op), but the live-task branch deliberately does **not** write its own `record_audit()` call for this reason: only `_execute_run()`'s `except asyncio.CancelledError` handler does, and that handler runs exactly once per task no matter how many times `.cancel()` was called on it, making it the single source of truth. The direct-DB-write branch (no live task, e.g. post-restart) is the only place that *does* write its own audit record — nothing else will ever run for that run — and even there the `UPDATE ... WHERE status IN (...)` is checked via `result.rowcount` before writing, so two concurrent requests hitting that branch for the same run still produce only one audit entry, not two. Verified live, before and after, by querying `config_audit_log` directly during a real concurrent-cancel race.

Why a dedicated `except asyncio.CancelledError` branch is mandatory, not stylistic. Since Python 3.8, `asyncio.CancelledError` inherits from `BaseException`, not `Exception` — the pre-existing generic `except Exception as exc:` branch in `_execute_run()` silently does not catch it. Without an explicit branch, a cancelled task's DB row would simply stay `RUNNING` forever (the task dies, but no code ever runs to update the row), which is worse than doing nothing: a stuck `RUNNING` row with no way to distinguish it from a genuinely slow scan. The fix adds a branch *before* the generic `except Exception`, which updates the row to `CANCELLED`, records the audit event `security_assessment.run_cancelled`, and then re-`raise`s — re-raising matters so the `Task` object's own `.cancelled()` bookkeeping inside asyncio stays accurate, rather than being silently swallowed into a normal return.

Subprocess cleanup. The same `BaseException` distinction applies one layer down, inside `nmap_tool.py`: the existing `await asyncio.wait_for(proc.communicate(), timeout=...)` already had an `except asyncio.TimeoutError` branch to kill a hung process, but a *cancelled* run reaches the same `await` point via `CancelledError`, not `TimeoutError` — a second, explicit `except asyncio.CancelledError: proc.kill(); await proc.wait(); raise` branch was added alongside it. Without this, cancelling a run mid-scan would stop the Python-side bookkeeping but leave the real `nmap` OS process running untouched — an orphaned subprocess that would keep consuming CPU/network resources and, if it later happened to write to a since-closed pipe, could error unpredictably.

Testing. `test_security_assessment_api.py` adds a `_SlowFakeTool` (an `await asyncio.sleep(30)` stand-in) specifically so the in-flight-cancellation test (`test_cancelling_an_in_flight_run_kills_it_cleanly`) exercises a genuine mid-flight `task.cancel()` — the pre-existing `_FakeTool` resolves before a cancel request could ever reach it, which would only prove the already-completed-run rejection path, not real cancellation. A second test constructs a `SecurityAssessmentRun` row directly at `RUNNING` status with no corresponding task in `_run_tasks`, simulating exactly what a post-restart process sees, and confirms it still resolves to `CANCELLED` via the direct-DB-write branch above. Beyond the mocked suite, this was also verified

against a real, live `nmap` "standard" profile scan through the actual HTTP API in a running Docker container: cancellation completed in ~1.6s (against a natural completion time of ~12s for that profile/target), confirmed via manual `/proc/[0-9]*/cmdline` inspection (the container's minimal `python:3.12-slim` image has no `ps` binary) that no orphaned `nmap` process remained.

A schema lesson surfaced while adding the `CANCELLED` enum value. The migration (`8f4a1c2d9e6b_add_cancelled_security_assessment_status.py`) originally added the value as lowercase `'cancelled'`, matching `SecurityAssessmentRunStatus.CANCELLED`'s Python `.value`. Two tests then failed against a real Postgres container with `invalid input value for enum securityassessmentrunstatus: "CANCELLED"`. Querying `pg_enum` directly showed the four pre-existing labels are `PENDING` / `RUNNING` / `COMPLETED` / `FAILED` — uppercase, matching the Python enum members' *names*, not their lowercase `.value` strings — because a plain `sa.Enum(SomeEnum)` column with no `values_callable` override serializes `.name` on the wire, not `.value`. The migration was corrected to `ADD VALUE IF NOT EXISTS 'CANCELLED'` (uppercase); since Postgres has no `ALTER TYPE ... DROP VALUE`, the throwaway test database had to be torn down and recreated rather than patched in place. The Python-side `.value = "cancelled"` string is unaffected and correct as-is — it's what `_serialize_run()` returns to the API/frontend; it was never what gets sent to Postgres.

8. Bugs Found by Independent Re-verification (v0.2.2)

An independent re-verification pass (nine parallel reviewers, each reading the real code fresh rather than trusting §7's own claims) re-proved the cancellation fix correct, then found four new, real defects — all fixed and covered by new tests.

IPv6 scans never actually ran. `nmap` requires a literal `-6` flag for an IPv6 target specification; without it, nmap logs `"<target> looks like an IPv6 target specification -- you have to use the -6 option"` to stderr and exits `0` having scanned 0 hosts. Since `run()`'s only failure check is `proc.returncode != 0`, this was indistinguishable from, and reported identically to, a genuine clean scan (`completed`, 0 findings, `error_message: null`) — despite `supported_types` having always included `IOCType.IPV6`. Fixed: `_PROFILE_ARGS`'s argv now gets `-6` prepended whenever `ioc_type == IOCType.IPV6`. Live-reproduced: manually replaying the exact quick-profile command against `::1` with and without `-6` confirmed the root cause and the fix (with `-6`, nmap correctly reports the host up with all common ports closed — a real, current answer, not a skipped scan).

Profile id was never validated before spawning a run. `start_run()` validated `tool_id` (existence and IOC-type support) up front but never checked `profile_id` against the selected tool's own `profiles` dict. The only check lived deep inside each tool's `run()` (e.g. `nmap_tool.py`: `if profile_id not in _PROFILE_ARGS: return self._error(...)`), which returns an ordinary `ProviderStatus.ERROR` result rather than raising — `_execute_run()` doesn't treat that as exceptional, so the run reached `COMPLETED` with `error_message: null` and empty findings, exactly the fake-clean-result failure mode this module's own design principles rule out. Fixed with a new `UnknownProfileError`, raised in `start_run()`'s existing per-tool validation loop (`if profile_id not in tool.profiles: raise UnknownProfileError(...)`), mapped to a `400` in the route layer alongside the existing `UnknownToolError`. This check is tool-agnostic — it applies uniformly to every tool in `tool_ids`, not just Nmap, since the underlying gap was in the shared service-layer validation, not any one tool adapter.

A malformed CIDR lookup value crashed the endpoint. `_validate_scope()`'s `ipaddress.ip_network(lookup.ioc_value, strict=False)` call for `IOCType.CIDR` had no error handling; a lookup reaching this code with a value that isn't a valid network string (only reachable via the pre-existing, unrelated lookup-creation `ioc_type_hint` override, which doesn't itself validate value-matches-hint) raised an uncaught `ValueError`, surfacing as a raw `500` — the only validation failure in this function that didn't produce a clean `400`. Fixed with a new `InvalidTargetError`, raised from a `try/except ValueError` around the `ip_network()` call, mapped to `400` in the route layer. Not an injection/RCE vector either way — the crash happens before any run row exists and before Nmap is ever invoked.

Completion could clobber another writer's terminal state. `_execute_run()`'s three terminal UPDATEs (CANCELLED/FAILED/COMPLETED) had no `WHERE status IN (...)` guard, unlike `cancel_run()`'s own direct-DB-write fallback branch (§7), which already had one. Confirmed live via a genuine race: the startup orphan-recovery sweep (`_recover_orphaned_running_lookups()`) marked a run FAILED while its task was still genuinely executing (triggered out-of-band by a concurrent test process calling that same recovery function against the same live database — not a normal in-process restart); when the task's own success path ran moments later, its unguarded UPDATE unconditionally overwrote the row back to COMPLETED, but never touched the (COMPLETED-path UPDATE doesn't set) `error_message` column, leaving the stale FAILED-only message attached to a COMPLETED row — a self-contradictory result. Fixed: all three terminal UPDATEs now carry the same `.where(status.in_([PENDING, RUNNING]))` guard `cancel_run()` already used, each gated on `result.rowcount` before writing its own audit record. Findings/`ProviderResultRecord`s are still always persisted regardless of the guard's outcome — only the run's own `status` / `completed_at` write is guarded, since real scan data stays valid evidence even if an unrelated writer already finalized the row.

Also fixed in the same pass: an empty `tool_ids` array is now rejected at the Pydantic layer (`Field(min_length=1)`) instead of silently producing a no-op `COMPLETED` run.

9. Installer / Deployment

`backend/Dockerfile` installs `nmap` via `apt-get` alongside the pre-existing `gcc libpq-dev curl`. `GET /api/v1/security-assessment/tool-health` calls `shutil.which("nmap")` at request time rather than assuming installation succeeded — a deployment that somehow lacks the binary degrades to "unavailable" in the UI rather than a scan silently failing partway through. `backend/requirements.txt` gained one new dependency, `dnspython`, for the DNS enumeration tool (stdlib alone does not expose MX/TXT/NS/CAA record types). No `k8s/` manifest change was needed — the k8s backend Deployment already builds from this same Dockerfile.

Pentest Suite Architecture

This chapter documents the Pentest Suite (`backend/app/pentest/`, `backend/app/api/routes/pentest.py`, `backend/app/api/routes/pentest_exploit.py`) — a standalone, scope-enforced assessment lifecycle built as a *second* orchestration layer over the same tool adapters `backend/app/security_assessment/` already provides, plus one further capability neither that toolkit nor anything else in this platform has: real, gated Metasploit exploit-module execution. For the user-facing reference, see PENTEST_SUITE.md — this chapter covers the *implementation*, that one covers *behavior*.

1. Why a Second Orchestration Layer, Not a Fork

`SecurityAssessmentTool.run(target, ioc_type, profile_id) -> ToolRunResult` (`app/security_assessment/base.py`) is already fully decoupled from `IOCLookup` — it takes a plain string target and an `IOCType`, nothing lookup-specific. The one thing tightly coupled to a single investigation is `app/core/security_assessment.py`'s own service layer, not the tools themselves. So `app/pentest/orchestrator.py` targets a new, standalone `PentestTarget` model and calls `app/security_assessment/registry.py::get_tool()` directly — the exact same tool instances, the exact same `ToolRunResult` contract, mapped into `PentestFinding` rows instead of `SecurityAssessmentFinding` rows. Nothing in `app/security_assessment/` or `app/core/security_assessment.py` was touched.

2. Data Model

Three tables reuse `UUIDPrimaryKeyMixin` / `TimestampMixin` (`app/models/base.py`), the same cascade-delete convention `Case` established:

- **`PentestAssessment`** — `status` (`DRAFT/ACTIVE/PAUSED/COMPLETED/CANCELLED/ EXPIRED`), `profile` (`PASSIVE/LOW_IMPACT/STANDARD/COMPREHENSIVE/CUSTOM`), `scope_definition` JSONB (`{"cidrs": [...], "domains": [...]}` — a handful of entries per assessment, JSONB is sufficient at this scale), `max_runtime_minutes`, `max_requests`, `emergency_stopped` bool, `created_by` FK.
- **`PentestTarget`** — reuses the platform's own `IOCType` values as a plain string column (`ipv4/ipv6/cidr/domain/hostname/url`) rather than a parallel enum, since the orchestrator hands this straight to the same `SecurityAssessmentTool.run()` every per-lookup scan already uses.
- **`PentestFinding`** — reuses the existing `Severity` vocabulary (string column) and adds a new `PentestFindingConfidence` enum (`CONFIRMED/LIKELY/POTENTIAL/ INFORMATIONAL`) distinct from severity. `evidence` is a JSONB **list** of `{captured_at, tool_id, data}` entries, not a single blob, so a later intrusive-validation re-check appends rather than overwrites the original observation.
- **`PentestExploitAttempt`** (added for the exploit-validation feature, §6) — `module_fullname`, `module_options` (post-lock, i.e. what was *actually* sent, not what a caller supplied), `mode` (`CHECK/EXPLOIT`), `status` (`PENDING/RUNNING/ SUCCEEDED/SESSION_OPENED/FAILED/ERROR`), `result_transcript` (verbatim, capped at 20,000 chars), `session_id` nullable int, `requested_by` FK.

Two migrations: `9273d7b21c79` (assessments/targets/findings, `down_revision = '8f4a1c2d9e6b'`) and `9123b075e962` (exploit attempts, chained onto the first).

3. Orchestrator: Concurrency Discipline Reused Verbatim

`app/pentest/orchestrator.py`'s own docstring is explicit that its concurrency handling is copied from the Security Assessment Toolkit's own hard-won fixes (see `backend-09-security-assessment-toolkit.md` §7), not reinvented: every terminal-state write is a conditional `UPDATE ... WHERE status.in_([...])`, and exactly one path writes a given audit event — the live in-process task when tracked (`_assessment_tasks: dict[uuid.UUID, asyncio.Task]`), falling back to a rowcount-gated direct write only when no live task is tracked (e.g. after a backend restart). `cancel_assessment()` branches on exactly this:

```
async def cancel_assessment(assessment_id, actor_user_id, actor_email) -> dict:
    task = _assessment_tasks.get(assessment_id)
    if task is not None and not task.done():
        task.cancel()  # _run_assessment's own except-branch writes status + audit exactly once
    else:
        # No live task -- this process didn't start it (e.g. post-restart).
        # Rowcount-gated so a genuine race still produces one audit record, not two.
        ...
```

4. Scope Enforcement (`_is_in_scope`)

```
def _is_in_scope(scope_definition: dict, target_type: str, value: str) -> bool:
    cidrs = scope_definition.get("cidrs") or []
    domains = scope_definition.get("domains") or []
    if target_type in (IOCType.IPV4.value, IOCType.IPV6.value, IOCType.CIDR.value):
        candidate = ipaddress.ip_network(value, strict=False)  # ValueError -> False
        return any(candidate.subnet_of(ipaddress.ip_network(c, strict=False)) for c in cidrs)
    if target_type in (IOCType.DOMAIN.value, IOCType.HOSTNAME.value, IOCType.URL.value):
        host = value.split("://", 1)[-1].split("/", 1)[0].split(":", 1)[0].lower()
        return any(host == d or host.endswith("." + d) for d in (dom.lower().lstrip("*.") for dom in domains))
    return False
```

An empty `scope_definition` returns `False` for everything — a brand-new assessment with no declared scope is never treated as wide-open. `add_target()` calls this *before* creating the row, and rejects with an audit record (`pentest.target_rejected_out_of_scope`) on failure. There is exactly one scope-check implementation; `app/pentest/exploit.py` imports this same private function rather than duplicating the logic, so scope semantics can never drift between the two modules.

5. Confidence Heuristic and Intrusive Validation

```
def _confidence_for_finding(finding) -> PentestFindingConfidence:
    if finding.cve_ids:
        return PentestFindingConfidence.LIKELY   # never auto-CONFIRMED from a CVE match alone
    if finding.tool_id == "hash_analysis":
        return PentestFindingConfidence.INFORMATIONAL
    return PentestFindingConfidence.POTENTIAL
```

`INTRUSIVE_VALIDATION_CHECKS` is a small, hardcoded dict (`recheck_service_banner` , `recheck_tls_config`) — `validate_finding()` re-runs the *same* tool against the *same* target and promotes to `CONFIRMED` only if the identical `finding_type` reappears. No exploit-payload execution exists anywhere in `orchestrator.py` , nor is that module ever imported by `exploit.py` 's separate feature — the autonomous `DISCOVER→ENUMERATE→ASSESS` pipeline can never reach real exploitation.

6. Exploit Validation: `msf_client.py` and `exploit.py`

The msgpack "raw string" bug

The single most consequential bug found while building this feature: Metasploit's Ruby-side msgpack encoder emits **old-spec "raw" strings**, not the newer str8/16/32 formats. Python's `msgpack.unpackb(..., raw=False)` — the *correct*, modern setting — only affects how the new string formats decode; a "raw"-encoded string still comes back as `bytes` regardless. Every field this client reads (`auth.login` 's own `"token"` key, `console.read` 's `"data"` / `"busy"` keys, etc.) was silently `bytes` instead of `str` , so `result.get("token")` never matched anything and `is_available()` permanently reported Metasploit as unreachable — even with `msfrpcd` running correctly. Found by manually calling `_raw_call()` and printing the raw decoded dict during live verification against a real `msfrpcd` , not by static reading. Fixed with a recursive `_decode_bytes()` normalizer applied to every response before it's returned:

```
def _decode_bytes(value):
    if isinstance(value, bytes):
        try:
            return value.decode("utf-8")
        except UnicodeDecodeError:
            return value
    if isinstance(value, dict):
        return {_decode_bytes(k): _decode_bytes(v) for k, v in value.items()}
    if isinstance(value, list):
        return [_decode_bytes(v) for v in value]
    return value
```

Covered by `test_msf_client.py` as a permanent regression test.

Console-driven execution, not raw `module.execute`

Rather than the RPC `module.execute` job/polling API (which has no clean way to retrieve a scanner module's human-readable output without a configured Metasploit database — deliberately not set up in this deployment), `exploit.py` drives a real `msfconsole` session via the RPC `console.*` methods: `console.create` → `console.write("use <module>\n")` → one `console.write("set KEY value\n")` per option → `console.write("check\n")` or `console.write("run\n")` → poll `console.read` until idle → `console.destroy` . This is the same interaction a human typing into `msfconsole` performs, and it means the transcript stored as evidence is the genuine, complete console output — including Metasploit's own banner, `check` 's real Vulnerable/Not-Vulnerable/Cannot-reliably-check response, and any error output — never a synthesized summary.

RHOSTS lock, ordering matters

```
def _lock_rhosts(options: dict, target_value: str) -> dict:
    locked = {k: v for k, v in (options or {}).items() if k.lower() not in {"rhost", "rhosts"}}
    locked["RHOSTS"] = target_value   # inserted LAST -- dict insertion order guarantees this
    return locked
```

Because Python dicts preserve insertion order and the real `RHOSTS` is always the last key inserted, it is also always the **last** `set RHOSTS ...` line written to the console — confirmed, during adversarial review, to hold even against a homoglyph or whitespace-padded key smuggled past the case-insensitive filter, since any such key survives filtering but still gets overridden by the guaranteed-last real assignment.

Console-injection guard

```
def _reject_console_injection(options: dict) -> None:
    for key, value in options.items():
        if any(ch in str(key) for ch in ("\n", "\r")) or any(ch in str(value) for ch in ("\n", "\r")):
            raise PentestError(f"Option {key!r} contains a newline/carriage-return character...")
```

Applied to the already-locked options (so it also covers the forced `RHOSTS` value), and the module fullname is separately validated against `^[A-Za-z0-9_./-]+$` before ever appearing in a `use {module_fullname}\n` line. Without this, a crafted option value containing an embedded newline could inject an unintended additional console command — found and fixed proactively (not by the later adversarial review) while reasoning through the console-write loop's line-by-line behavior.

Defense in depth found by adversarial review

An independent adversarial-review pass (reading the finished code fresh, trying concretely to break each stated guarantee) confirmed the RHOSTS lock, the confirmation gate, and the scope re-check all hold, and found one real, confirmed issue plus several hardening opportunities, all fixed in the same pass:

- **Confirmed confidentiality bug:** `GET /pentest/assessments/{id}/exploit-attempts` was gated on `pentest:read` — a permission `ANALYST` / `VIEWER` also hold — even though the list includes real post-exploitation transcripts (potentially containing dumped credentials/session output) that only `ADMIN` can ever produce. Fixed: re-gated on `pentest:exploit`.
- **Hardening:** `run_module()` now also re-checks `actor_role == Role.ADMIN` itself, not only via the route's `require_permission` dependency — redundant on every call path that exists today, kept anyway so this specific function is never one dependency-injection change away from becoming reachable by a lower role.
- **Hardening:** the global kill switch is now re-checked on every console poll tick (about once per second) during a running check/exploit, not only before it starts — aborts an in-flight run within roughly a second instead of only blocking the next one. The abort path is tracked via a return value from `_drive_console()` (`(transcript, aborted)`), not a re-read of the global switch after the fact, since the switch could have been re-disengaged by the time the caller checked again.
- **Hardening:** `_load_context()` now asserts the loaded target's `assessment_id` matches the finding's own `assessment_id` — not reachable via any existing code path, but cheap insurance given the stakes.

7. API Surface

`app/api/routes/pentest.py` (assessment CRUD, scope, targets, start/pause/resume/ cancel/emergency-stop, findings, AI summary, intrusive validation) and `app/api/routes/pentest_exploit.py` (module search/options, run, list past attempts, stop session) are deliberately separate files — the higher-risk surface is easy to find, review, or remove in isolation. `app/pentest/ai_analyst.py` mirrors `app/ai/service.py` 's own "AI narrates a fact, never computes it" discipline: `explain_finding()` / `generate_executive_summary()` are grounded strictly in real, already-persisted `PentestFinding` rows, and `generate_executive_summary()` 's `top_priorities` list is filtered to only real finding titles the AI was actually given — confirmed live to correctly drop an AI-invented title that didn't match any real finding.

8. Installer / Deployment

`backend/Dockerfile` installs the real Metasploit Framework via Rapid7's official omnibus installer (`msfinstall`), requiring `gnupg` / `apt-transport-https` / `lsb-release` in addition to the pre-existing `gcc libpq-dev curl nmap ca-certificates` — the omnibus installer's apt-key fetch step silently produced an unusable keyring without `gnupg` present, found during the first real build attempt (apt reported the repo as unsigned; `apt-get install metasploit-framework` then failed with "Unable to locate package"). The module cache is pre-built at image-build time (`msfconsole -qx 'exit'` , non-fatal if it exits non-zero) so a fresh container's first boot isn't mistaken for a hang. `docker-compose.yml` 's backend `command:` starts `msfrpcd -f -P "$MSF_RPC_PASSWORD" -S -a 127.0.0.1 -p 55553` before `alembic upgrade head && uvicorn` ; `-S` disables TLS (acceptable since the daemon is bound to loopback, reachable from nowhere but this same container's own backend process) and `-f` keeps it in the foreground of its own backgrounded shell job so its lifetime is tied to the container. `backend/requirements.txt` gained one new dependency, `msgpack` , for the hand-rolled RPC client. Backend `mem_limit` raised `2g → 4g` for the added Ruby runtime + module cache + RPC daemon footprint; the healthcheck's `start_period` raised `30s → 60s` for the same reason.

9. Testing

`test_pentest_orchestrator.py` (scope enforcement, confidence heuristic) and `test_pentest_api.py` (RBAC, scope-bypass rejection, full lifecycle with a fake tool, emergency stop, kill switch) mirror the Security Assessment Toolkit's own test structure. `test_msf_client.py` regression-tests the bytes-decoding fix directly. `test_pentest_exploit.py` and `test_pentest_exploit_api.py` cover the RHOSTS lock, console-injection guard, module-fullname validation, the mandatory `confirmed=true` gate, the redundant role check, scope re-verification at execution time, the kill-switch mid-run abort, and — as regression tests for the adversarial review's own finding — that `ANALYST` cannot list exploit attempts while `ADMIN` can. All fake-client-based; this suite never talks to a real `msfrpcd` . Separately, the real, live client (module search, module options, and a real `check` run against a loopback target with no listening service, honestly reporting "cannot reliably check exploitability") was verified against an actual running `msfrpcd` before release, not only against mocks.